言語処理系

Language Processing System

- ミドルパスにおける最適化
  - データ依存解析
  - ループ・リストラクチャリング
  - マルチプロセッサ用自動並列化

早稲田大学理工学術院
情報理工学科
笠原博徳

2026年7月10配布

TOP500 The List. HIGHLIGHTS - JUNE 2026

https://top500.org/lists/top500/2026/06/highs/

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	National Supercomputing Centre in Shenzhen (NSCS) China	**LineShine** - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS Shenzhen Cloud Computing Center Co., Ltd.	13,789,440	2,198.40	2,735.82	42,220
2	DOE/NNSA/LLNL United States	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS HPE	11,340,000	1,809.00	2,821.10	29,685
3	DOE/SC/Oak Ridge National Laboratory United States	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS HPE	9,066,176	1,353.00	2,055.72	24,607
4	DOE/SC/Argonne National Laboratory United States	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11 Intel	9,264,128	1,012.00	1,980.01	38,698
5	EuroHPC/FZJ Germany	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux Bull	4,801,344	1,000.00	1,226.28	15,794
6	Eni S.p.A. Italy	**HPC7** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, RHEL 9 HPE	3,461,472	571.50	861.13	8,735
7	Microsoft Azure United States	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR Microsoft Azure	2,073,600	561.20	846.84	
8	Eni S.p.A. Italy	**HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9 HPE	3,143,520	477.90	606.97	8,461
9	RIKEN Center for Computational Science Japan	**Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D Fujitsu	7,630,848	442.01	537.21	29,899
10	Swiss National Supercomputing Centre (CSCS) Switzerland	**Alps** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS HPE	2,121,600	434.90	574.84	7,124

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	National Supercomputing Centre in Shenzhen (NSCS) China	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS Shenzhen Cloud Computing Center Co., Ltd.	13,789,440	2,198.40	2,735.82	42,220

https://www.nsccsz.cn/nsccsz2024/zxdt2024/202606/19c3a18b-f9f5-4ba7-a557-9eb890f73615.shtml

Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
2	DOE/NNSA/LLNL United States	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS HPE	11,340,000	1,809.00	2,821.10	29,685

https://www.hpe.com/lamerica/en/compute/hpc/lawrence-livermore-national-laboratory.html

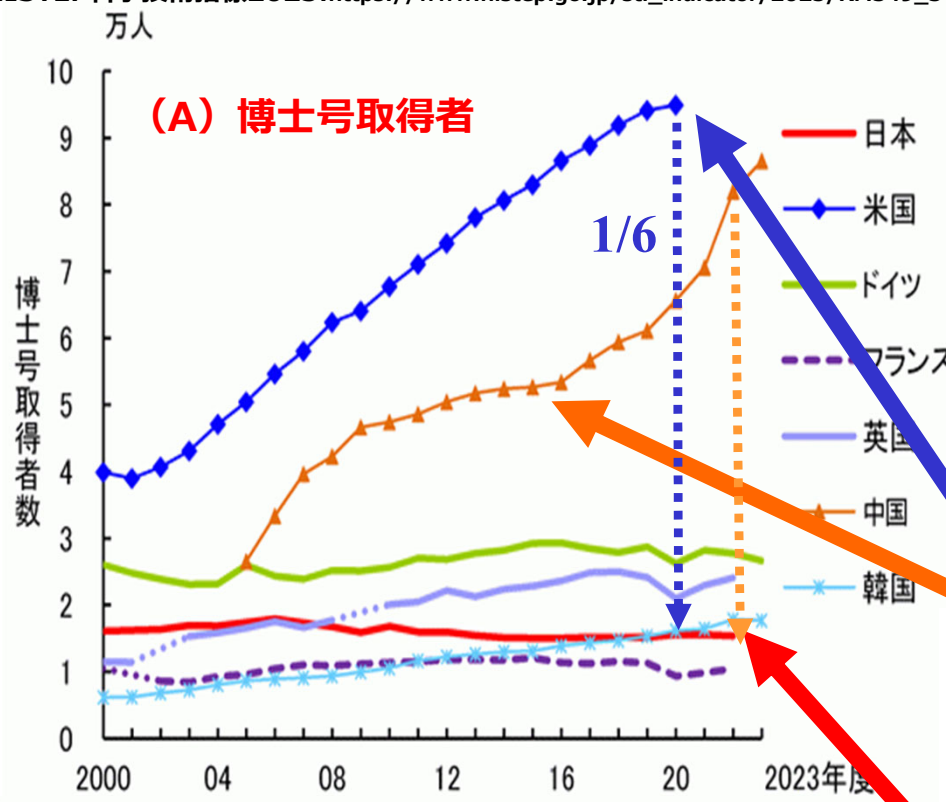
Rank	Site	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
3	DOE/SC/Oak Ridge National Laboratory United States	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS HPE	9,066,176	1,353.00	2,055.72	24,607

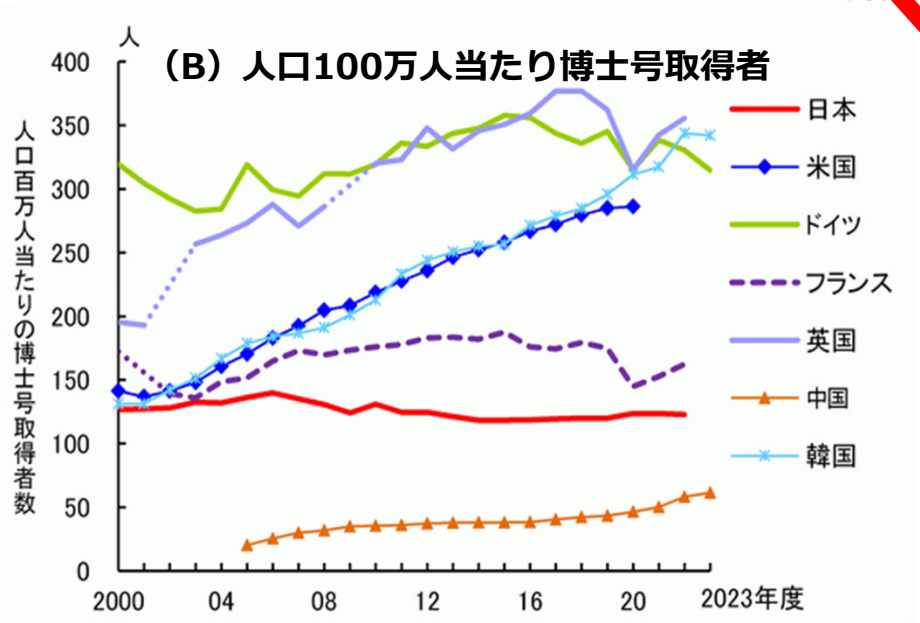
https://www.olcf.ornl.gov/frontier/#2

主要国の博士取得者数とAI特許出願数の関係

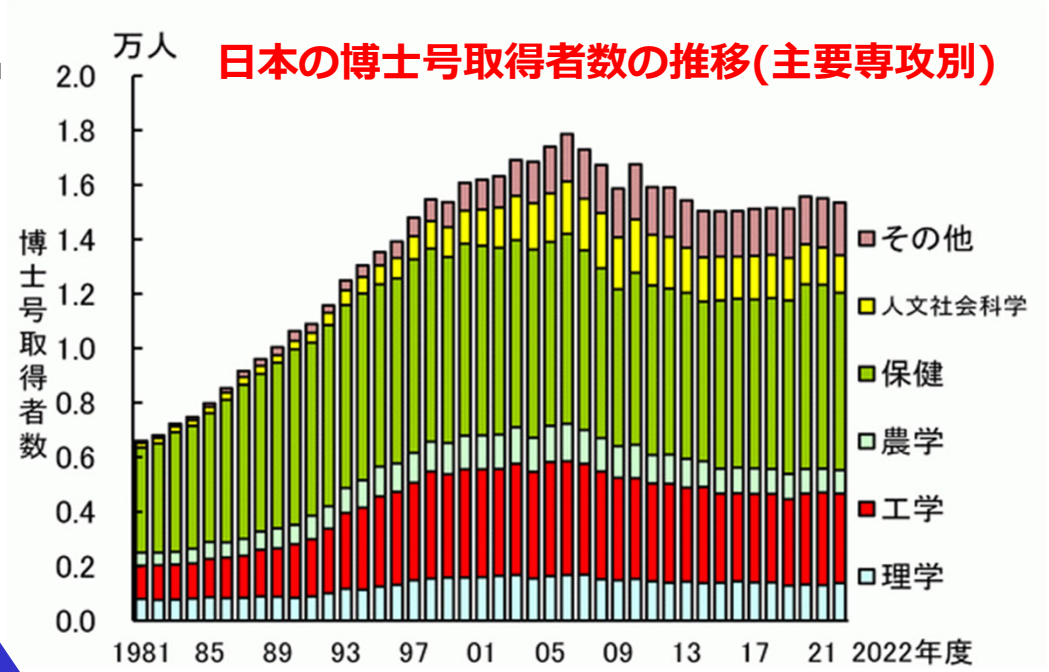
主要国の博士号取得者数の推移（A)(B)

NISTEP科学技術指標2025:https://www.nistep.go.jp/sti_indicator/2025/RM349_34.html





日本の博士号取得者数の推移(主要専攻別)



https://www.jpo.go.jp/e/system/patent/gaiyo/ai/ai_shutsugan_chosa.html

Recent Trends in AI-related Inventions

March 20 Examin Research Office, Patent Examination Department (Electronic Technology), Japan Patent Office

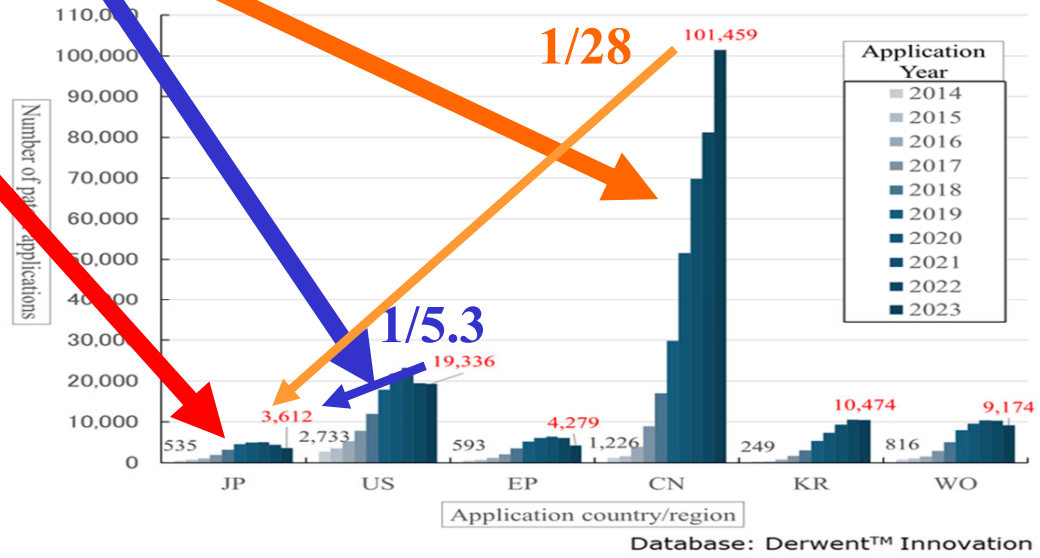


Figure 4. Trend in Patent Applications for AI Core Inventions by Country/Region

(Figures for 2014 Are Shown in Black, and Figures for 2023 Are Shown in Red)

Notes: Country codes shown in the figure indicate the following: JP: Japan, US: United States, EP: Europe (EPO), CN: China, KR: South Korea, WO: PCT international applications (regardless of applicant nationality).

データ依存解析 Data Dependence Analysis (DA)

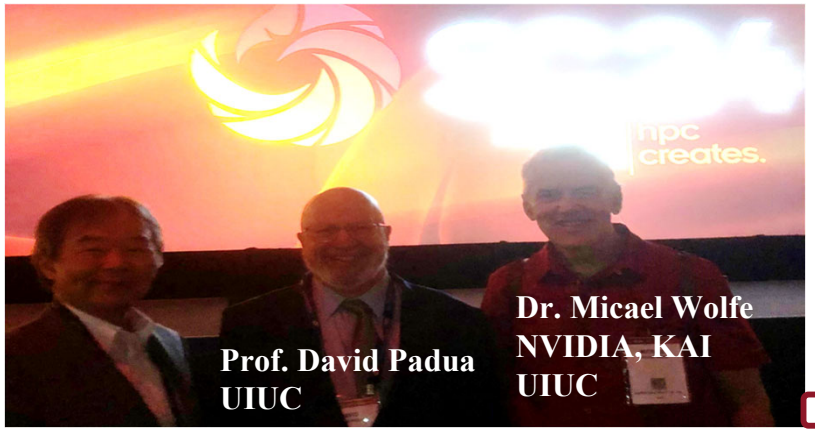
並列性解析:フロー依存、出力依存、逆依存

For parallelization: Flow, Output, Anti dependences

データの分割配置:入力依存

For data locality Optimization: Input dependence

- シンボリックデータ依存解析: **Symbolic DA**
- セマンティックデータ依存解析 **Semantic DA**
- イタレーションスペースデータ依存解析**Iteration space DA**
- 依存方向ベクタ解析 **Dependence vector DA**
- GCD (Greatest Common Devisors)テスト **GCD test**
- **Banerjee's exact test**
- **Banerjee-Wolfe's inexact test**
- **Pugh's OMEGA test**



Prof. David Kuck

Univ. Illinois at Urbana-Champaign

Prof. Emeritus

Founder of Kuck & Associates

- ILLIAC IV (Member)
- Parallelizing Compiler
  - Data dependence analysis
  - Automatic vectorization
  - Automatic loop parallelization
  - Loop restructuring
  - Cedar supercomputer (Alliant FX8)
  - OpenMP

Intel Senior Fellow (Past CTO):

- Intel Parallel Tuning Tools & Debuggers

- National Academy of Engineering Member
- IEEE ACM Eckert-Mauchly Award
- IEEE Charles Babbage Award
- ACM/IEEE Ken Kennedy Award
- Okawa Prize
- IEEE Computer Pioneer Award
- IEEE Fran Allen Medal

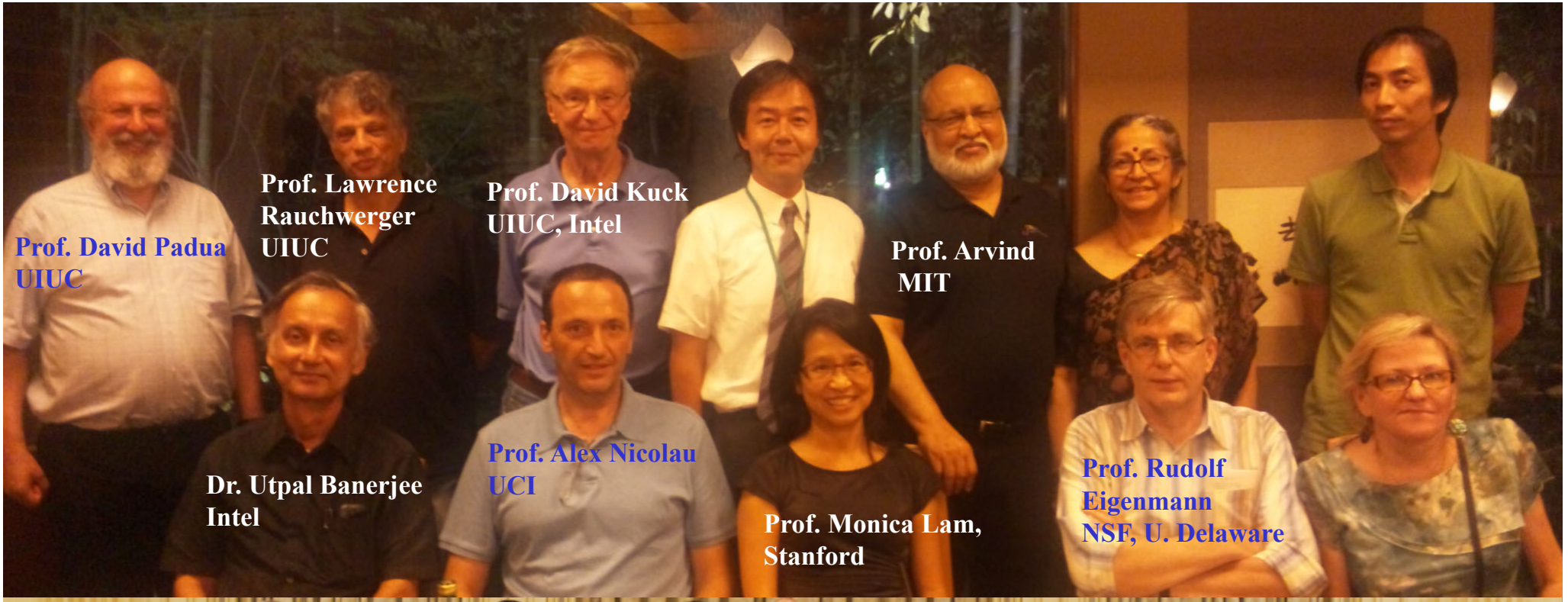
Prof. David J Kuck (Univ. Illinois, Intel)
IEEE Computer Pioneer Award 2011
Ms. Diane B. Greene (VMware Cofounder & CEO)
IEEE Computer Society Computer Entrepreneur Award 2011

2018.03.08 WasedaUniv. Symposium on Future of High Performance Green Computing 2018 (HPGC2018)

46

世界の並列化コンパイラ・トップ研究者

（LCPC 2012, 25th Languages and Compilers for Parallel Computing,
早稲田大学グリーンコンピューティングセンター開催時））



プログラム・リストラクチャリング
Program Restructuring

ベクトル化あるいはループ並列化できないループの自動変換

Automatic code change for non-parallelizable or –vectorizable loops

(a)ステートメントの実行順序の変更: Change of st atement execution order

(b)ループディストリビューション: Loop distribution

(c)ノードスプリッティング、スカラエクスパンション:

Node splitting, Scaler expansion

(d)ループインターチェンジ、パーミュテーション:

Loop Inter-change, Permutation

(e)ループアンローリング: Loop unrolling

(f)ストリップマイニング: Strip-mining

(g)アレイプライベタイゼーション: Array privatization

(h)ユニモジュラー変換: Uni-modular transform: **Prof. Monica Lam, Stanford**

（ループリバーサル、パーミュテーション、スキューイング Loop reversal, Permutation, Skewing by Affine Transformation）, **SUIF Compiler: US National Compiler Infrastructure)**

SC25
ST. LOUIS NOV 16–21

PROGRAM EXHIBITS STUDENTS SCINET MEDIA ATTEND

https://sc25.supercomputing.org/program/awards/

SIDNEY FERNBACH MEMORIAL AWARD

Presented at SC 2025

EWA DEELMAN

Sidney Fernbach Award

The Sidney Fernbach Memorial Award is presented annually to an individual for an outstanding contribution in the application of high-performance computers using innovative approaches.

Ewa Deelman, University of Southern California Information Sciences Institute (ISI)

SEYMOUR CRAY COMPUTER ENGINEERING AWARD

Presented at SC 2025

JOHN M. SHALF

Seymour Cray Computer Engineering Award

The IEEE-CS Seymour Cray Computer Engineering Award recognizes innovative contributions to high performance computing systems that best exemplify the creative spirit demonstrated by Seymour Cray. Recipients may be recognized for technologies that have contributed to working machine deployed at scale, which has had long-term impact valued by the HPC community.

John Shalf, Lawrence Berkeley National Laboratory (LBNL)

KEN KENNEDY AWARD

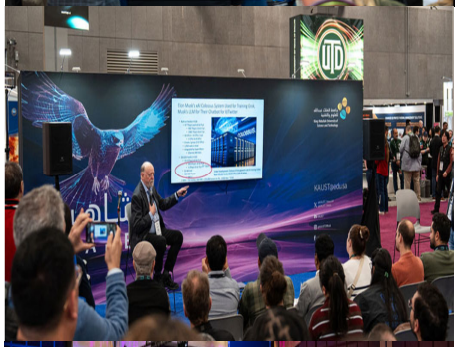
SAMAN AMARASINGHE

Ken Kennedy Award

The ACM/IEEE-CS Ken Kennedy Award recognizes substantial contributions to programmability and productivity in computing and substantial community service or mentoring contributions.

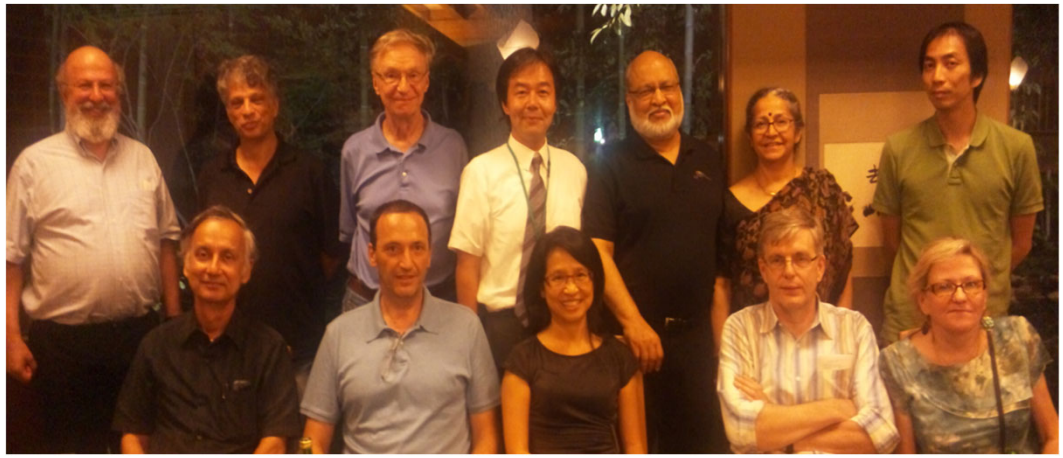
Saman Amarasinghe, Massachusetts Institute of Technology (MIT)

SUIF Compiler, PhD @ Stanford, Monica Lam's Group



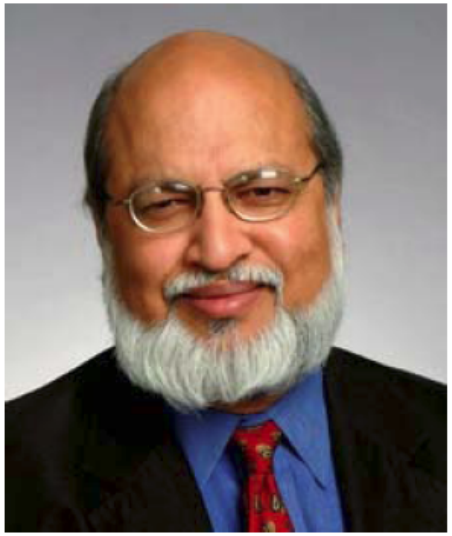
https://scconferencephotos.pixieset.com/sc25stlouisconferencephotos/

ダイナミック・データフローマシンのパイオニア　Prof. Arvind, MIT



MIT Professor Arvind Wins IEEE Computer Society Goode Award

LOS ALAMITOS, Calif., 18 April, 2012 – Massachusetts Institute of Technology Professor Arvind, a world-renowned leader in computer languages for parallel processing, has been named the winner of the IEEE Computer Society's 2012 Harry H. Goode Award.



Arvind, MIT's Johnson Professor of Computer Science and Engineering, was recognized "for fundamental contributions to research in dataflow computing, memory models, and cache coherence protocols."

The Goode Award was established to recognize achievement in the information-processing field–either a single contribution of theory, design, or technique of outstanding significance; or the accumulation of important contributions on theory or practice over an extended period.

Arvind received his MS and PhD in Computer Science from the University of Minnesota in 1972 and 1973, respectively, and his B. Tech. in Electrical Engineering from the Indian Institute of Technology, Kanpur, in 1969. Prior to joining MIT, Arvind taught at the University of California, Irvine, from 1974 to 1978, and at IIT Kanpur from 1977 to 1978.

Arvind has contributed to the development of dynamic dataflow architectures, the implicitly parallel programming languages Id and pH, and the compilation of these types of languages on parallel machines. Dr. R. S. Nikhil and Arvind published the book "Implicit Parallel Programming in pH" in 2001.

In 1992, Arvind's group, in collaboration with Motorola, completed the Monsoon dataflow machine and its associated software. A dozen of these machines were built and installed at Los Alamos National Labs and other universities, before Monsoon was retired to the Computer History Museum in the Silicon Valley.

In 2000, Arvind started Sandburst, a fabless semiconductor company to produce a chip set for 10G-bit Ethernet routers. He served as its President until his return to MIT in September 2002. Sandburst was acquired by Broadcom in 2006. In 2003, Arvind co-founded Bluespec Inc, an EDA company to produce a set of tools for high-level synthesis, and serves on its board.

Arvind's current research focus is on enabling rapid development of embedded systems using Guarded Atomic Actions. He continues to be interested in memory models and cache coherence protocols for parallel architectures and languages.

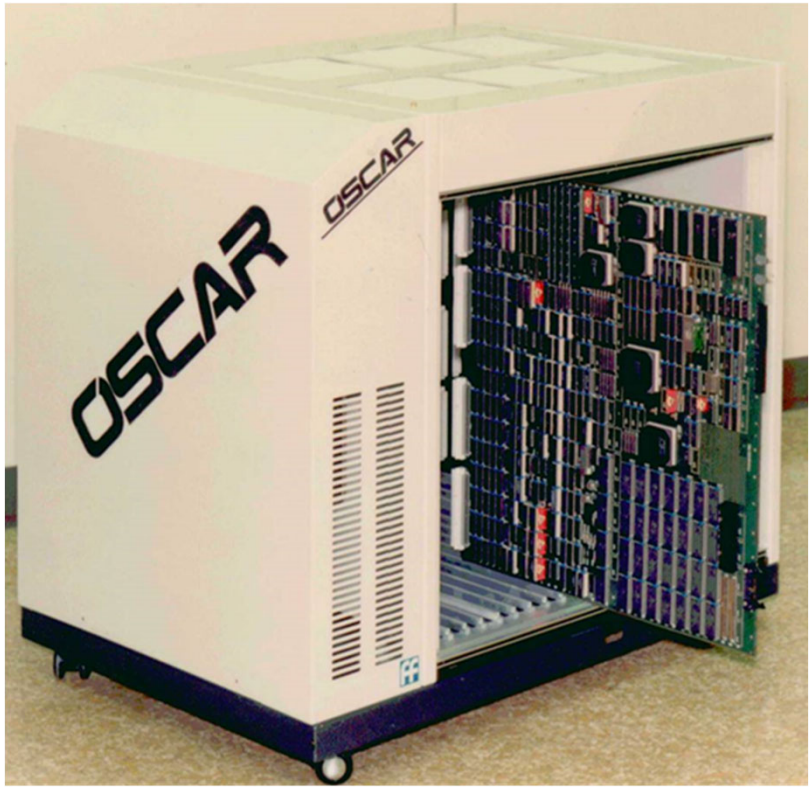
Arvind is a Fellow of IEEE and ACM, and a member of the National Academy of Engineering. Further information about the Goode Award, including a list of past recipients, can be found at: http://www.computer.org/portal/web/awards/harrygoode.

Functional Language Id

MIT CSAIL

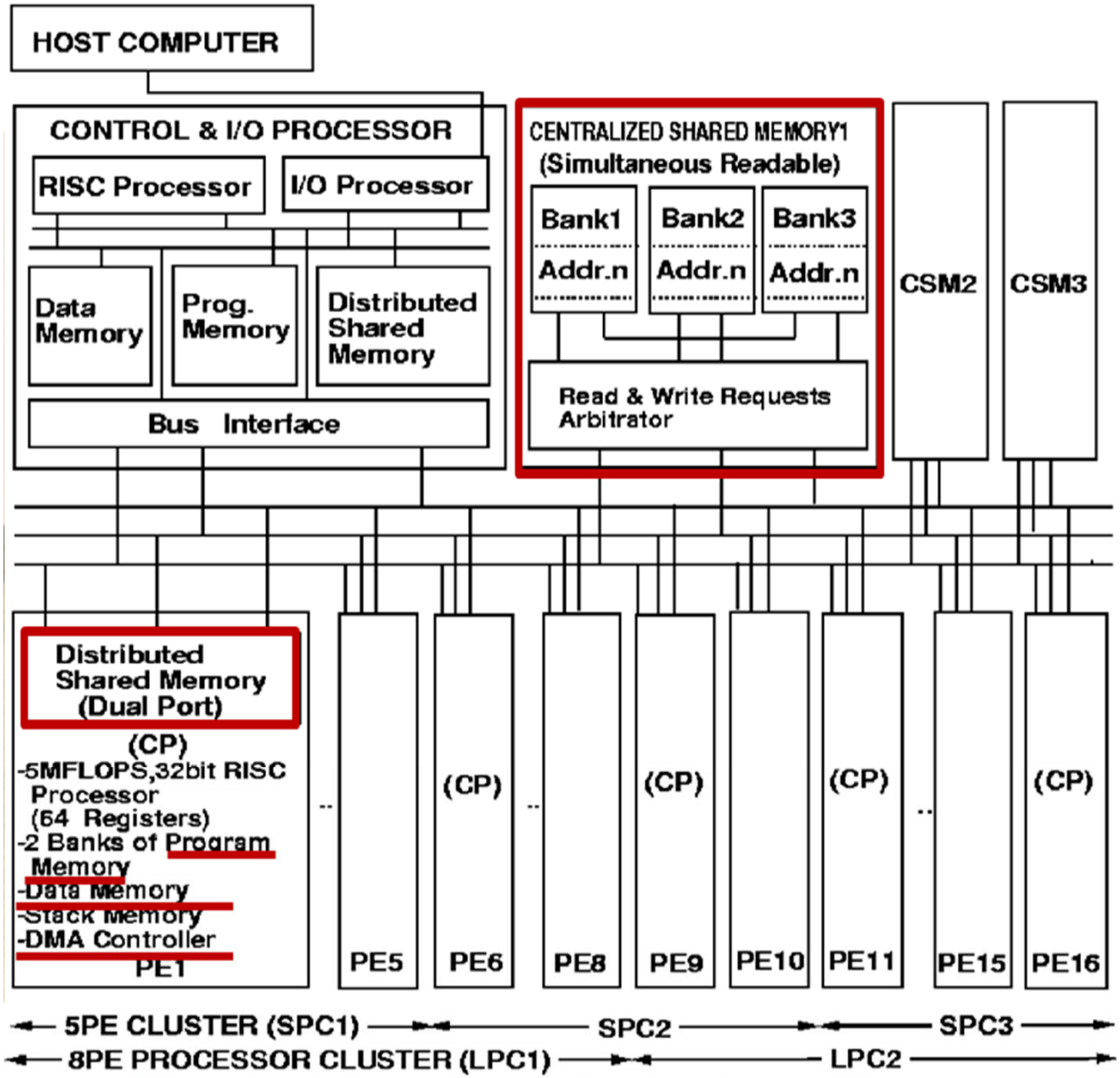
The First Compiler Codesigned Multiprocessor

OSCAR (Optimally Scheduled Advanced Multiprocessor) in 1987



AMD29325 32-bit Floating-point unit

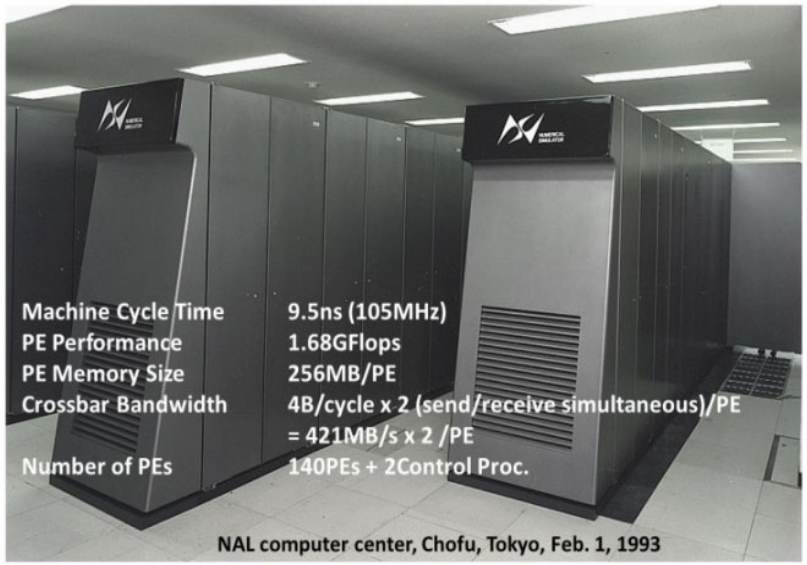
H. Kasahara, "OSCAR Fortran Multigrain Compiler", Stanford University, Hosted by Professor John L. Hennessy and Professor Monica Lam, May. 15. 1995.



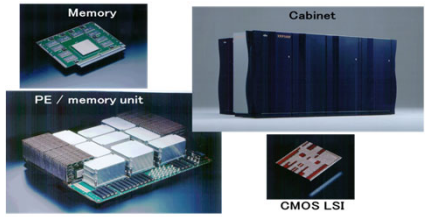
Hierarchical Group Barrier Synchronization Hardware

笠原博徳が設計・開発に参加した3つの世界No.1 スーパーコンピュータ “NWT：数値風洞”, “Earth Simulator：地球シミュレータ”, and “K：京”

日本スパコンの父
三好甫氏
航空宇宙技術研究所
早稲田大学数学卒
Father of Japanese Supercomputer
Mr. Hajime Miyoshi,
Waseda Alumnus,
Leader of NWT,
Earth Simulator

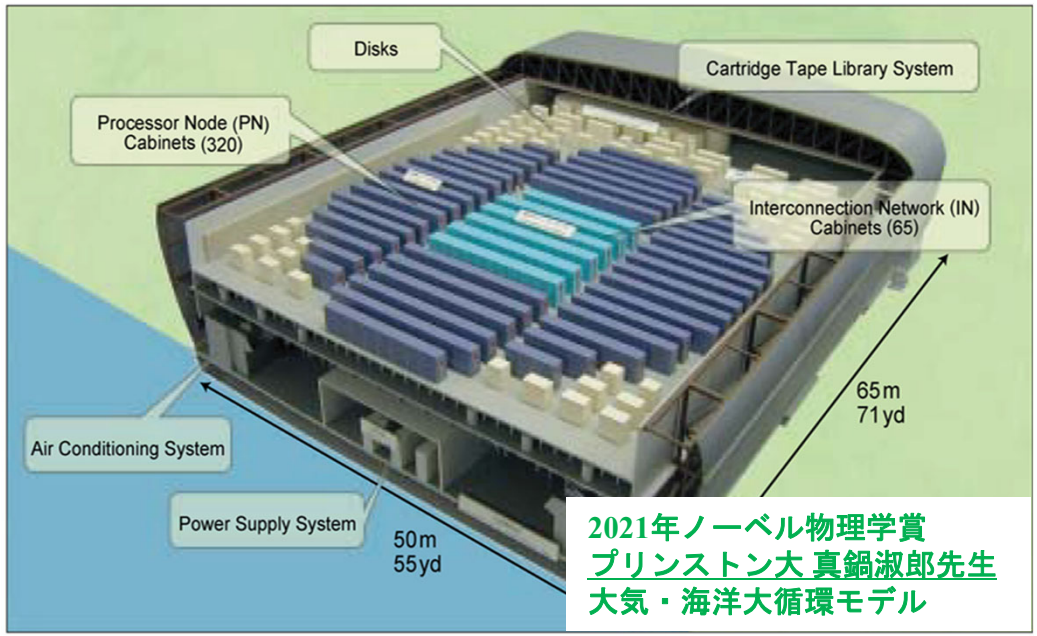


NWT：数値風洞
(Numerical Wind Tunnel) ,
1993,1.68GFLOPS
<Fujitsu VPP 500, 5000>



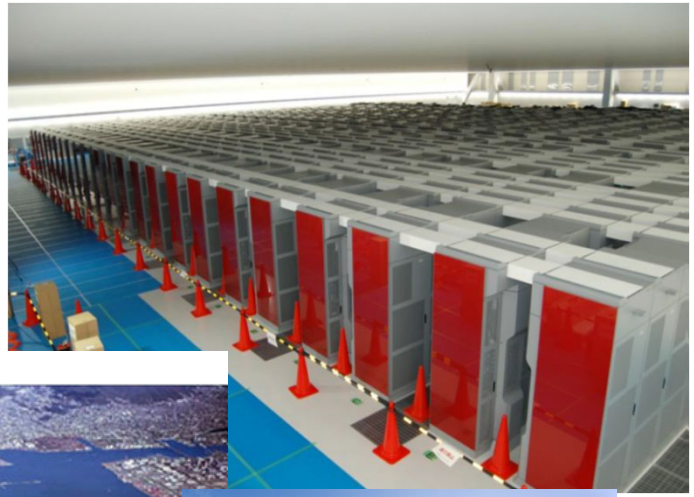
Earth Simulator,
2002

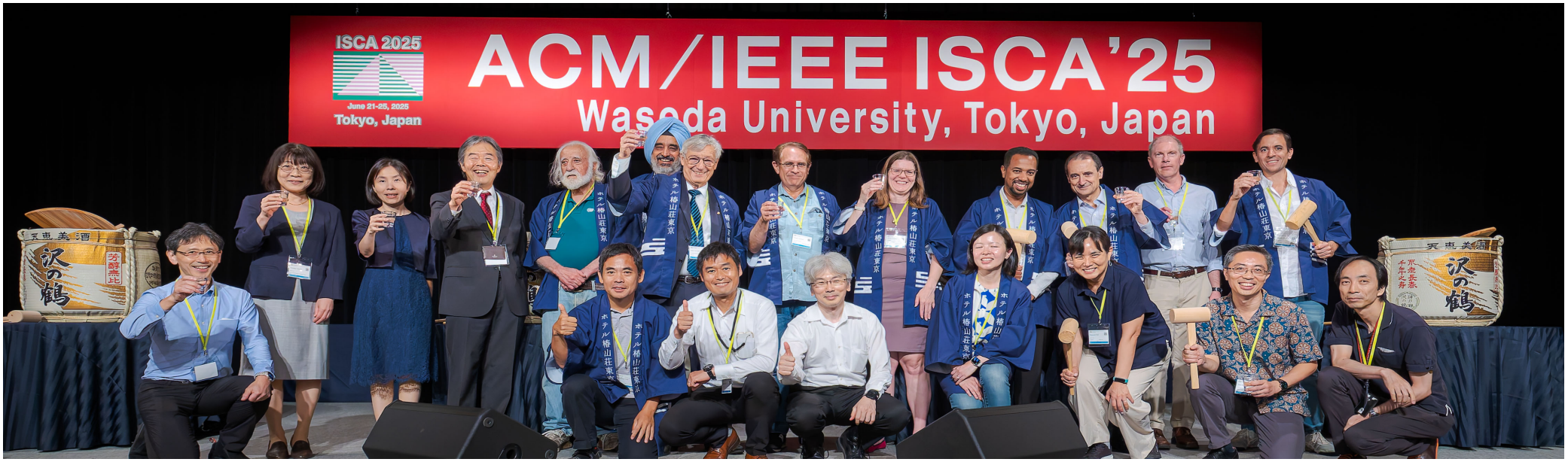
40 TFLOPS Peak ($40*10^{12}$)
35.6 TFLOPS Linpack
3.2MW

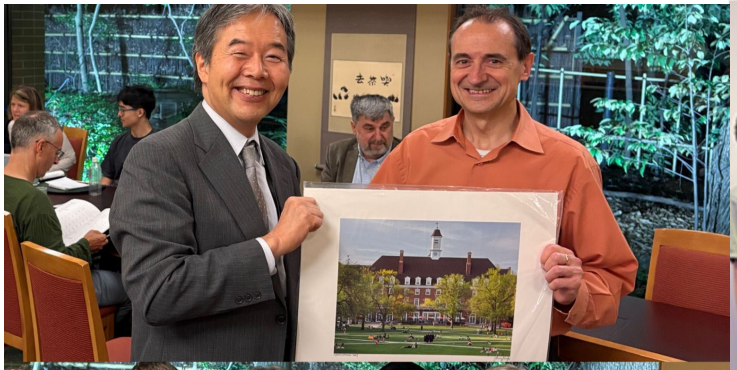


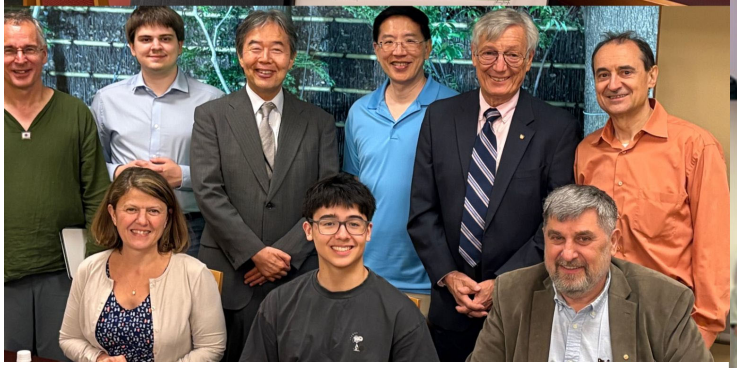
2021年ノーベル物理学賞
プリンストン大 真鍋淑郎先生
大気・海洋大循環モデル

K「京」,
2011
10PFLOPS
11.3MW

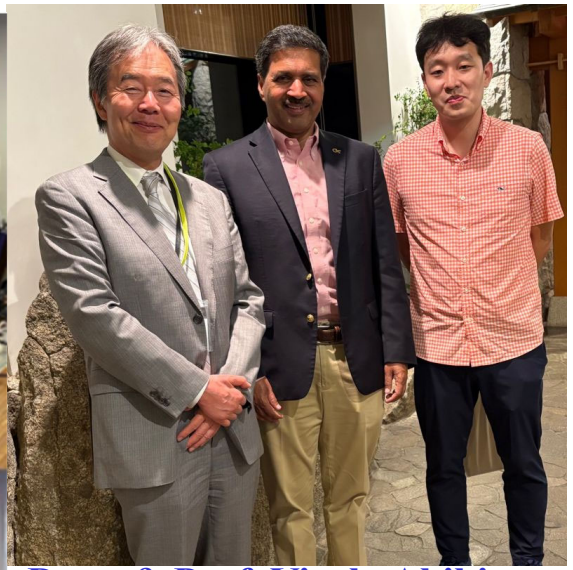








WS Meeting: Prof. Josep Torrellas (UIUC), VP Steve Woo (Rambus), VP Dejan Milojicic (HPE), Prof. Jean-Luc Gaudiot (UCI), Prof. Peter Braam (Oxford)



Dean & Prof. Vivek, Akihiro, Dr. Akihiro Hayashi (Georgia Tech.)

Demo Discussion in GCS with Kasahara & Kimura Lab Graduate Students.

The International Symposium on Computer Architecture
June 21-25, 2025
Waseda University, Tokyo, Japan

Jean-Luc Gaudiot

Hironori Kasahara

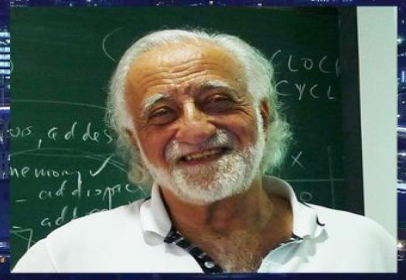
Hironori Kasahara and Jean-Luc Gaudiot, the General Chairs, cordially invite you to attend ACM/IEEE ISCA in Tokyo.

Join us for an exciting technical and social program, featuring a special invited talk and a super-panel on June 22, keynotes on June 23 and June 25, along with excellent accepted papers, tutorials, and workshops.

For their complete message, go to:
https://iscaconf.org/isca2025/

ISCA 2025

June 21-25, 2025
Tokyo, Japan



Special Invited Talk on Sunday
Time: 3:30 PM - 4:30 PM, June 22nd JST
Venue: Okuma Auditorium, Waseda University
Speaker: Prof. Yale Patt (UT Austin)
Title: "Solving the Sustainability Problem -- What must come first. ...and 2nd. ...and 3rd."

https://www.linkedin.com/in/hironori-kasahara-b6712855/

Satoshi Matsuoka

Keynote on Monday
Time: 8:45 AM - 9:45 AM, June 23rd JST
Venue: Okuma Auditorium, Waseda University
Speakers: Prof. Satoshi Matsuoka (RIKEN)
Title: "Of Oxen and Chickens: Seymour Cray's Legacy in the Convergent Future of HPC and AI"

Fitsum Assamnew Andargie

Todd Austin

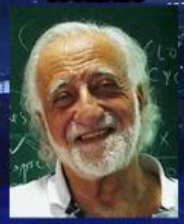
Keynote on Wednesday
Time: 8:30 AM - 9:30 AM, June 25th JST
Venue: Okuma Auditorium, Waseda University
Speakers: Prof. Fitsum Andargie (Addis Ababa University)
Prof. Todd Austin (University of Michigan)
Title: "First Global Trust, then Zero Trust"

Dejan Milojicic
HPE
(Moderator)

Wen-Mei Hwu
NVIDIA

Norm Jouppi
Google

Hironori Kasahara
Waseda Univ.


Yale Patt
UT Austin

Guri Sohi
Wisconsin Univ.

Carole-Jean Wu
Meta

Special Panel on "Sustainable Computer Architecture"
Time: 4:45 PM - 6:15 PM, June 22nd JST
Venue: Okuma Auditorium, Waseda University

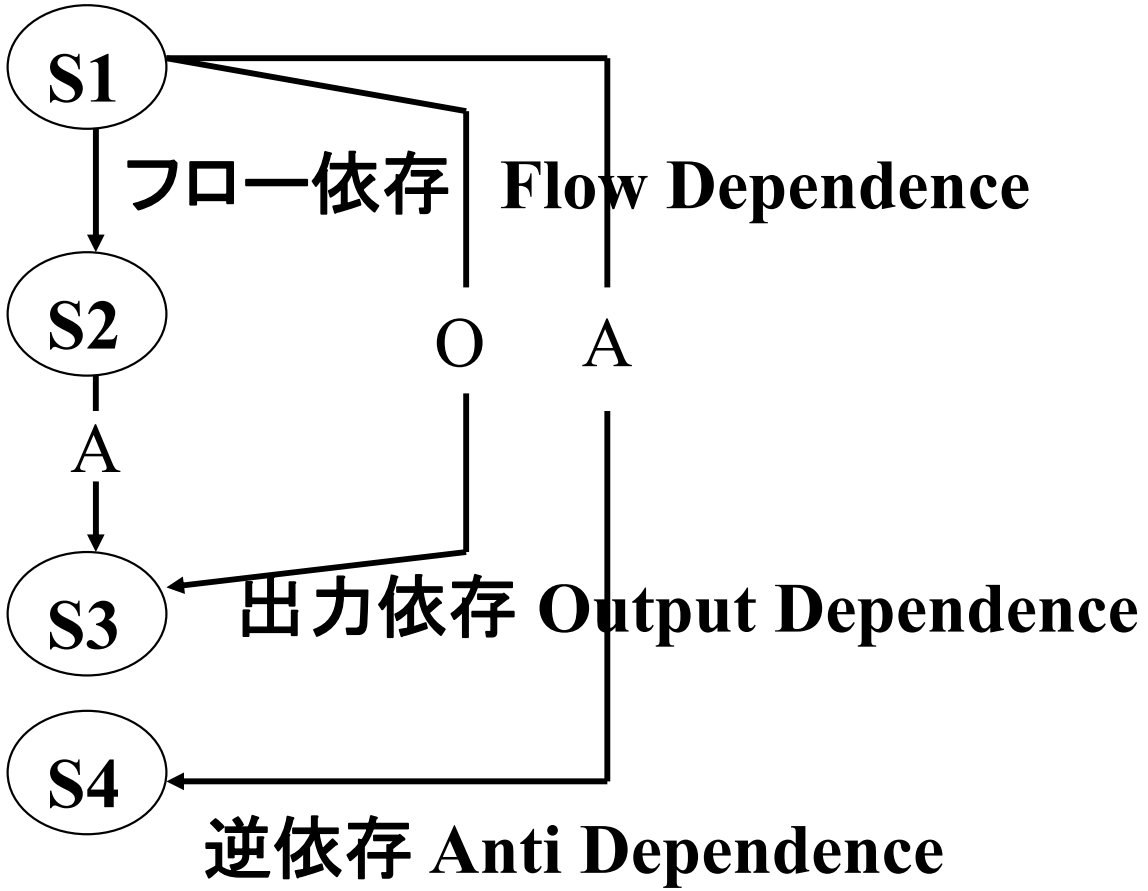
データ依存　Data　Dependency

A=B+C : S1

D=A+E : S2

A=C+F : S3

B=E+F : S4



参考書

笠原博徳「並列処理技術」，コロナ社

Reference

Hironori Kasahara, Parallel Processing Technology, Corona Publisher

GCD (Greatest Common Devisors) Test

```
do I=0,20
   A(9*I+16)= ・・・                    :S
    ・・・   = A(6*I-11) ・・・          :T
enddo
```

A(9*I+16)とA(6*I-11)の間にデータ依存があるかを決定

Checking if the data dependency between A(9*I+16) and A(6*I-11) exists.

$$9*I_1+16=6*I_2-11$$

即ち $9*I_1-6*I_2=-27$

ただし $0 \leqq I_1 \leqq 20, \quad 0 \leqq I_2 \leqq 20$

A problem is finding integers I_1 and I_2 to satisfy the above equation.

を満たすような異なるIの値I_1とI_2が存在するかチェックする問題

Diophantine方程式

$ax+by=c$

a, b and c are integer

ただし　a, b, c　は整数

を満たすような解xとyが存在するかチェックする問題

A problem to find an integer x and y to satisfy the above equation.

方程式が解を持つ条件：

d: the greatest common divisor of a and b

aとbの最大公約数d, $d=gcd(a,b)$, がcを割り切れる

A condition in which the equation has a solution is “c is divisible by d”.

＜例＞　$9*I_1-6*I_2=-27$

where　ただし　$0 \leqq I_1 \leqq 20$,　$0 \leqq I_2 \leqq 20$

I_1とI_2の係数である9と6の最大公約数3が-27を割り切れれば解を持つ

3=gcd(9,6). If -27 is divisible by 3, solutions may exist.

この例の場合は割り切れるので解を持つことが分かる

GCDチェックによりデータ依存が無いことが分かる例
An example GCD test shows there is no data dependence.

```
do I=1,50
   A(3*I+4)=B(I)+C(I)
   D(I)=A(3*I+9)+C(I)
enddo
```

のようなDoループでは、**Diophantine** 方程式は

$$3I_1+4=3I_2+9$$

$$3I_1-3I_2=5$$

Diophantine Equation

となり、この場合 gcd(3,3)=3 の3で5は割り切れないので、この方程式には解が存在しないことが分かり、従って2つのステートメント間にはデータ依存がないことが分かる。

3=gcd(3,3), 5 is indivisible by 3, there is no data dependence.

Diophantine方程式の一般解

$$x = c' x_0 - b' t$$

$$y = c' y_0 + a' t$$

ただし　x_0, y_0は　$ax+by=d$ を満たす特定解

$a' = a/d$,　$b' = b/d$,　$c' = c/d$

tは任意の整数

特定解x_0, y_0は拡張ユークリッドアルゴリズム（Extended Euclid's Algorithm）により見つけることができる

拡張ユークリッドアルゴリズム
(Extended Euclid's Algorithm)

$ax+by=d$, $d=gcd(a,b)$を満たす特定解$x0, y0$
をみつける アルゴリズム

Find a specific solution x_0 and y_o, by the following algorithm,

$$|a| \cdot x_1 + |b| \cdot y_1 = c_1 \quad (1)$$
$$|a| \cdot x_2 + |b| \cdot y_2 = c_2 \quad (2)$$

において、変数x_1, y_1, c_1, x_2, y_2, c_2を以下の手順のように操作する。 Calculate x_1,y_1,c_1,x_2,y_2,c_2 as follow:

1. $(x_1, y_1, c_1) \leftarrow (1, 0, |a|)$
   $(x_2, y_2, c_2) \leftarrow (0, 1, |b|)$
2. もし $c_2=0$ ならば If $c_2 = 0$, end as the following solution (x_0, y_0, d)
   $(x_o, y_o, d) \leftarrow (sign(a)*x_1, sign(b)*y_1, c_1)$とし終了。
3. $k \leftarrow \lfloor c_1/c_2 \rfloor$
4. $(s_1, s_2, s_3) \leftarrow (x_1, y_1, c_1) - k(x_2, y_2, c_2)$
   $(x_1, y_1, c_1) \leftarrow (x_2, y_2, c_2)$
   $(x_2, y_2, c_2) \leftarrow (s_1, s_2, s_3)$
   ステップ2. へ Got step 2

拡張ユークリッドアルゴリズムの適用例

$9*I_1-6*I_2=d$ の特定解を求めると

x_1, y_1, c_1, x_2, y_2, c_2は以下のように変化する

x_1	y_1	c_1	x_2	y_2	c_2	
1	0	9	0	1	6	k=1
0	1	6	1	-1	3	k=2
1	-1	3	-2	3	0	<-- ゼロ

$(x_o, y_o, d) \leftarrow (sign(9)*1,\ sign(-6)*-1,\ 3)$

を行い、 $x_o=1$, $y_o=1$, $d=3$ が得られる。

この特定解を用い $9*I_1-6*I_2=-27$ の一般解を求めると、

$a'=9/3=3$, $b'=-6/3=-2$, $c'=-27/3=-9$

すなわち $I_1=x=-9*1+2t=-9+2t$

$I_2=y=-9*1+3t=-9+3t$

ここで $0 \leqq I_1 \leqq I_2 \leqq 20$ 即ち $0 \leqq -9+2t \leqq -9+3t \leqq 20$

であるので、tの取りうる範囲は $5 \leqq t \leqq 9$ であり、

TがSにフロー依存するI_1, I_2の組は、

(1, 6), (3, 9), (5, 12), (7, 15), (9, 18)

シンボリック・データ依存解析(Symbolic data dependence analysis)

```
do I=1,N
   A(I+m)=B(I+N)        : S1
   B(I)=A(I+m-1)        : S2
end do
```

mの値がコンパイル時には分からない場合

A(I+m)とA(I+m-1)　：Cancelling m, change into analysis of A(i) and A(i-1)

同一シンボルmをキャンセルしA(I)とA(I-1)の解析に置き換えて解析を行う

B(I+N)とB(I)　：B(I+N) accesses B(1+N) through B(N+N) and B(I) accesses B(1) through B(N). There is no overlap and no data dependence.

B(I+N)で参照される範囲B(1+N)からB(N+N)は、B(I)で定義される範囲B(1)からB(N)とオーバーラップが無いのでデータ依存が生じない

グローバル先行代入(Global Forward Substitution)

目的：ベクトル化あるいは並列化の対象となるＤｏループ内部たけでなく、ループ外部の情報を使用することによりループ内の依存解析をより正確に行う

```
N1=N+1
N2=N+2
  .
  .
do  I=1,N
    A(I)=B(I,N1)+1.0
    B(I,N2)=A(I+N1)*2.0
end do
```

Substitute N1 and N2 into N+1 and N+2. No dependence between A(I) and A(I+N+1) and B(I,N+1 and B(I, N+2)

Ｄｏループ以前で定義されている　N1=N+1とN2=N+2をＤｏループ中の配列添字中に使われている　N1及びN2にシンボリックに代入し、A(I)とA(I+N1)間、B(I,N1)とB(I,N2)間のデータ依存解析を行う

A(I)とA(I+N+1)間，　B(I,N+1)とB(I,N+2)に依存無し

逐次処理用コンパイラにおける定数の伝搬の拡張

セマンティック・データ依存解析(Semantic data dependence analysis)

目的：Doループ外部で手にはいるセマンティック情報
を使用してDoループ内のデータ依存を調べる

```
if (L≧0) then
    do I=1,N
        B(I)=A(I)*B(I+L)
    end do
```

Using semantic information available outside the Do loop, check the data dependence.

L>=0, vectorizable

L<0, non-vectorizable

B(I+L)のL＜0の場合にはベクトル化不可
L≧0の場合にはベルトル化可能

Doループ直前のif文をチェックするとこのDoループが実行されるのは常にL≧0の場合だけでありベクトル化できることが分かる

This loop is executed in a case of “L>=0”, so this loop can be vectorizable.

マルチバージョンループ (Multi-version loop)

```
do I=1,N
    B(I)=A(I)*B(I+L)
end do
```

上例のようなＤｏループを検出し、コンパイラがプログラムを自動的に以下のように変換

```
if (L≧0) then
    B(1:N)=A(1:N)*B(1+L:N+L)
else
    do I=1,N
        B(I)=A(I)*B(I+L)
    end do
end if
```

The above loop without a conditional branch is restructured to the right two version loops and vectorized.

コンパイラがL＜0の場合とL≧0の場合の２つのバージョンを自動生成するので２バージョンループと呼ばれる

セマンティック・データ依存解析(Semantic data dependence analysis)

目的：Doループ外部で手にはいるセマンティック情報を使用してDoループ内のデータ依存を調べる

```
if (L≧0) then
    do I=1,N
        B(I)=A(I)*B(I+L)
    end do
```

Using semantic information available outside the Do loop, check the data dependence.

L>=0, vectorizable

L<0, non-vectorizable

B(I+L)のL＜0の場合にはベクトル化不可

L≧0の場合にはベルトル化可能

Doループ直前のif文をチェックするとこのDoループが実行されるのは常にL≧0の場合だけでありベクトル化できることが分かる

This loop is executed in a case of “L>=0”, so this loop can be vectorizable.

マルチバージョンループ (Multi-version loop)

```
do I=1,N
    B(I)=A(I)*B(I+L)
end do
```

上例のようなＤｏループを検出し、コンパイラがプログラムを自動的に以下のように変換

```
if (L≧0) then
    B(1:N)=A(1:N)*B(1+L:N+L)
else
    do I=1,N
        B(I)=A(I)*B(I+L)
    end do
end if
```

The above loop without a conditional branch is restructured to the right two version loops and vectorized.

コンパイラがL＜0の場合とL≧0の場合の２つのバージョンを自動生成するので２バージョンループと呼ばれる

図3.10 ループ崩壊 Loop collapsing

```
real A(6,6),B(6,6)
do I=1,6
  do J=1,6
    A(I,J)=B(I,J)+C(I,J)
  end do
end do
```

(a)

```
real A(36),B(36)
do IJ=1,36
  A(IJ)=B(IJ)+(IJ)
end do
```

(b)

図3.11 ストリップマイニング Strip-mining

```
do I=1,N
  C(I)=A(I)+B(I)
end do
```

(a)

```
do I=1,N,64
  do J=I,min(I+63,N)
    C(I)=A(I)+B(I)
  end do
end do
```

(b)

ループフュージョン(Loop Fusion)

複数のDoループ

```
do  I=1,N
      A(I)=B(I)*C(I)
end  do
do  I=1,N
      D(I)=A(I)+C(I)
end  do
```

を一つのDoループ

Above Multiple loops are fused into a single loop below.

```
do  I=1,N
      A(I)=B(I)*C(I)
      D(I)=A(I)+C(I)
end  do
```

に融合

For reducing loop control overheads including Doall initiation and using fast registers instead of slow shared memory

＜目的＞　ループ制御のオーバーヘッドの軽減

・　　メモリアクセスを減らし　　レジスタの有効利用

・マルチプロセッサシステムではDoallのためのオーバーヘッドの軽減

ループフュージョンの適用できるループの条件

Conditions of loops that Loop Fusion can be applied

・ループのリミットが同一

Loop limits of loops are the same.

・ステートメント間のデータ依存関係がフュージョン前後で異ならない

Data dependences should be kept before and after loop fusion.

ただしリミットが同一でない場合でも、リストラクチャリングによりフュージョンできる場合がある

＜ループリミットの変更＞

Loops having different loop limits

```
do I=1,N
      C(I)=A(I)+B(I)
end do
do I=2,N+1
      F(I)=D(I)*E(I)
end do
```

2番目のループのリミットをずらし同時に配列の添字もずらすことにより次のようにフュージョン可能

Changing Loop limits shifting array indexes.

```
do I=1,N
      C(I)=A(I)+B(I)
      F(I+1)=D(I+1)+E(I+1)
end do
```

ループフュージョン不可の例: Non-fusible loops

ループリミットが同一でも、ステートメント間のデータ依存関係がフュージョン前後で異なる場合には融合不可

Loops having data dependence

```
do  I=1,N
      C(I)=A(I)+B(I)          :S
end  do
do  I=1,N
      A(I+1)=D(I)*E(I)        :T
end  do
```

フュージョン前：1番目のループ中で配列Aの値を使用してから2番目のループでAを定義

Fused loop has different data dependence from the above loops related to A(I) and A(I+1)

```
do  I=1,N
      C(I)=A(I)+B(I)          :S
      A(I+1)=D(I)*E(I)        :T
end  do
```

フュージョン後：SがA(I)の値を使用する時、そのA(I)の値は一つ前のイタレーションのTで定義されたA(I+1)の値となり、フュージョン前後で計算結果が異なる

データ依存以外でフュージョン時に注意を要する場合
Cases careful loop fusion is required

- フュージョン前のループにループ外への条件分岐や入出力命令がある場合：

There are conditional branches to outside loops and I/O instructions in the loop bodies.

フュージョン前後で結果が変化しないように慎重に行う必要有り

アンローリング (Loop Unrolling)

```
do I=1,N
      C(I)=A(I)+B(I)
end do
do I=2,N
      F(I)=D(I)*E(I)
end do
```

の場合には、前のループを1イタレーション分だけアンローリングすることにより次のようにフュージョン可能

```
C(1)=A(1)+B(1)
do I=2,N
      C(I)=A(I)+B(I)
      F(I)=D(I)*E(I)
end do
```

By unrolling the first iteration of the the first loop, the two loos can be fused.

ループ インタチェンジ (Loop Interchange)

```
do I=1,100
  do J=2,20
    B(I,J)=A(I,J)+B(I,J-1)
  end do
end do
```

(a)

```
do J=2,20
  do I=1,100
    B(I,J)=A(I,J)+B(I,J-1)
  end do
end do
```

(b)

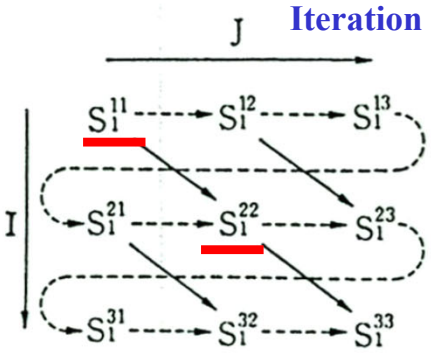
図 3.9　ループ インタチェンジ

図 3.15 ループインタチェンジが適用できる多重ループ
A nested loop to which loop interchange can be applied

```
do I=1, 3
  do J=1, 3
    A(I, J)=A(I-1, J-1)* B(I, J)      :S1
  end do
end do
```

A loop to which loop interchange can be applied

(a) インタチェンジ可能なループの例



Iteration space

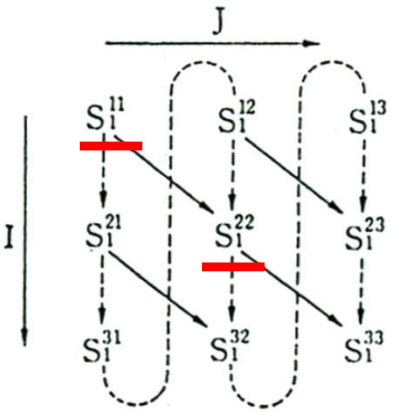
イタレーションスペース

Data dependence direction vector

S1 δ << S1
(<, <)：データ依存方向ベクタ

(b) インタチェンジ前の実行順序

Iteration execution order before interchanger



Iteration execution order after interchange.
This order satisfy the data dependences.

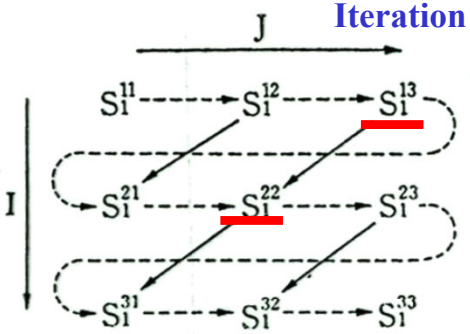
(c) インタチェンジ後の実行順序を表し，
チェンジしてもデータ依存が満足される

図 3.16 ループインタチェンジが適用できない多重ループ
A nested loop to which loop interchange cannot be applied

```
do  I=1, 3
  do  J=1, 3
    A (I, J)=A(I-1, J+1)*B(I, J)     :S1
  end do
end do
```

A loop to which loop interchange can not be applied directly

(a) インタチェンジ不可能なループの例



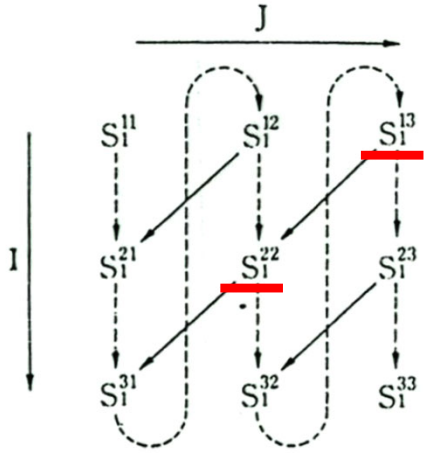
Iteration space

Data dependence direction vector

S1 δ <> S1
(<, >)：データ依存方向ベクタ

(b) インタチェンジ前の実行順序

Iteration execution order before interchanger



Iteration execution order after interchange.
This order does not satisfy the data dependences.

(c) インタチェンジした場合の実行順序を表し，
データ依存が満足されないことを示している

図3.12　Doall とDoacross ループ　　Doall and Doacross loops

```
do I = 1,N
  do J=2,N
S1:   C(I,J)=A(I,J-1)+B(I,J)
S2:   D(I,J)=3.0*C(I,J)
S3:   A(I,J)=C(I,J)/2.0
S4:   B(I,J)=A(I,J)+D(I,J)
  end do
end do
```

I: Doall
J: Doacross because of J-I and J
A(I,J-1) should wait for A(I,J) calculated by the previous processor

(a)　Do allおよびDo acrossループの例

(a) An example of Doall and Doacross loop

```
S1 δ= S2
S1 δ= S3
S1 δA= S4
S2 δ= S4
S3 δ= S4
S3 δ< S1
```

Dependence direction
=: dependence inside I-th iteration
<: J-th iteration has a loop carried dependence on previous iteration.

(b)　(a)のデータ依存方向

(b)Data dependence directions for (a)

この2重ループは I に関して Doall でき，J に関しては J=k の S1 が J=k−1 の S3 の計算終了を待つこと（同期）により Doacross 処理できる。

$\delta_{=<}$ における =< はそれぞれ I と J に関するデータ依存の向きを表し，= は I=k の中だけで依存があり，< は J=k−1 から J=k の向きへの依存を表す。

図 3.12　Doall ループおよび Doacross ループの例

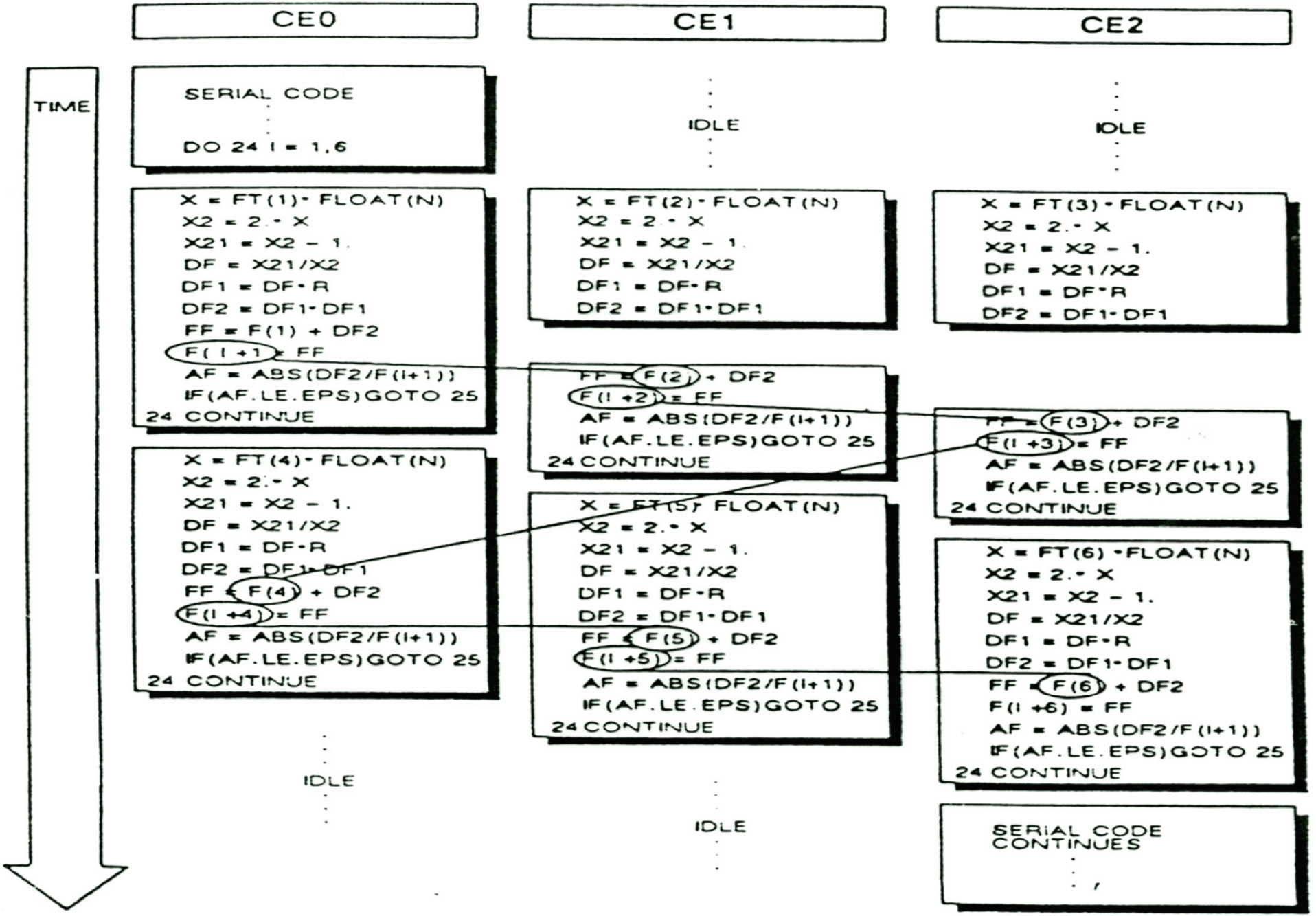
図3.13 Doallの例　An example of Doall loop

```
(I=1)                (I=2)                (I=p)
do J=2,N             do J=2,N             do J=2,N
 C(1,J)=A(1,J-1)      C(2,J)=A(2,J-1)      C(p,J)=A(p,J-1)
 D(1,J)=3.0*C(1,J)    D(2,J)=3.0*C(2,J)    D(p,J)=3.0*C(p,J)
 A(1,J)=C(1,J)/2.0    A(2,J)=C(2,J)/2.0    A(p,J)=C(p,J)/2.0
 B(1,J)=A(1,J)+D(1,J) B(2,J)=A(2,J)+D(2,J) B(p,J)=A(p,J)+D(p,J)
end do               end do               end do
(I=p+1)              (I=p+2)              (I=2p)
(I=2p+1)             (I=2p+2)             (I=3p)
  ⋮                    ⋮                    ⋮
 PE 1                 PE 2                 PE p
```

図3.14 Doacrossの例 An example of Doacross loop

```
I=1
(J=2)                (J=3)                (J=p+1)
                      [wait(A(1,2))]       [wait(A(1,p))]
 C(1,2)=A(1,1)        C(1,3)=A(1,2)        C(1,p+1)=A(1,p)
 D(1,2)=3.0*C(1,2)    D(1,3)=3.0*C(1,3)    D(1,p+1)=3.0*C(1,p+1)
 A(1,2)=C(1,2)/2.0    A(1,3)=C(1,3)/2.0    A(1,p+1)=C(1,p+1)/2.0
 [send(A(1,2))]       [send(A(1,3))]       [send(A(1,p+1))]
 B(1,2)=A(1,2)+D(1,2) B(1,3)=A(1,3)+D(1,3) B(1,p+1)=A(1,p+1)+D(1,p+1)
(J=p+2)              (J=p+3)              (J=2p+1)
(J=2p+2)             (J=2p+3)             (J=3p+1)
  ⋮                    ⋮                    ⋮
 PE 1                 PE 2                 PE p
I=2
  ⋮
I=N
```

Doacross Execution used by Alliant FX systems with (Courtesy of Alliant Computer Systems Corp.)



Doallループ・スケジューリング
Doall loop Scheduling

A. スタティックスケジューリング Static

- ブロック分割　　Block decomposition
- サイクリック分割　Cyclic decomposition

B. ダイナミックスケジューリング　Dynamic

ループ内部に条件分岐等があり各イタレーションの処理時間が大幅に変化しプロセッサ間の負荷がアンバランスになる場合　(F&A, F&I: Fetch & Increment)

- セルフスケジューリング　Self-scheduling
- チャンクスケジューリング　Chunk-
- ガイデッドセルフスケジューリング　Guided Self-
- ファクタリング　Factorizing
- トラペゾイドセルフスケジューリング　Trapezoidal Self-
- アフィニティスケジューリング　Affinity-

Doallスタティック・スケジューリング
Doall Static Scheduling

p台のプロセッサの内m番目のプロセッサに

Iterations assigned to m-th processor in p processors by the following two methods:

Block ・ $I=(m-1)\lceil n/p\rceil+1 \sim m\lceil n/p\rceil \leqq N$ のイタレーションを割当てる方式

Cyclic $I=(l-1)p+m \leqq N$, ただし $l=1,\cdots,\lceil n/p\rceil$ のイタレーションを割当てる方式

Strengths: No run-time scheduling overhead,
Minimization of Data transfer over loops is possible/

＜長所＞
・実行時のスケジューリング・オーバーヘッドが無い
・ループ間でのプロセッサ間データ転送を最小化可能

Weaknesses: Poor load balancing may occur when each iteration execution time largely varies because of conditional branches in the target loop

＜欠点＞
・ループ中に条件分岐等があり各イタレーションの処理時間が大きく変動する場合にはプロセッサ間の負荷バランスが崩れる可能性がある

セルフ・スケジューリング Self Scheduling

共有メモリ上に処理を終えたイタレーション数を記憶しておく一種のカウンタを用意し、各プロセッサが1イタレーションの処理を終えるとそのカウンタを参照し、次に自分が処理すべきイタレーションを一つ決め、カウンタをインクリメントする方式

Each processor gets next one iteration incrementing a counter in a shared memory.

セルフ・スケジューリングを効率よく実行するためには、Fetch & Addのような同期命令の使用が好ましい

Fetch & Add can be used for this scheduling on shared memory multiprocessor systems.

＜欠点＞

・1イタレーション毎にスケジューリングを行なうため実行時のスケジューリング・オーバーヘッドが大

Strengths: Good load balancing when each iteration execution time largely varies

Weaknesses: Large run-time scheduling overhead because scheduling is made every iteration.

従って1イタレーションの処理時間が大きくまたイタレーションの処理時間変動が非常に大きい時に有効

チャンク･スケジューリング Chunk Scheduling

実行時のスケジューリング・オーバヘッドを減らすために、セルフ・スケジューリングのように1イタレーションずつ次に処理すべきイタレーションを取ってくるのではなく、各プロセッサが事前に設定された数の複数イタレーションを取ってくる方式

Each processor gets next a set of iteration incrementing a counter in a shared memory.

＜長所＞

複数イタレーションに1回の割合でスケジューリングが行われるため、セルフ・スケジューリングよりオーバーヘッドが小さい

Strengths: Scheduling overhead is smaller than self-scheduling because scheduling is made every several iterations.

＜短所＞

プロセッサ負荷がアンバランスになる可能性がある

Weaknesses: Poor load balancing may occur when each iteration execution time largely varies because of conditional branches in the target loop

ガイデッド・セルフ・スケジューリング Guided Self Scheduling

＜目的＞セルフ・スケジューリングとチャンク・スケジューリングの欠点を補う

Considering problems of self and chunk scheduling, chunk size is changed at run-time according to left iterations .

1）各プロセッサが次に処理すべきイタレーションを決める際、その時点で残っているイタレーション数Riをプロセッサ数mで割った数、即ち

$$X = \lceil R_i / m \rceil$$

Each processor gets X iterations

個のイタレーションを、次に処理するイタレーションとして獲得

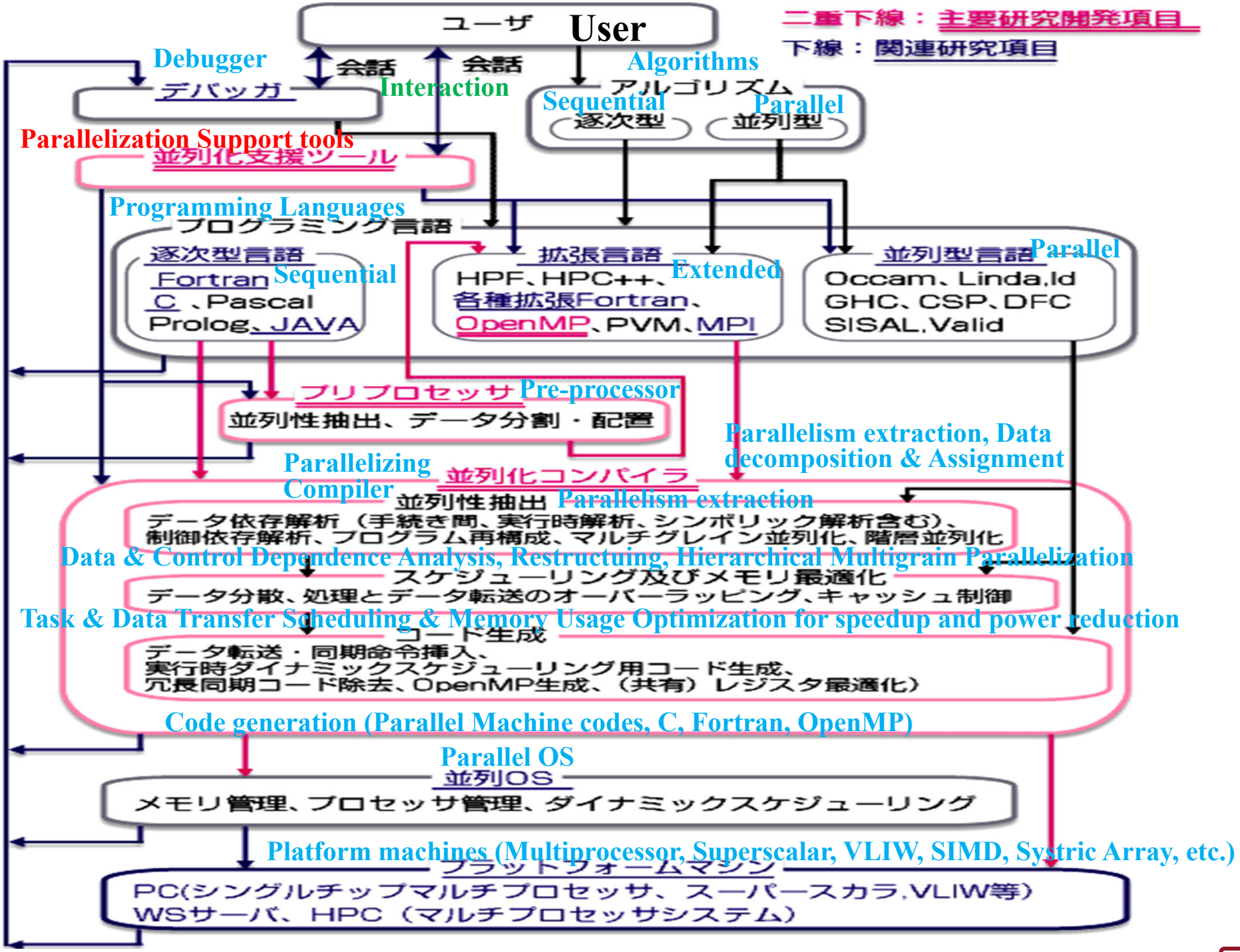
2）同時に共有メモリ上に置かれた残りイタレーション数を示すカウンタから自分が獲得したイタレーション数を引いて置く、即ち

$$R_{i+1} = R_i - X$$

Each processor updates the counter R representing the number of left iterations

METI Advanced Parallelizing Compiler Project

アドバンスト並列化コンパイラプロジェクト

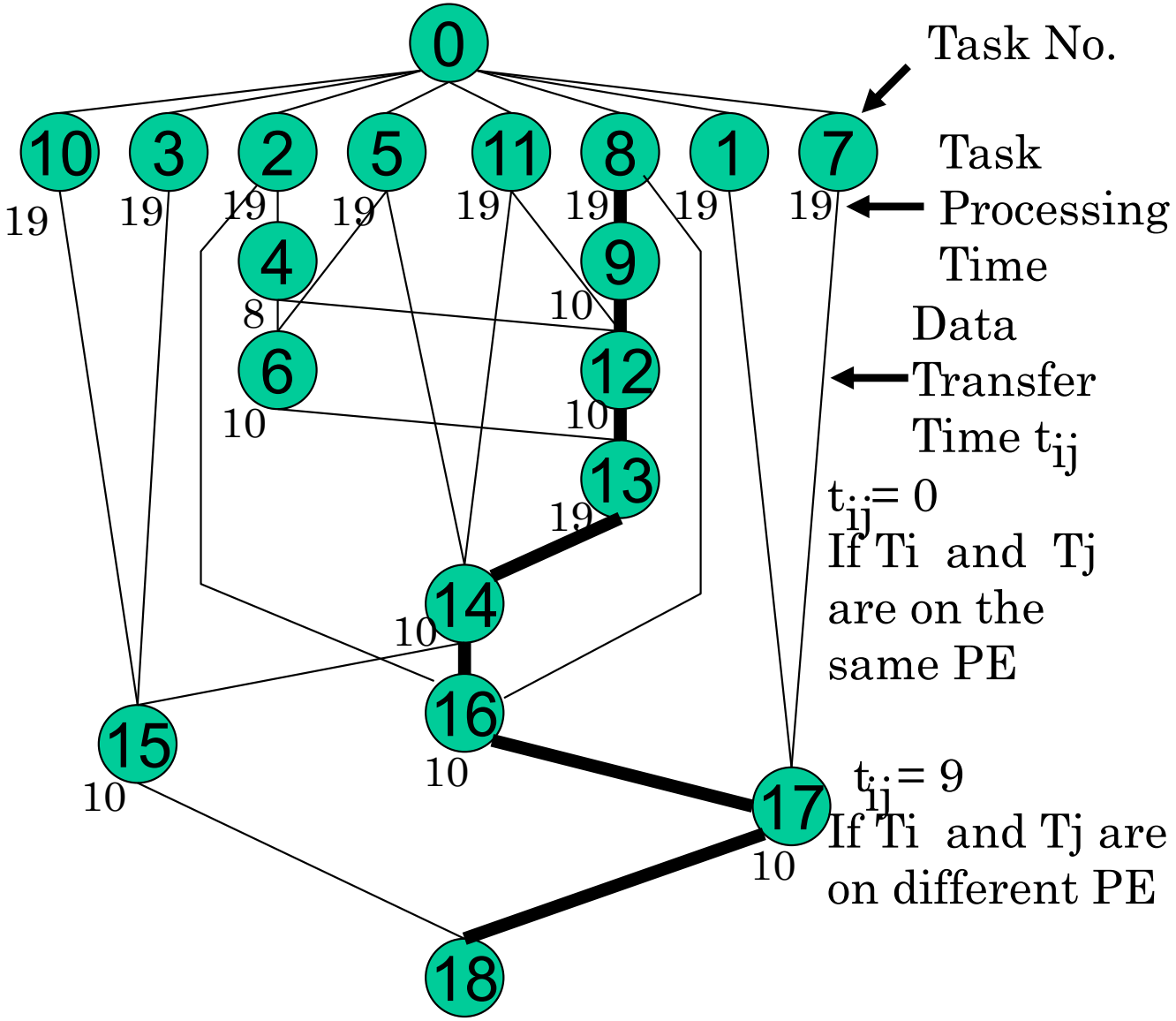


Statement Level Near Fine Grain Task

<<LU Decomposition>>
1) u12 = a12/l11
2) u24 = a24/l22
3) u34 = a34/l33
4) l54 = -l52 * u24
5) u45 = a45/l44
6) l55 = u55 - l54 * u45

<<Forward Substitution>>
7) y1 = b1 / l11
8) y2 = b2 / l22
9) b5 = b5 - l52 * y2
10) y3 = b3 / l33
11) y4 = b4 / l44
12) b5 = b5 - l54 * y4
13) y5 = b5 / l55

<<Backward Substitution>>
14) x4 = y4 - u45 * y5
15) x3 = y3 - u34 * x4
16) x2 = y2 - u24 * x4
17) x1 = y1 - u12 * x2



実行終了時間(スケジュール長)最小ノンプリエンプティブマルチプロセッサスケジューリング問題の複雑さ

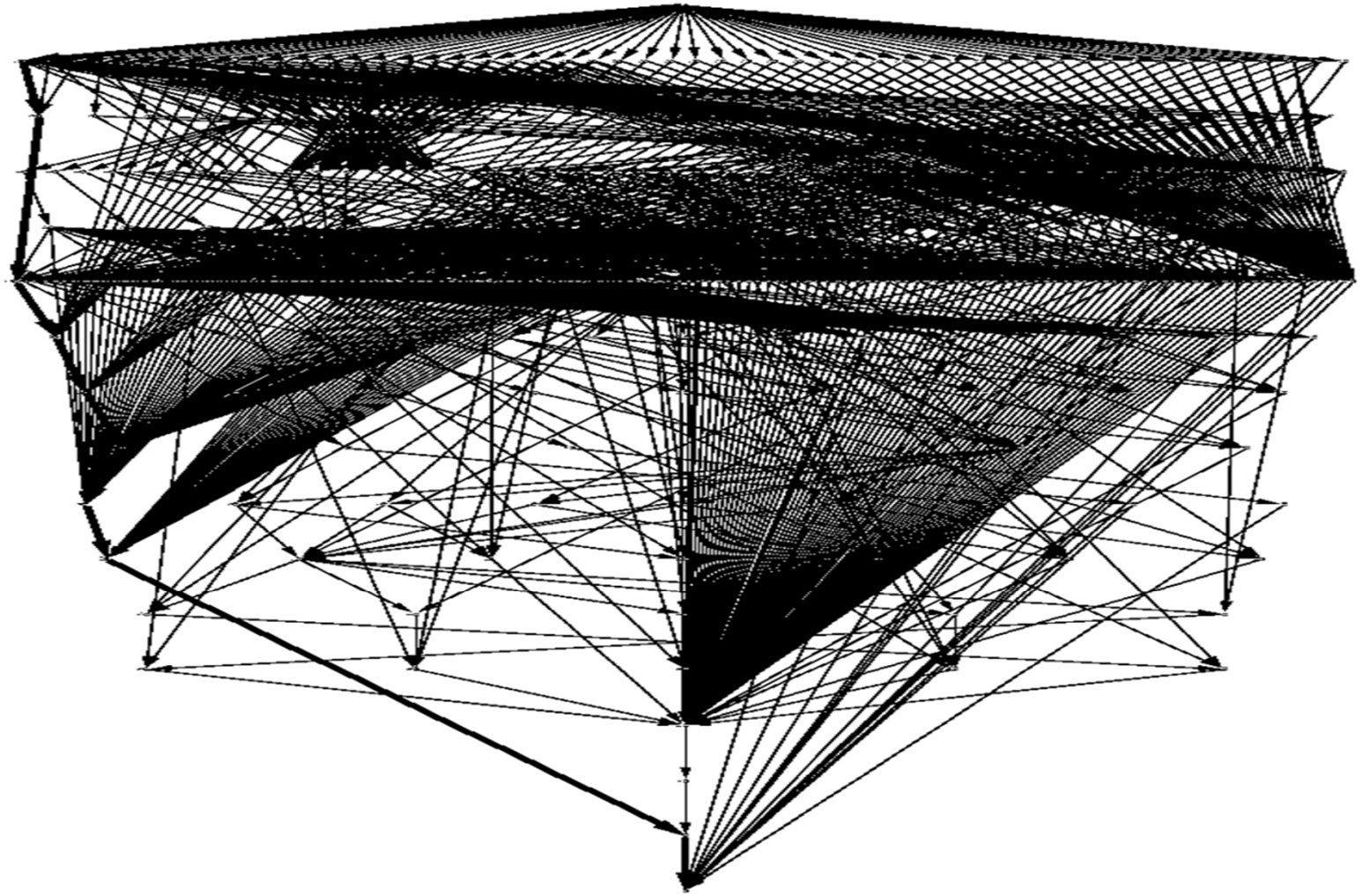
Computational Complexity of Minimum Execution-time Multiprocessor Scheduling Problems

問題番号	プロセッサ数 m No. of processors	タスク処理時間 t_j Task execution time	先行制約 (タスクグラフ形状) Task graph shape	複雑さ Complexity
1 Hu's algorithm	任意 Arbitrary	すべて第しい $t_j=1$ All one unit time	ツリー Arbitrary	$O(n)$
2 Coffman & Graham's algorithm	2	すべて等しい $t_j=1$ All one unit time	任意 Arbitrary	$O(n^2)$
3	任意 Arbitrary	すべて等しい $t_j=1$ All one unit time	任意 Arbitrary	$O(C^n)$ NP 困難 NP hard
4	$m \geqq 2$ の固定数	$t_j=1$ か 2 の 2 値 1 or 2	任意 Arbitrary	$O(C^n)$ NP 困難 NP hard
5	任意 Arbitrary	任意 Arbitrary	任意 Arbitrary	$\geqq O(C^n)$ 強 NP 困難 Strong NP hard

1：Hu のアルゴリズム、 2 : Coffman & Graham のアルゴリズム

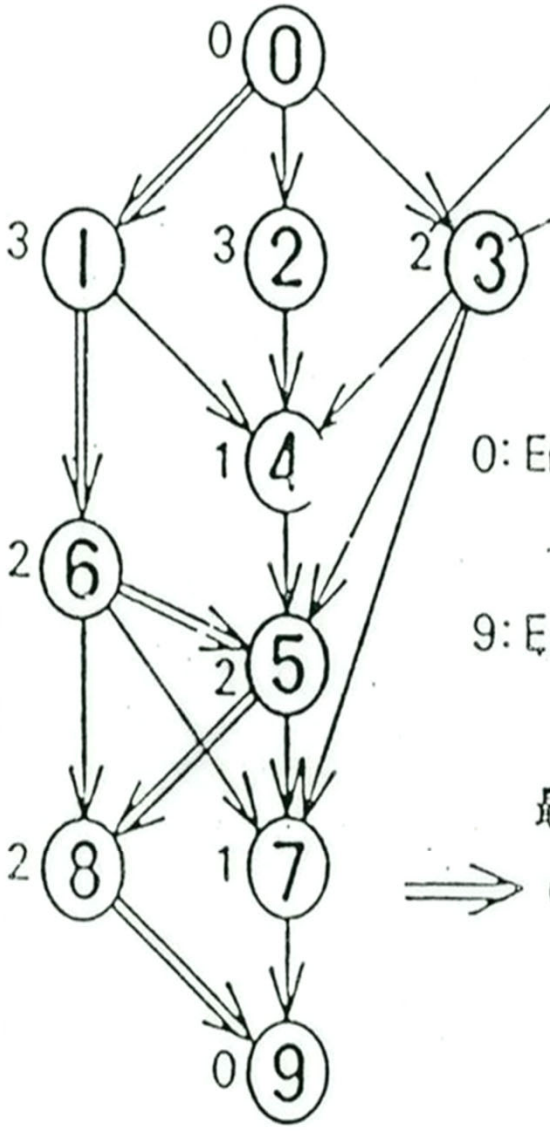
Task Graph for FPPPP

Statement level near fine grain parallelism



実行時間最小マルチプロセッサスケジューリング問題

タスク間に先行制約が有り、各タスクの処理時間が異なるn個のタスクから成るタスク集合T=(T1, ・・・, Tn)を、能力の等しいm台のプロセッサ上でノンプリエンプティブに並列処理する際に、その実行終了時間（スケジュール長）を最小にするスケジュールを求める問題



Processing Time

Task No.

$$t_{cr} = \max_{k} \sum_{i \in \phi_k} t_i$$

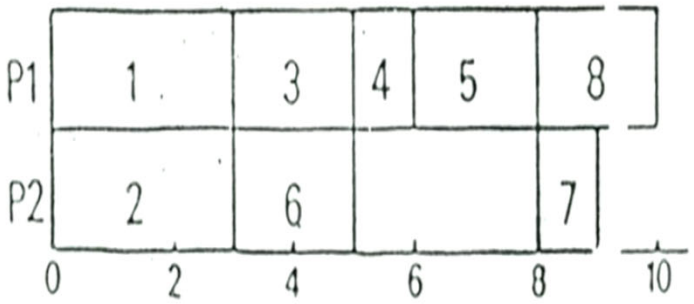
ϕ_k ：出口ノードから入口ノードへ至るk番目のパス.

0: Entrance Node
(dummy node)

9: Exit Node
(dummy node)

最小並列処理時間
⟹ Critical Path

Path Length 9[u.t]



CP（クリティカルパス）法

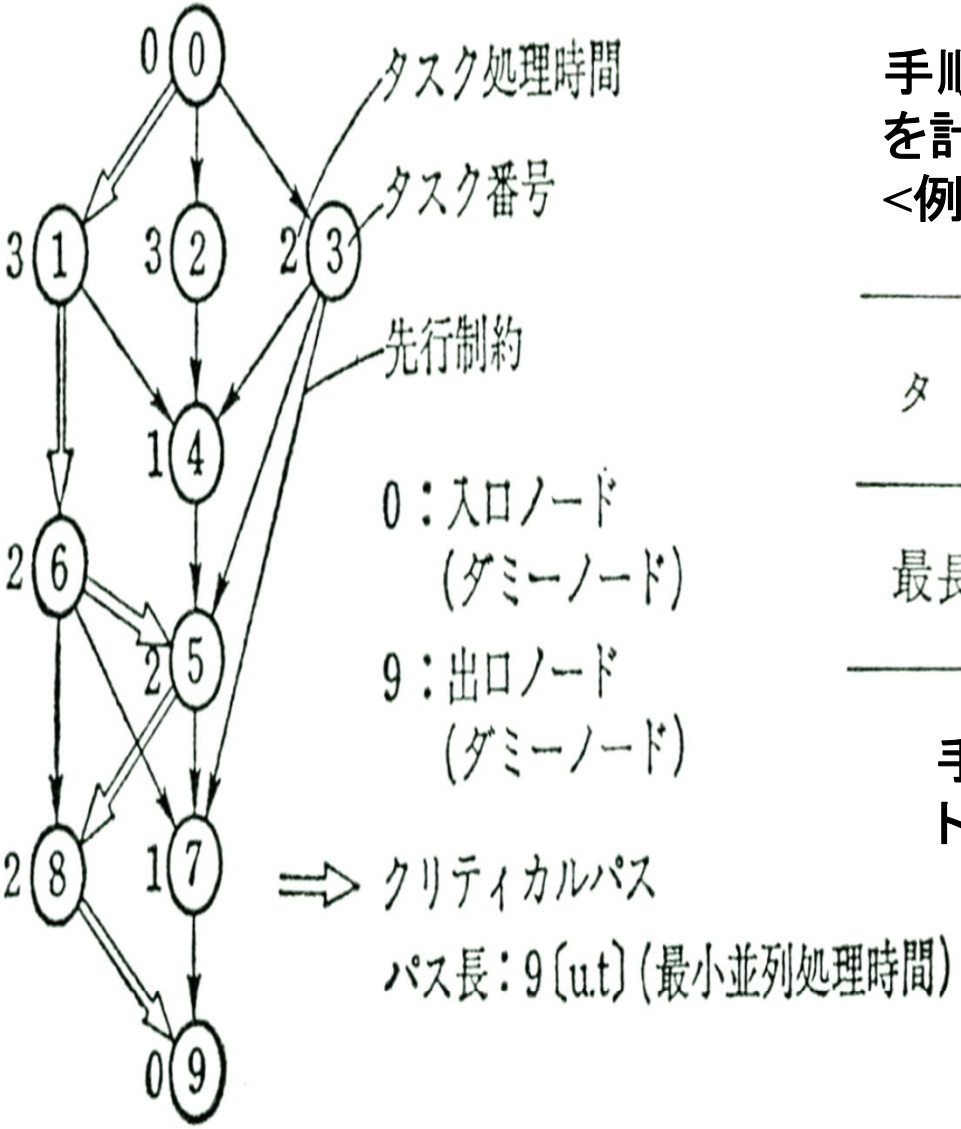


図3.20　タスクグラフ

手順1 各タスクから出口ノードまでの最長パス長を計算する。
<例>各タスクの出口タスクまでの最長パス長

タスク	1	2	3	4	5	6	7	8	9
最長パス長	9	8	7	5	4	6	1	2	0

手順2 最長パス長の大きい順にプライオリティリスト作成<例>プライオリティリスト：1,2,3,6,4,5,8,7
手順3 リストスケジューリングを行う。

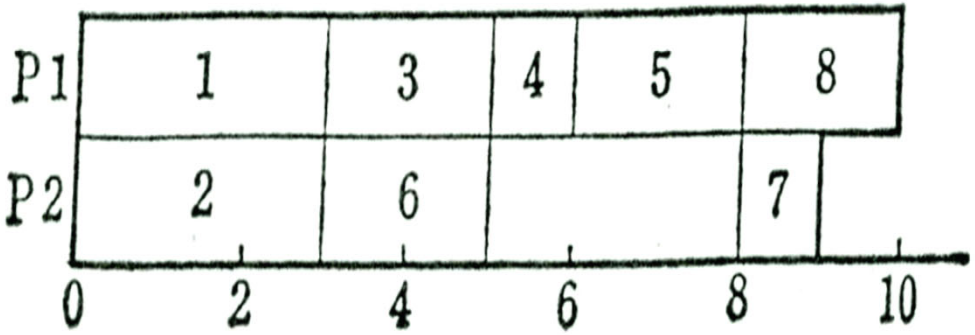
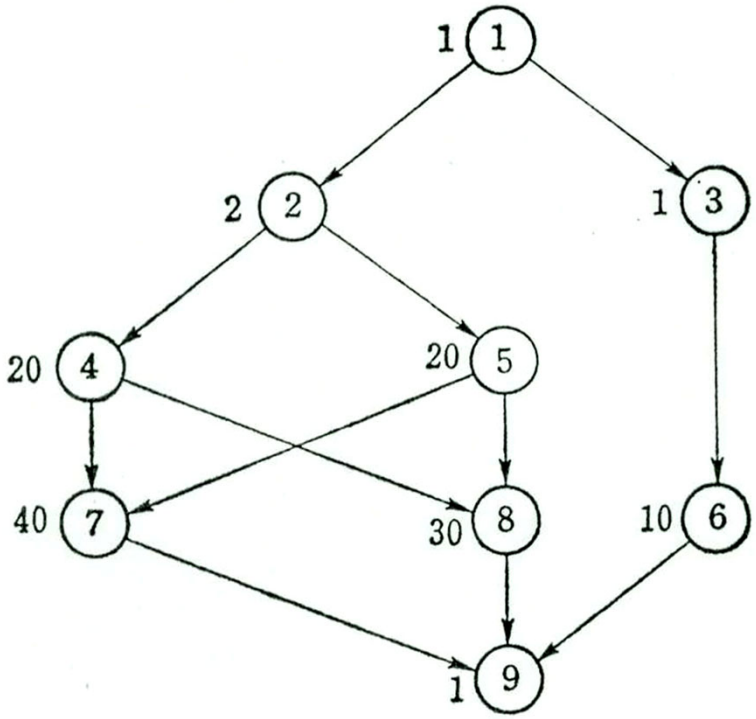
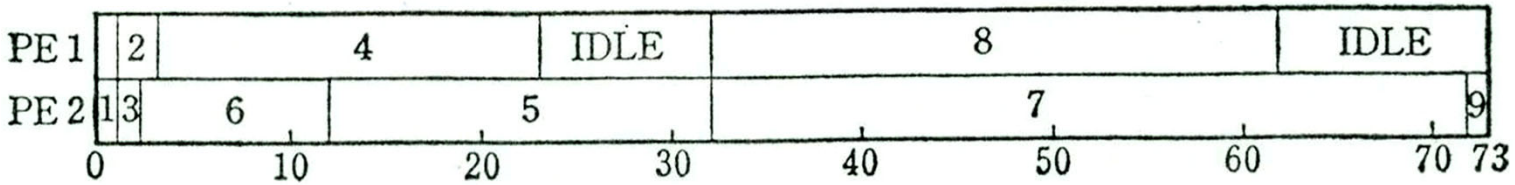


図3.23 CP法およびCP/MISF法を適用したときのスケジュール

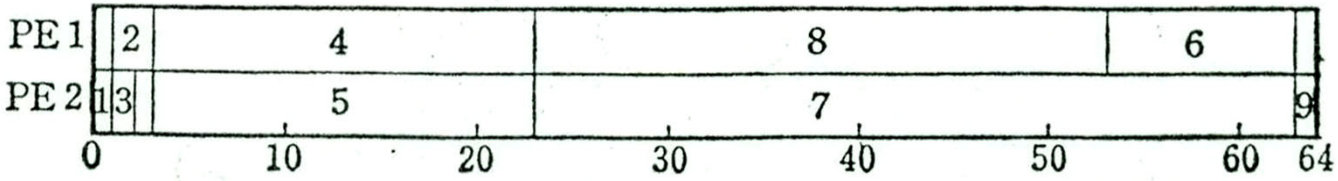
最適スケジューリングアルゴリズムの必要性



(a) リストスケジューリングでは最適解が得られないタスクグラフの例



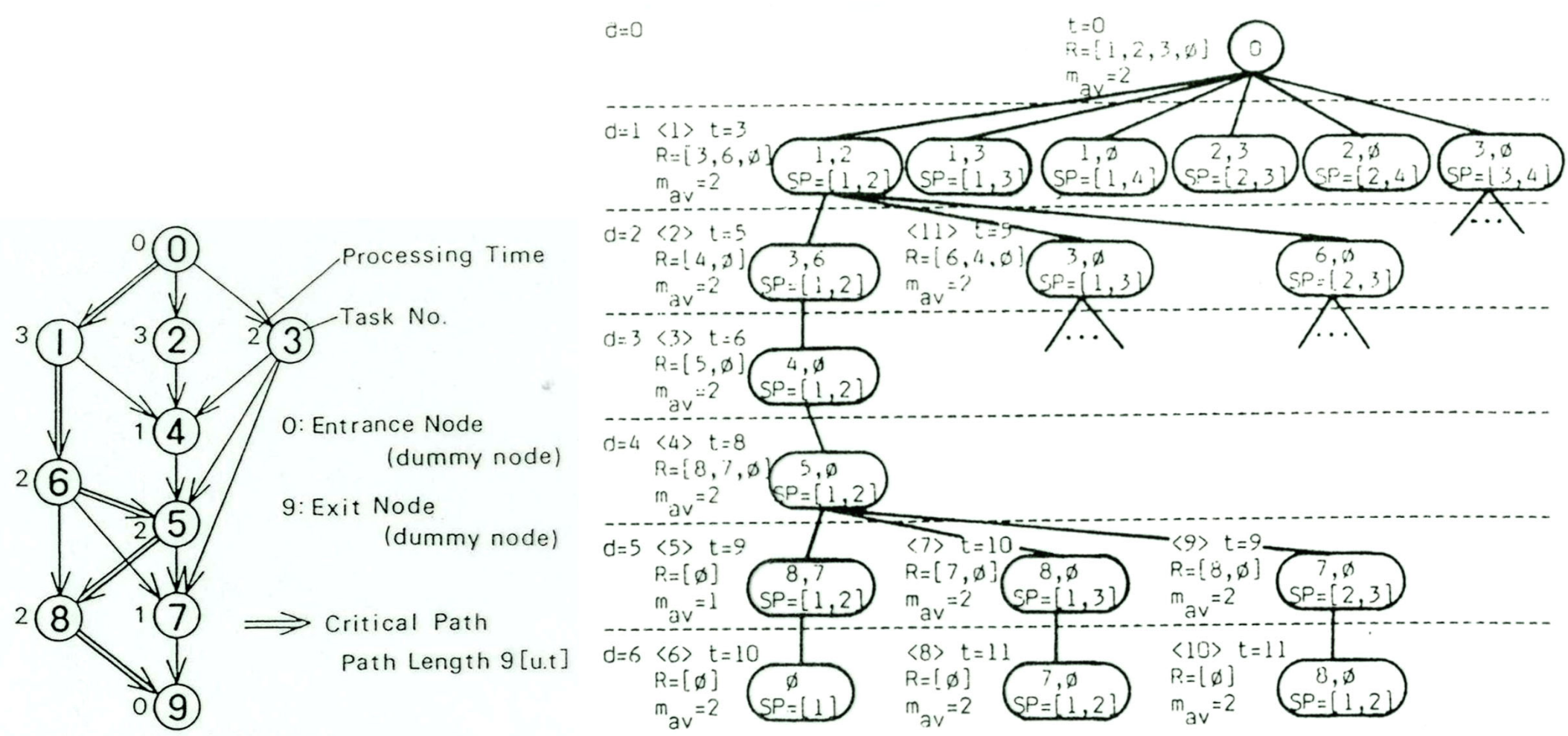
(b) リストスケジューリングにより得られるスケジュール



(c) 最適スケジュール

図3.24 最適スケジューリングアルゴリズムの必要性

タスクグラフとそのDF/IHSにおける分枝図



(a) A task graph with 8 tasks

(b) A partial search tree

図2　タスクグラフとそのDF/HISにおける分枝図

Fig.2 A task graph and a partial search tree for DF/IHS

CP/MISF法

Coffman Graham のアルゴリズム

↓

Ｈｕのアルゴリズム ⟶ ＣＰ法 ⟶ ＣＰ／ＭＩＳＦ法

（リストスケジューリング・アルゴリズム）

ＣＰ法：同レベルのタスクが複数ある時には優先順位を一意的に決定出来ない

レベル li

$$l_i = \max_k \sum_{j \in \pi_k} t_j$$

π_k : 出口ノードからノードNiにへ至るk番目のパス

CP/MISF法　レヴェルliの大きく直接の後続タスク数の多い順にプライオリティ・リストを作りリストスケジューリングを行う

CP/MISFの性能

(1)Adam等がCP法の評価を行ったのと同様、ランダムに生成した200の例題に対してCP/MISFを適用し得られた解と下限値を比較することにより平均的な問題に対する性能評価を行った

200ケース中　67%（最適）、87%（誤差5%以下）
98.5%（誤差10%以下）

(2)最悪の場合における性能の限界が保証されている

$$t \leq (2 - 1/m) \cdot t_{opt}$$

t　: CP/MISFにより得られる解

t_{opt}　: 最適解

近細粒度並列処理

- 基本ブロックをステートメントレベルの
  近細粒度タスクに分割
- タスク間のデータ依存を解析し，タスクグラフを生成
- 生成したタスクグラフに4種類のスケジューリング
  アルゴリズムを適用し，最短の処理時間を与えるスケジュールを用いて並列
  コード生成
- （各プロセッサ毎に異なるコードを生成）
  - CP/DT/MISF
  - DT/CP
  - CP/ETF/MISF
  - ETF/CP

https://ieeexplore.ieee.org/document/130111/similar#similar

Parallel processing of near fine grain tasks using static scheduling on OSCAR (optimally scheduled advanced multiprocessor)

Publisher: IEEE | Cite This | PDF

H. Kasahara ; H. Honda ; S. Narita All Authors

Published in: Supercomputing '90:Proceedings of the 1990 ACM/IEEE Conference on Supercomputing

https://ieeexplore.ieee.org/document/1653942

Journals & Magazines > Computer > Volume: 15 Issue: 2

A Second Opinion on Data Flow Machines and Languages

Publisher: IEEE | Cite This | PDF

Gajski ; Padua ; Kuck ; Kuhn All Authors

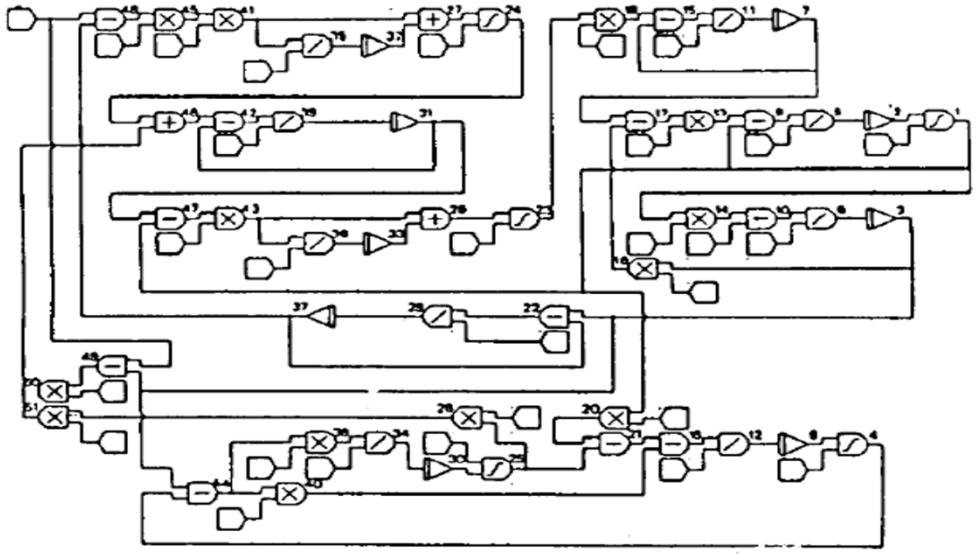


Fig.10 A kind of dataflow graph with near fine grain tasks

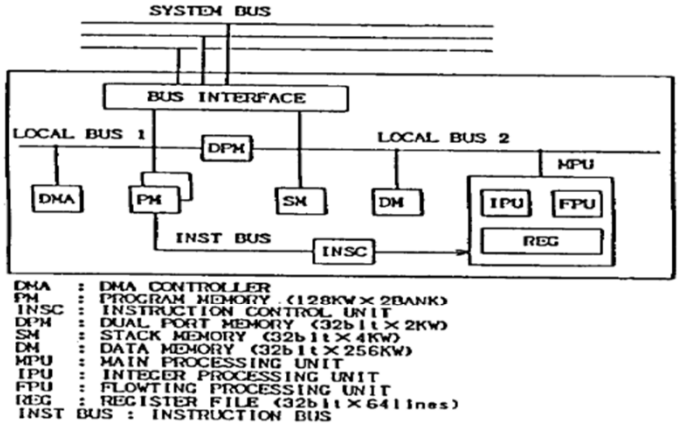


Fig.2 OSCAR's processor element

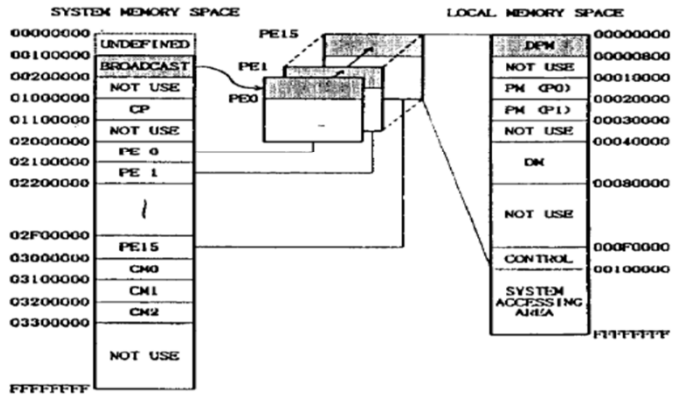


Fig.3 OSCAR's memory space

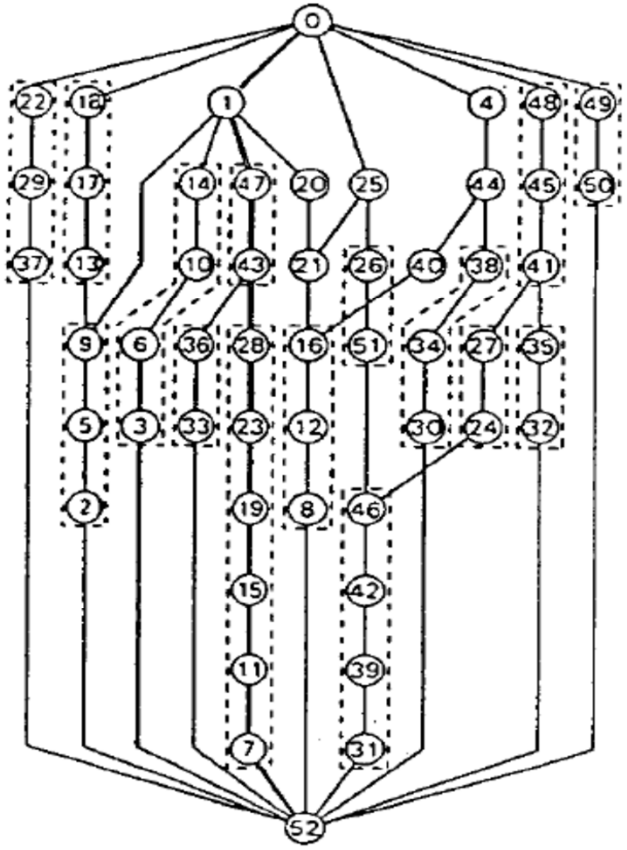
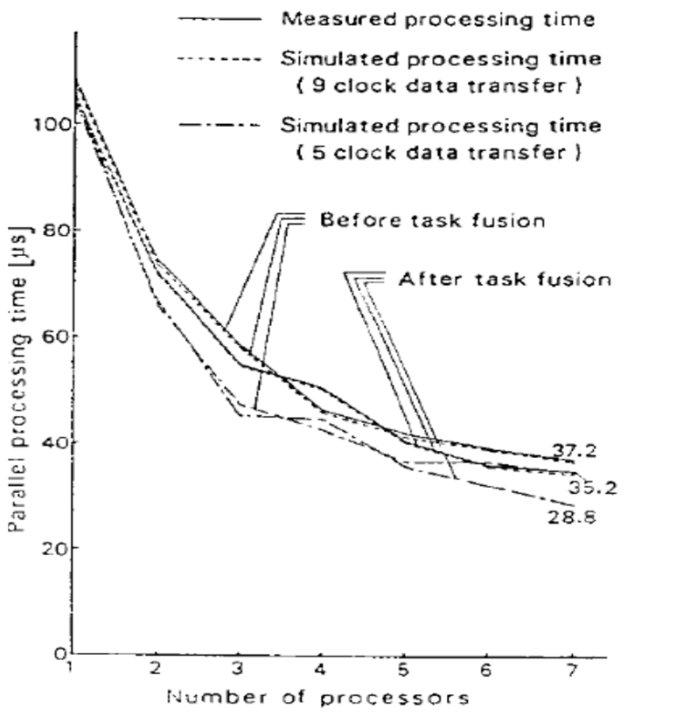


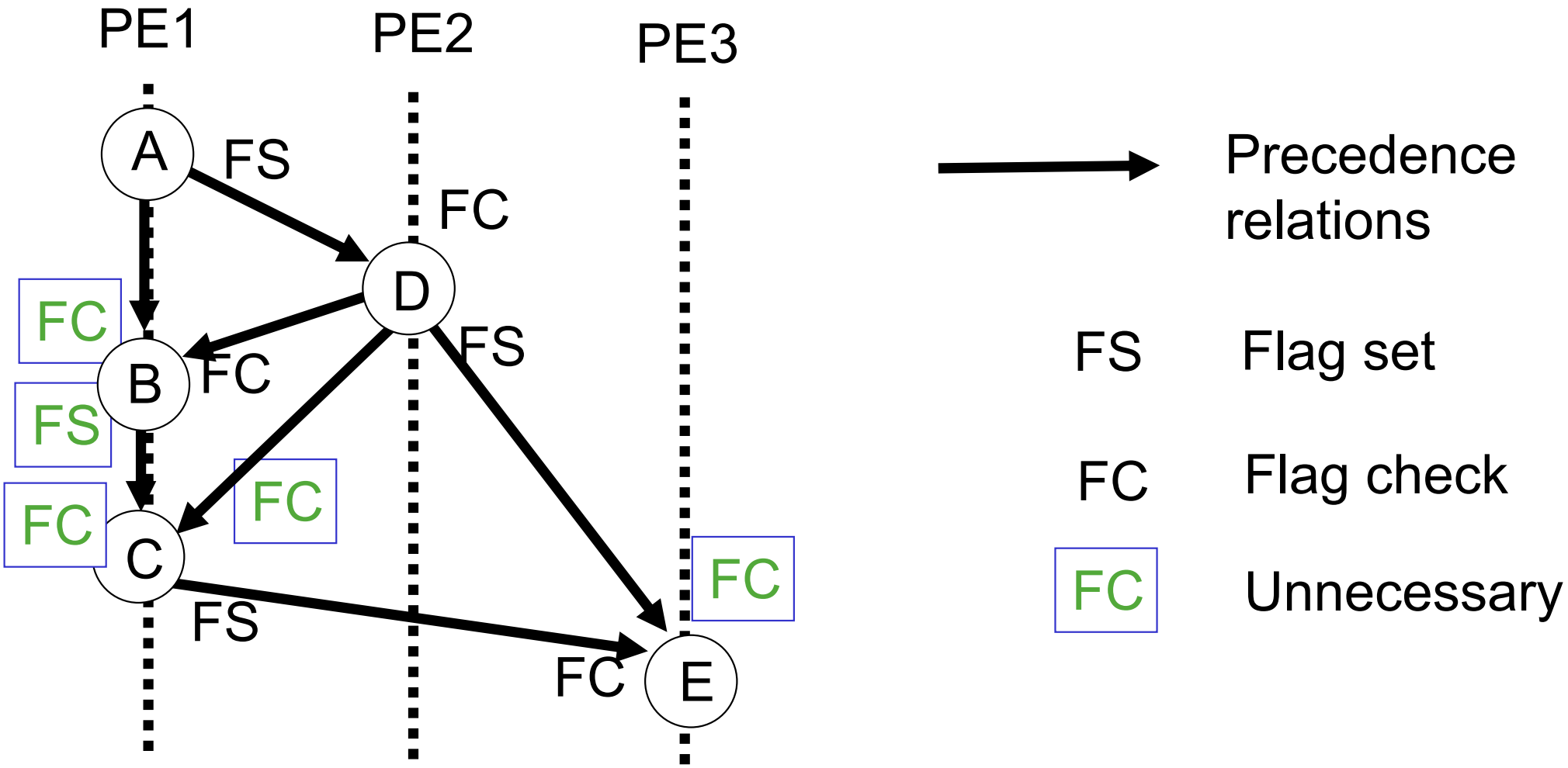
Fig.11 Task graph for Fig. 10



Parallel processing time of fig.10 on OSCAR

tioned in Section 3.1. The measured processing times were reduced from 108.7 [us] and 105.8 [us] for one PE to 37.2 [us] (1/2.92) and 35.2 [us](1/3.01) for seven PEs respectively.

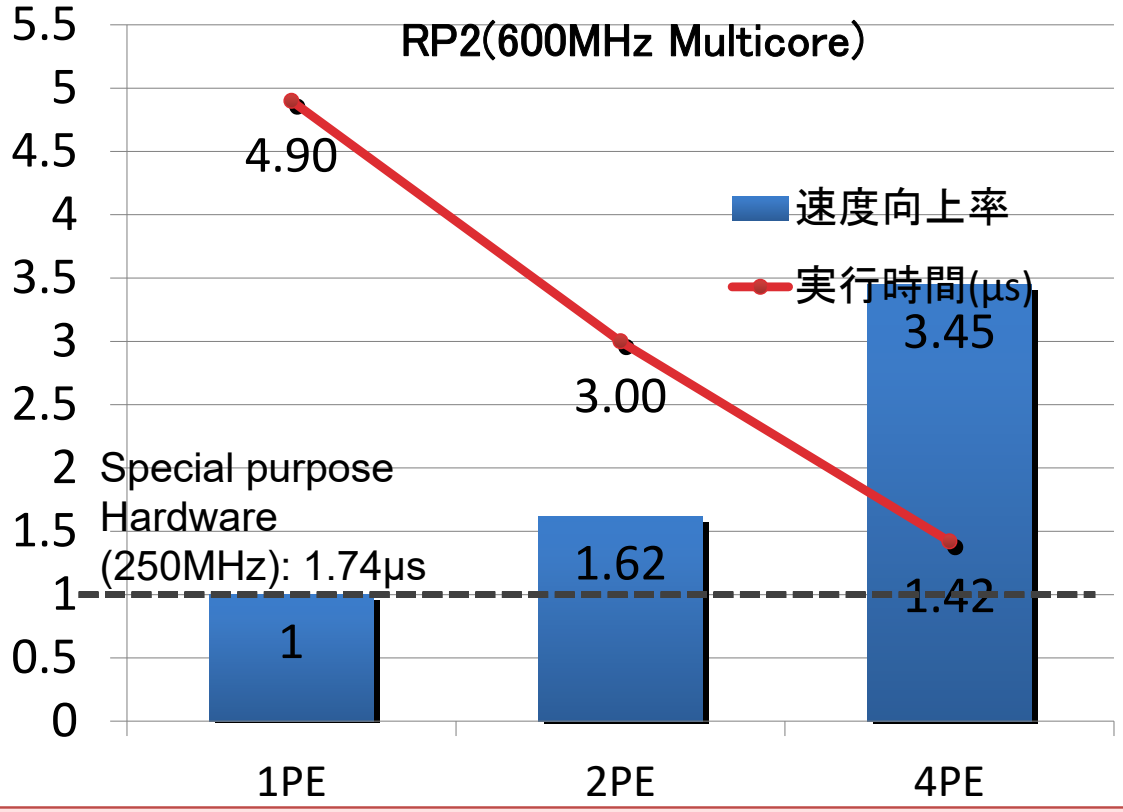
Elimination of Redundant Synchronization for Shared Data on Centralized Shared Memory after Static Task Scheduling



【W-CDMA Base Band Communication】

Near Fine Grain Parallel Processing of EAICH Detection Program on RP2 Multicore with 4 SH4A cores

- Hadamard transform often used in the signal processing
- Parallel Processing Method
  - Near fine grain parallel processing among statements
  - Static Scheduling



1.62 times speedup for 2cores, 3.45 times speedup for 4 cores for EAICH on RP2.

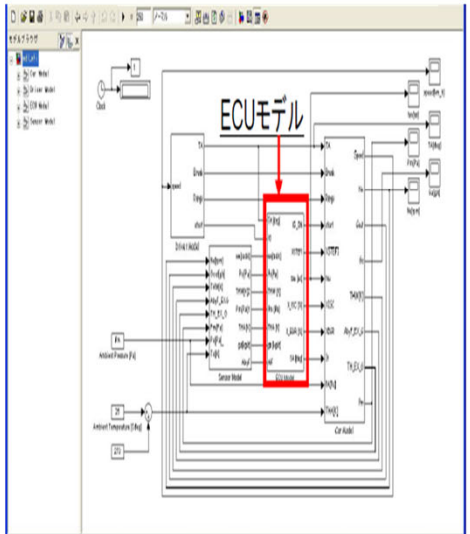
日本乗用車のエンジン制御計算をデンソー2コアECU上で、1．95倍の速度向上に成功。(見神、梅田）

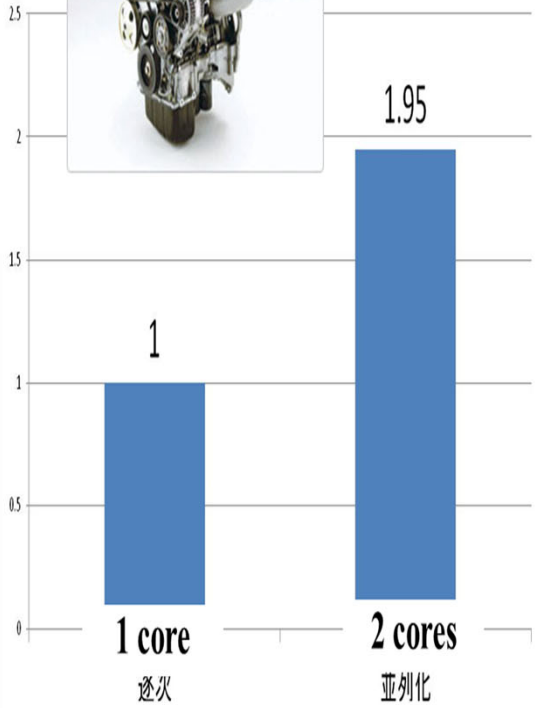
欧州農耕作業車エンジン制御計算をインフィニオン2コアプロセッサ上で8.7倍の高速化に成功。

Engine Control by multicore with Denso

Though so far parallel processing of the engine control on multicore has been very difficult, Denso and Waseda succeeded 1.95 times speedup on 2core V850 multicore processor.

- Hard real-time automobile engine control by multicore using local memories
- Millions of lines C codes consisting conditional branches and basic blocks

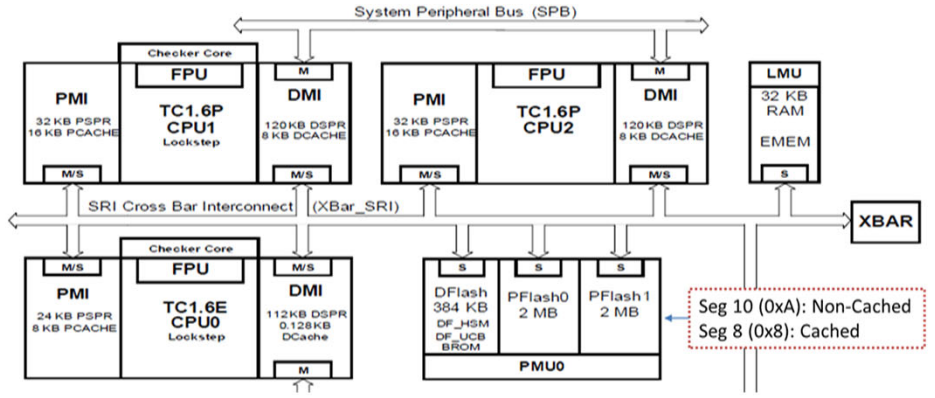




Automatic Parallelization of an Engine Control C Program with 400 thousands lines on AUTOSAR on 2 cores of Infineon AURIX TC277

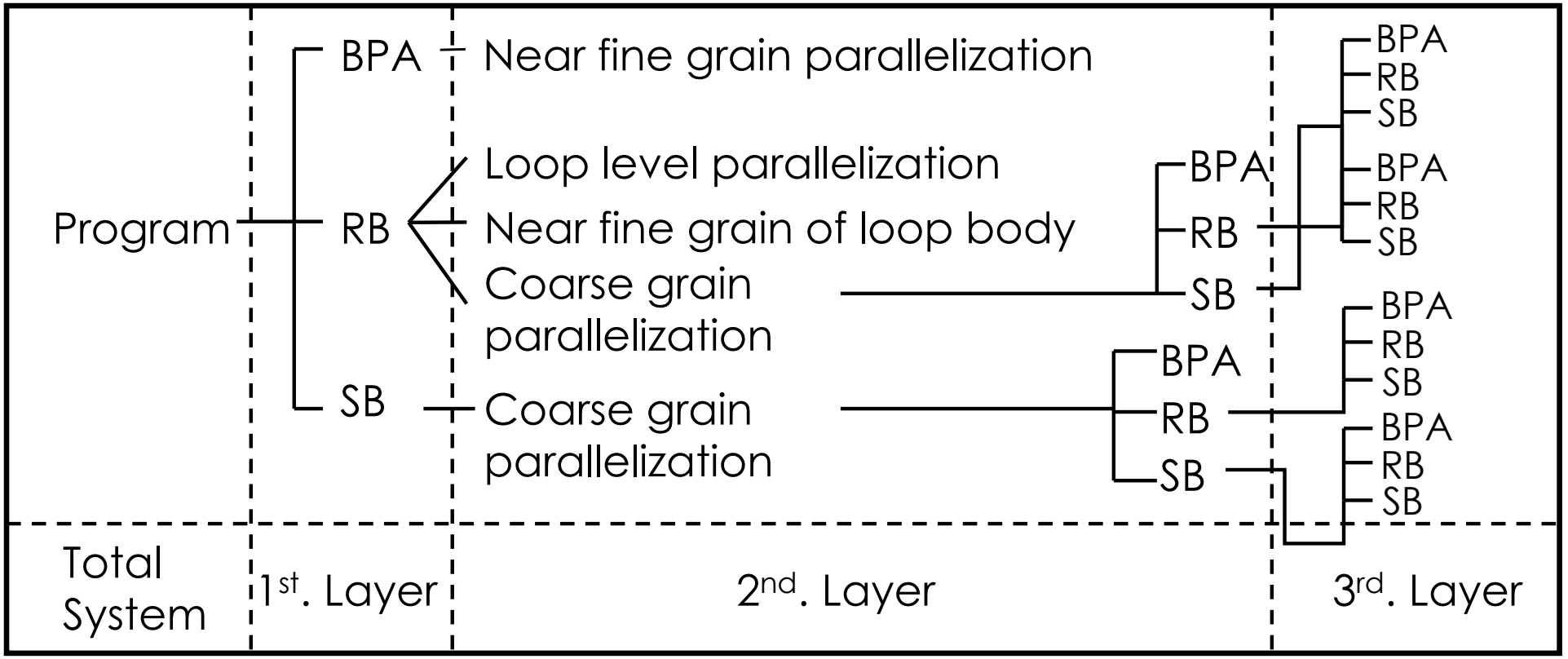
Infineon AURIX TC277

Abbreviations :
PCACHE: Program Cache
DCACHE: Data Cache
DSPR: Data Scratch-Pad RAM
PSPR: Program Scratch-Pad RAM
BROM: Boot ROM
PFlash: Program Flash
DFlash: Data Flash (EEEPROM)
S : SRI Slave Interface
M : SRI Master Interface

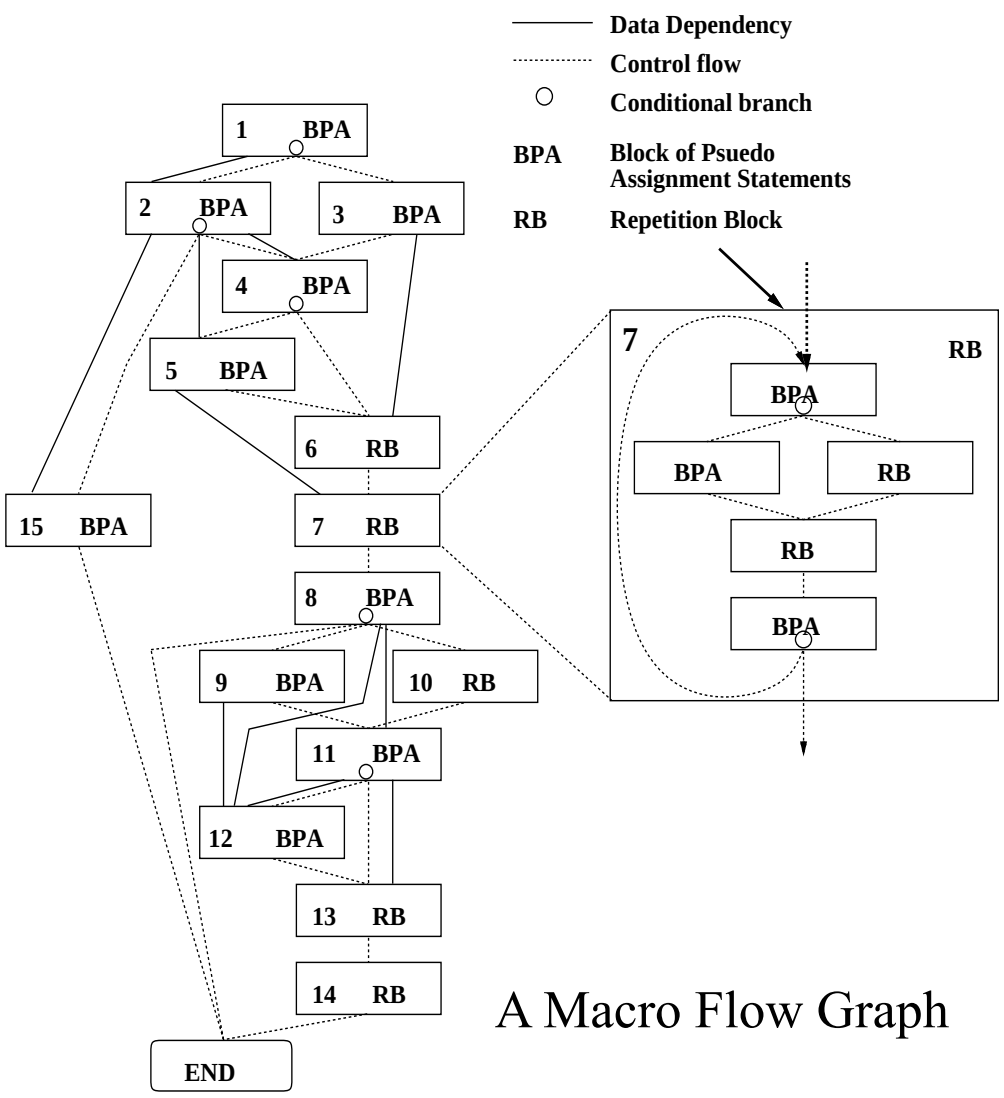


Generation of Coarse Grain Tasks

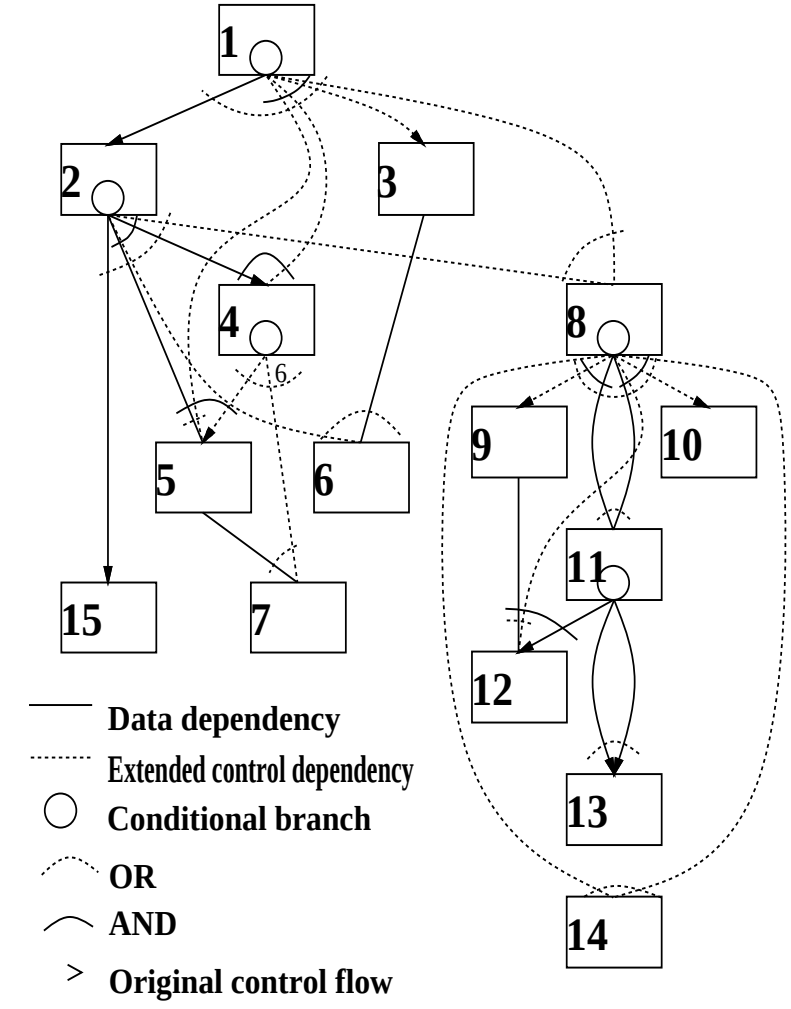
- Macro-tasks (MTs)
  - Block of Pseudo Assignments (BPA): Basic Block (BB)
  - Repetition Block (RB) : natural loop
  - Subroutine Block (SB): subroutine



Earliest Executable Condition Analysis for Coarse Grain Tasks (Macro-tasks)



A Macro Flow Graph



A Macro Task Graph

Earliest Executable Conditions

EEC: Control dependence + Data Dependence

Control dependences show executions of MTs are decided

Data dependences show data accessed by MTs are ready

MT2 may start execution after MT1 branches to MT2 and MT1 finish execution.

Macrotask No.	Earliest Executable Condition
1	
2	1_2
3	$(1)_3$
4	2_4 OR $(1)_3$
5	$(4)_5$ AND [2_4 OR $(1)_3$]
6	3 OR $(2)_4$
7	5 OR $(4)_6$
8	$(2)_4$ OR $(1)_3$
9	$(8)_9$
10	$(8)_{10}$
11	8_9 OR 8_{10}
12	11_{12} AND [9 OR $(8)_{10}$]
13	11_{13} OR 11_{12}
14	$(8)_9$ OR $(8)_{10}$
15	2_{15}

MT3 may start execution after MT1 branches to MT3.

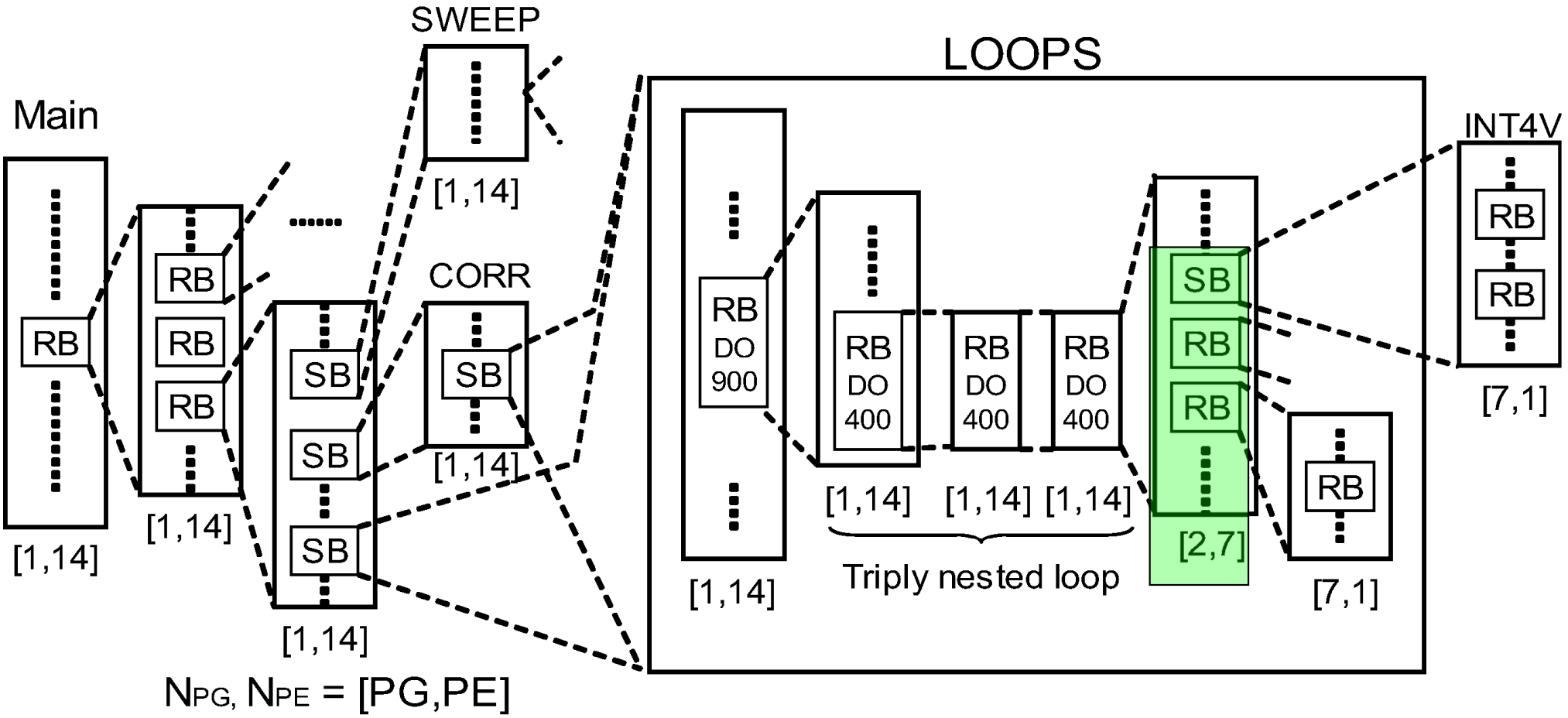
MT6 may start execution after MT3 finish execution or MT2 branches to MT4.

V1.0 © 2014, IEEE All rights reserved

Automatic processor assignment in 103.su2cor

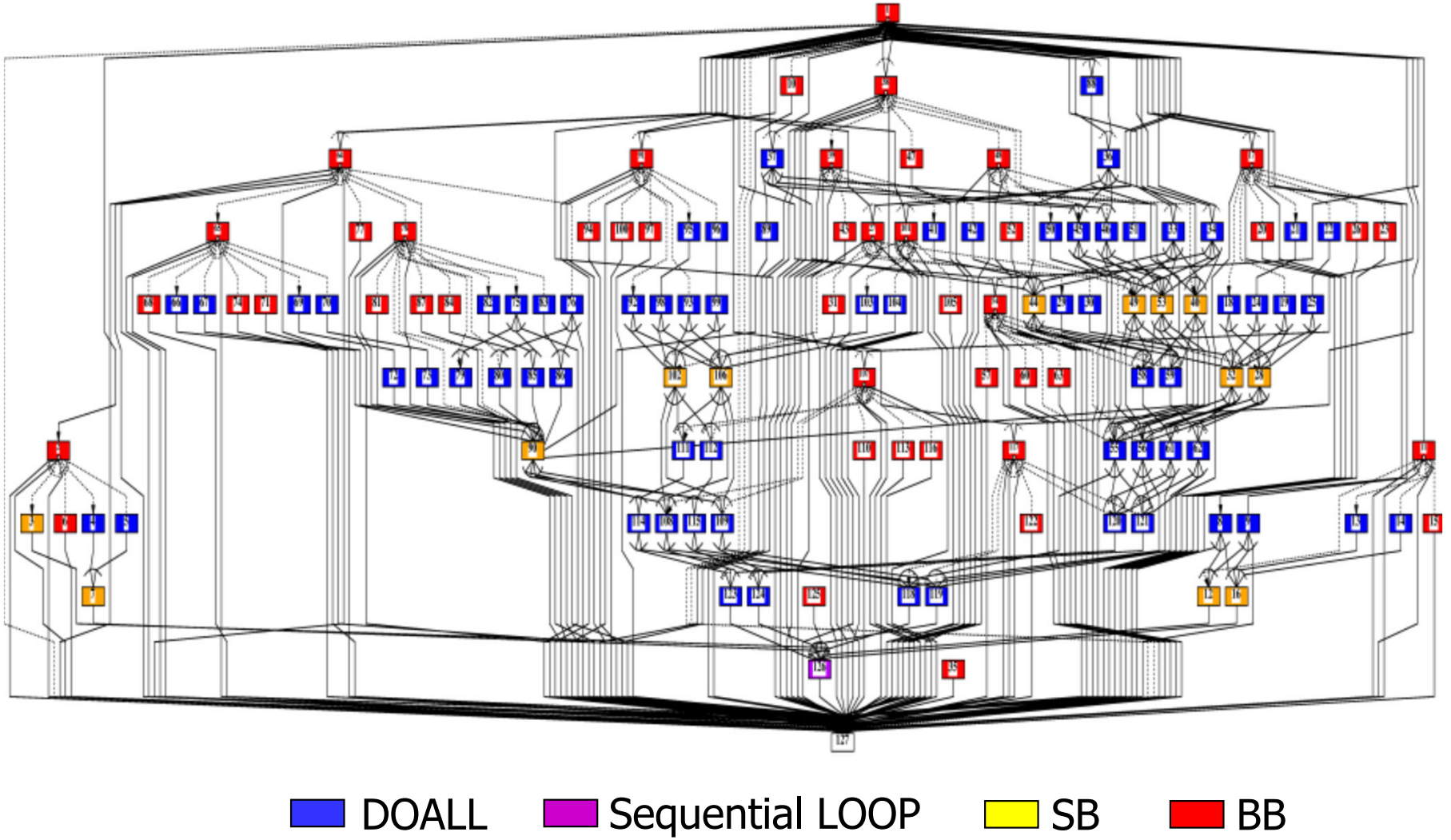
- **Using 14 processors**

Coarse grain parallelization within DO400

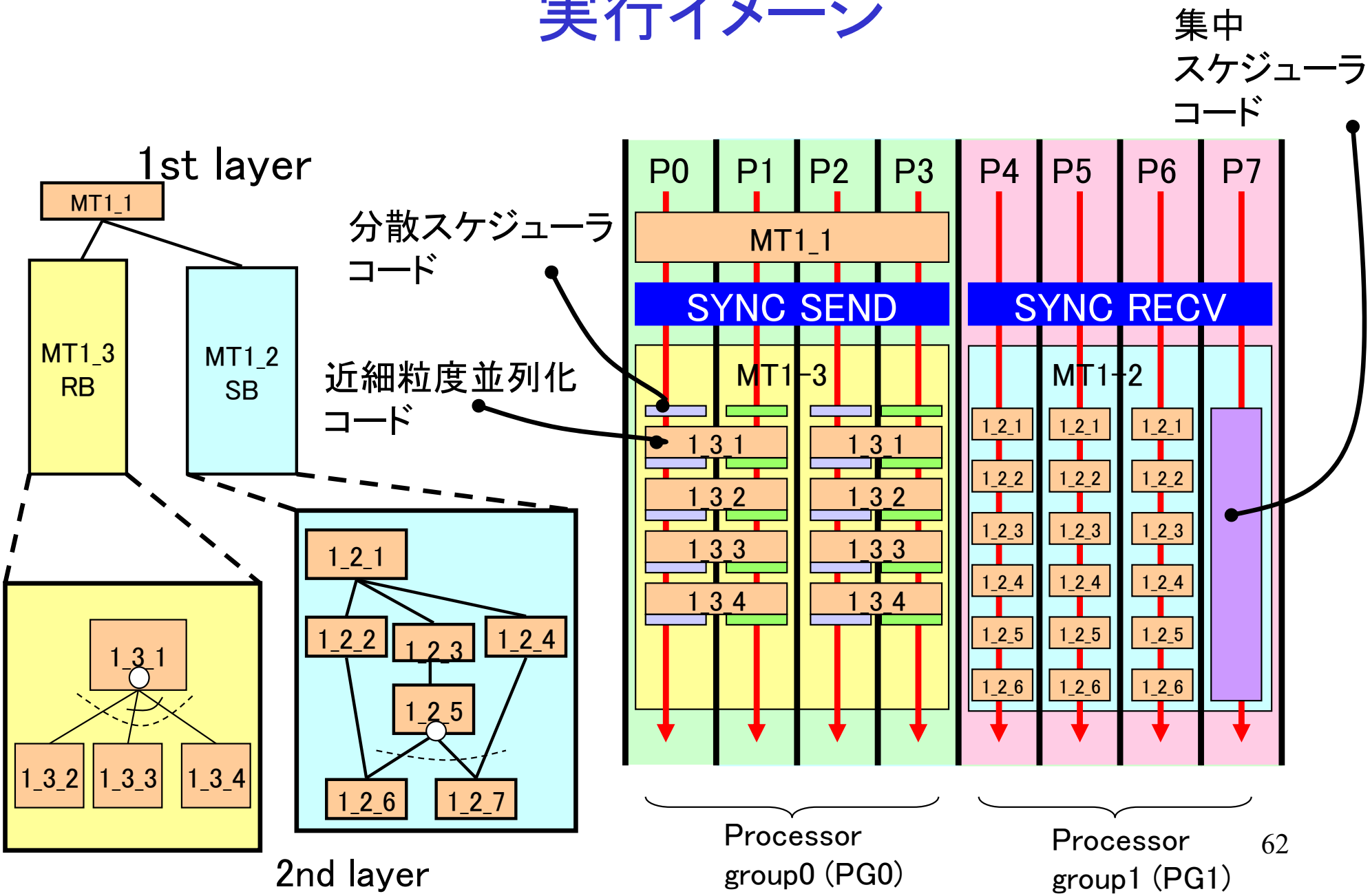


MTG of Su2cor-LOOPS-DO400

- Coarse grain parallelism PARA_ALD = 4.3



マルチグレイン並列処理の実行イメージ

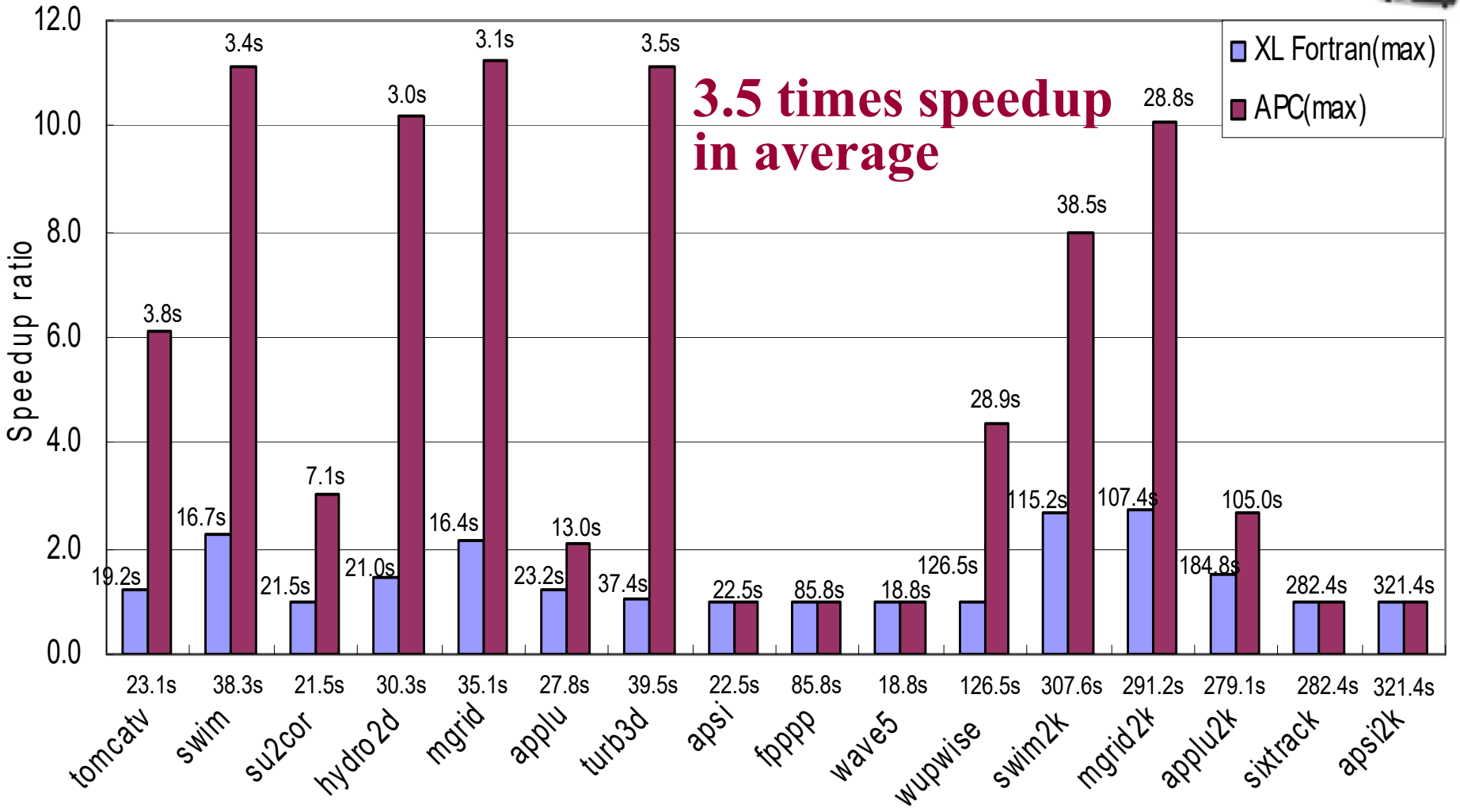


Code Generation Using OpenMP

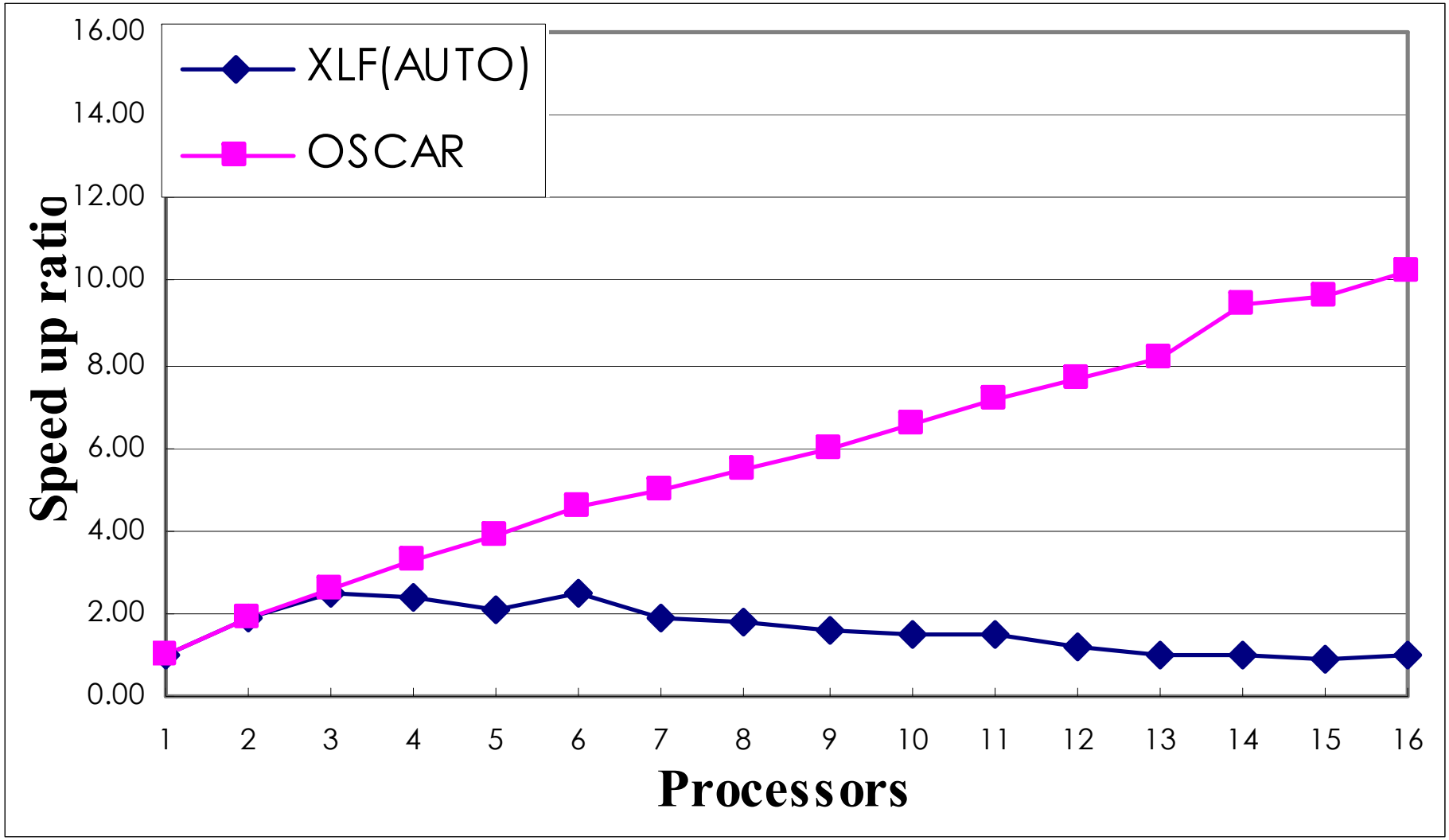
- **Compiler generates a parallelized program using OpenMP API**
- **One time single level thread generation**
  - **Threads are forked only once at the beginning of a program by OpenMP "PARALLEL SECTIONS" directive**
  - **Forked threads join only once at the end of program**
- **Compiler generates codes for each threads using static or dynamic scheduling schemes**
- **Extension of OpenMP for hierarchical processing is not required**

Performance of APC Compiler on IBM pSeries690 16 Processors High-end Server

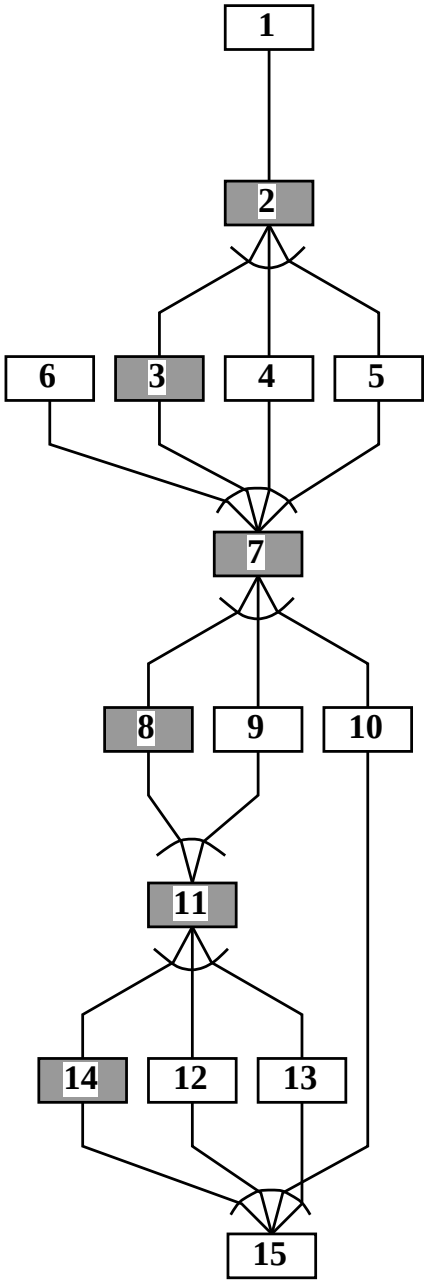
- IBM XL Fortran for AIX Version 8.1
  - Sequential execution : -O5 -qarch=pwr4
  - Automatic loop parallelization : -O5 -qsmp=auto -qarch=pwr4
  - OSCAR compiler : -O5 -qsmp=noauto -qarch=pwr4 (su2cor: -O4 -qstrict)



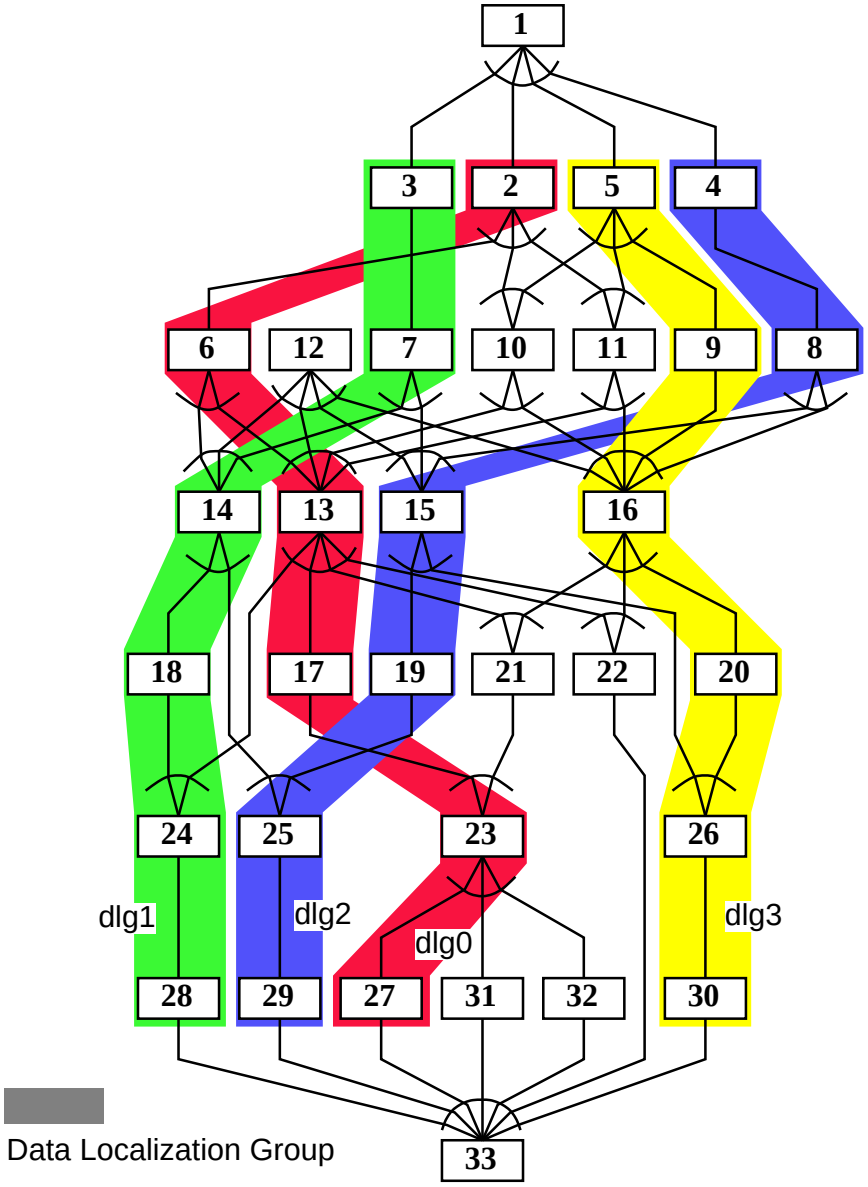
Performance of Multigrain Parallel Processing for 102.swim on IBM pSeries690



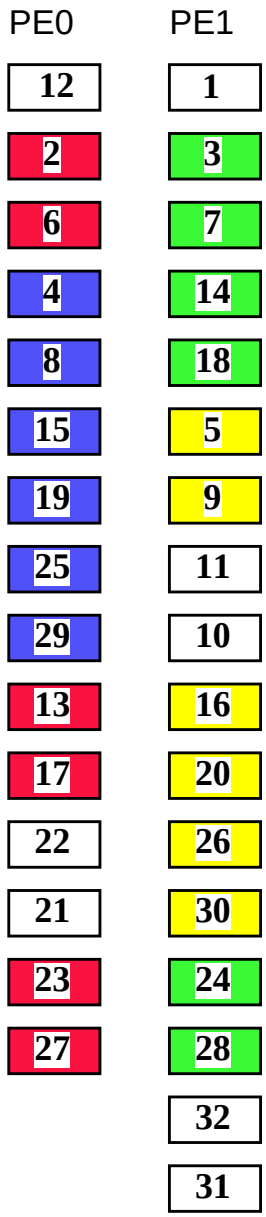
Data Localization



MTG



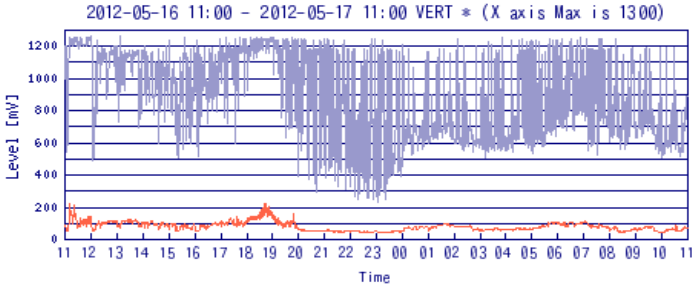
MTG after Division

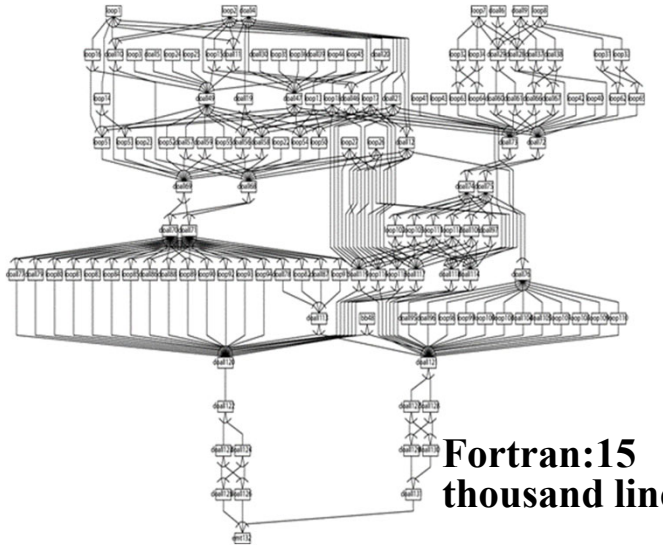


A schedule for two processors

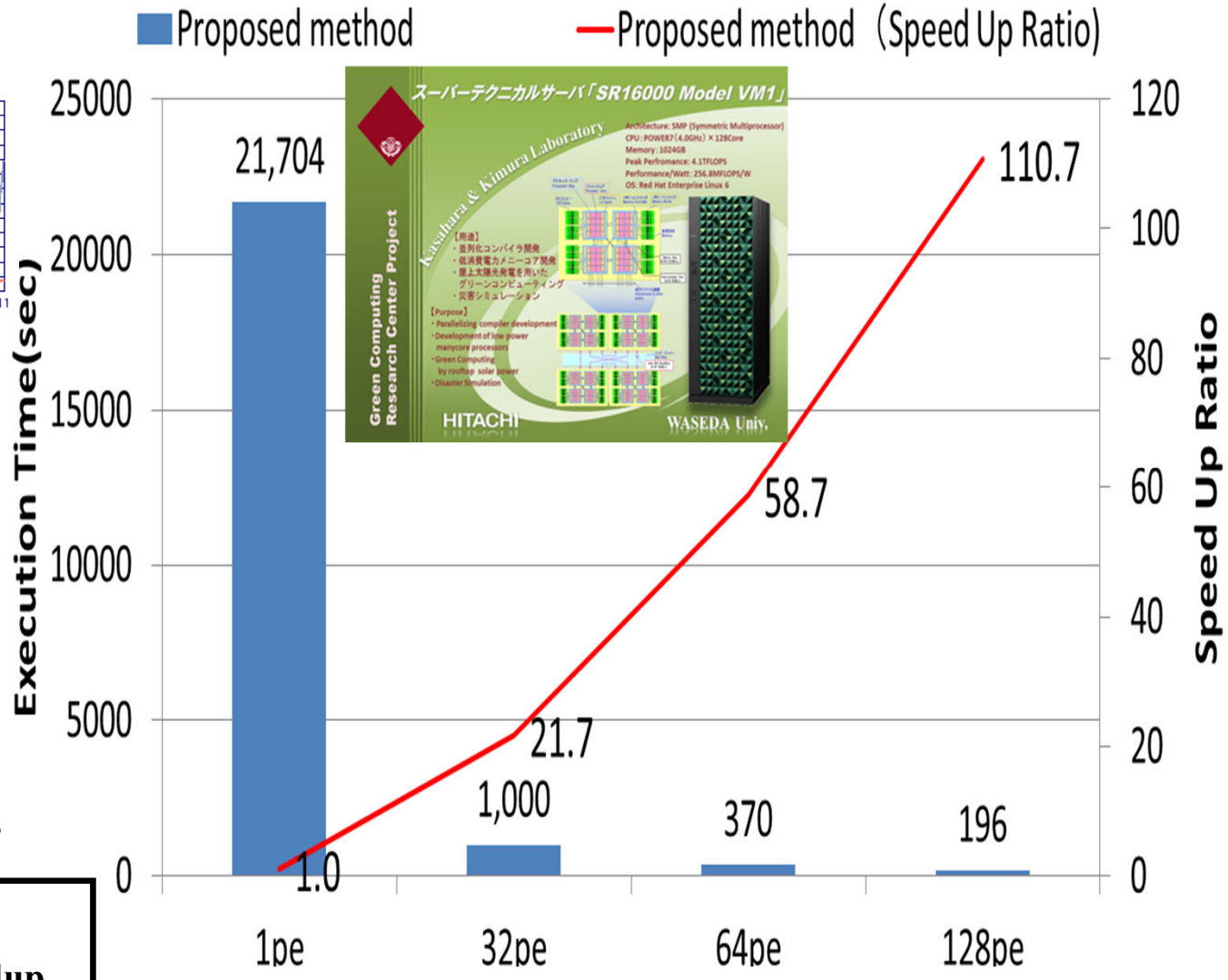
110 Times Speedup against the Sequential Processing for GMS Earthquake Wave Propagation Simulation on Hitachi SR16000

（Power7 Based 128 Core Linux SMP）(LCPC2015)





Fortran:15 thousand lines



First touch for distributed shared memory and cache optimization over loops are important for scalable speedup

スーパーテクニカルサーバ「SR16000モデルVM1」

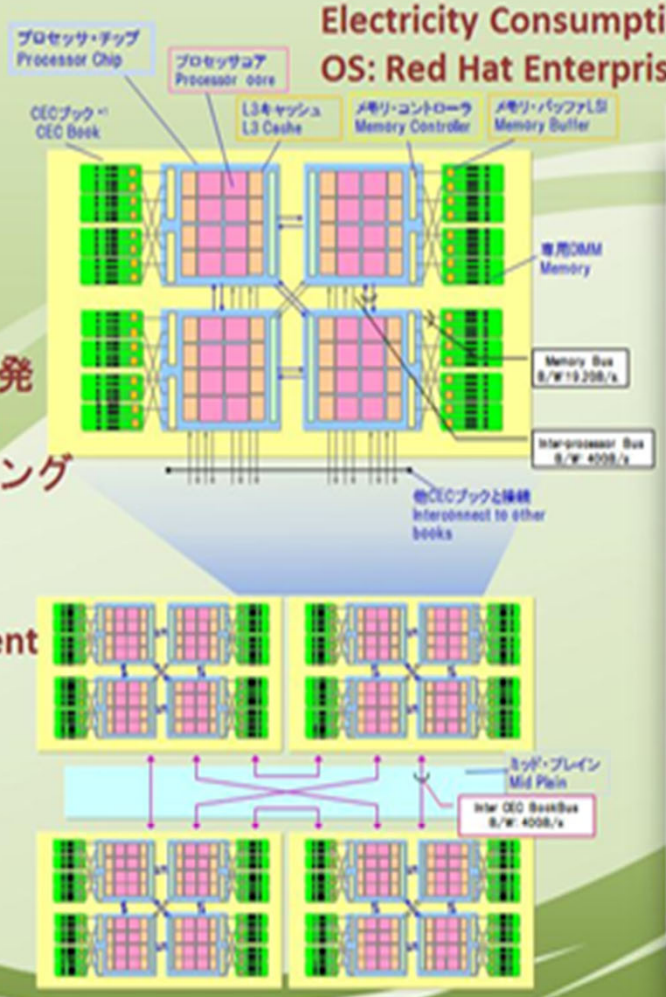
Kasahara & Kimura Laboratory

Green Computing
Research Center Project

CPU : POWER7（4.0GHz）×128Core
Memory : 1024GB
Peak Perfromance:4.1TFLOPS
Electricity Consumption : 256.8MFLOPS/W
OS: Red Hat Enterprise Linux 6

【用途】
・並列化コンパイラ開発
・低消費電力メニーコア開発
・屋上太陽光発電を用いた
グリーンコンピューティング
・災害シミュレーション

【Purpose】
・Parallelizing compiler development
・Development of low power
manycore processors
・Green Computing
by rooftop solar power
・Disaster Simulation

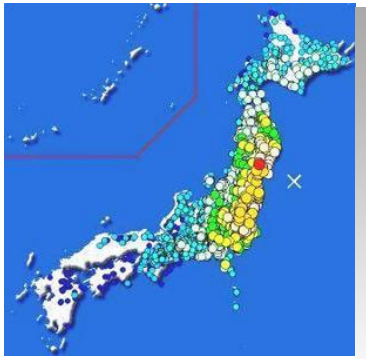


HITACHI

WASEDA Univ.

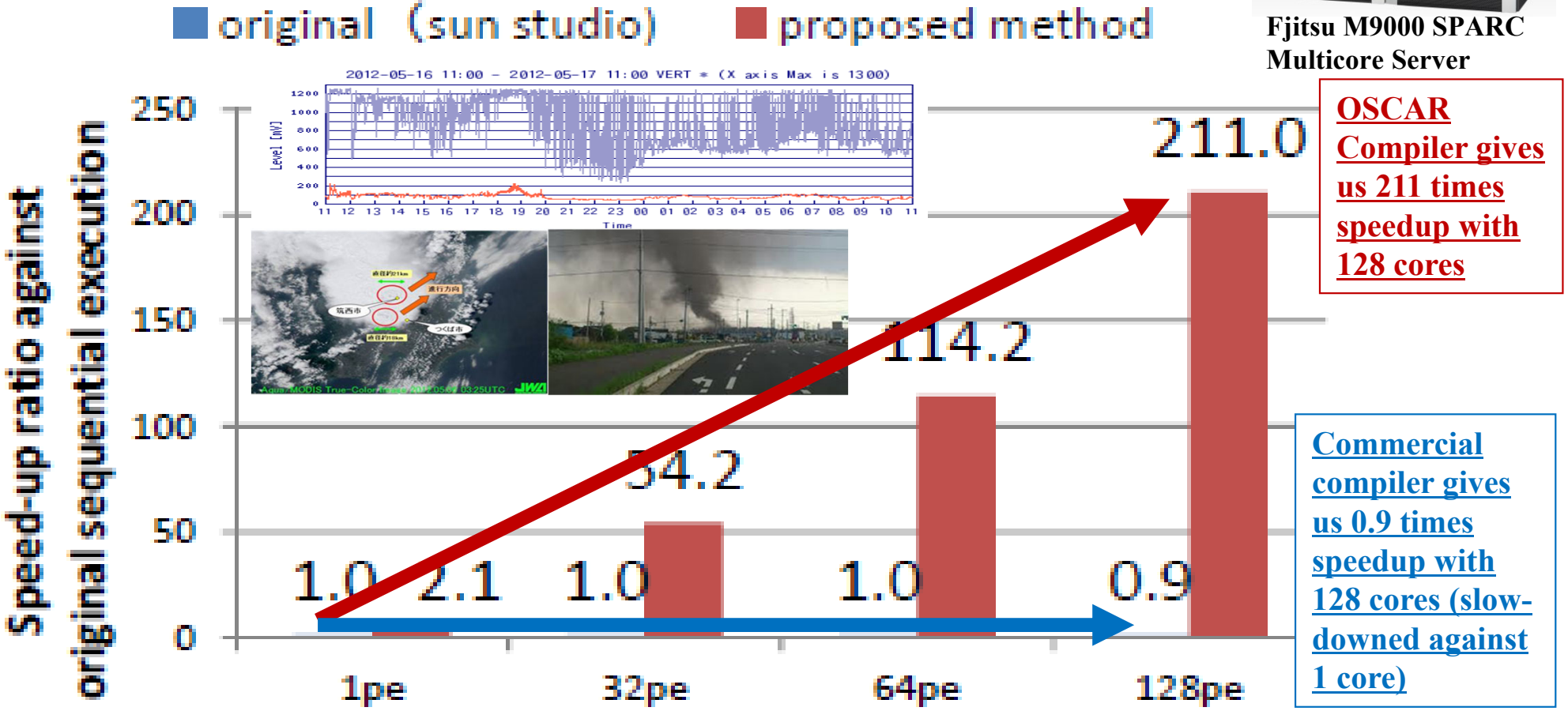
地震波伝搬シミュレーションの高速化：富士通スパコン128プロセッサコア上で、従来1コアより211倍の高速化に成功

- Just more cores don't give us speedup
- Development cost and period of parallel software are getting a bottleneck of development of embedded systems, eg. IoT, Automobile



Fjitsu M9000 SPARC Multicore Server

Earthquake wave propagation simulation GMS developed by National Research Institute for Earth Science and Disaster Resilience (NIED)

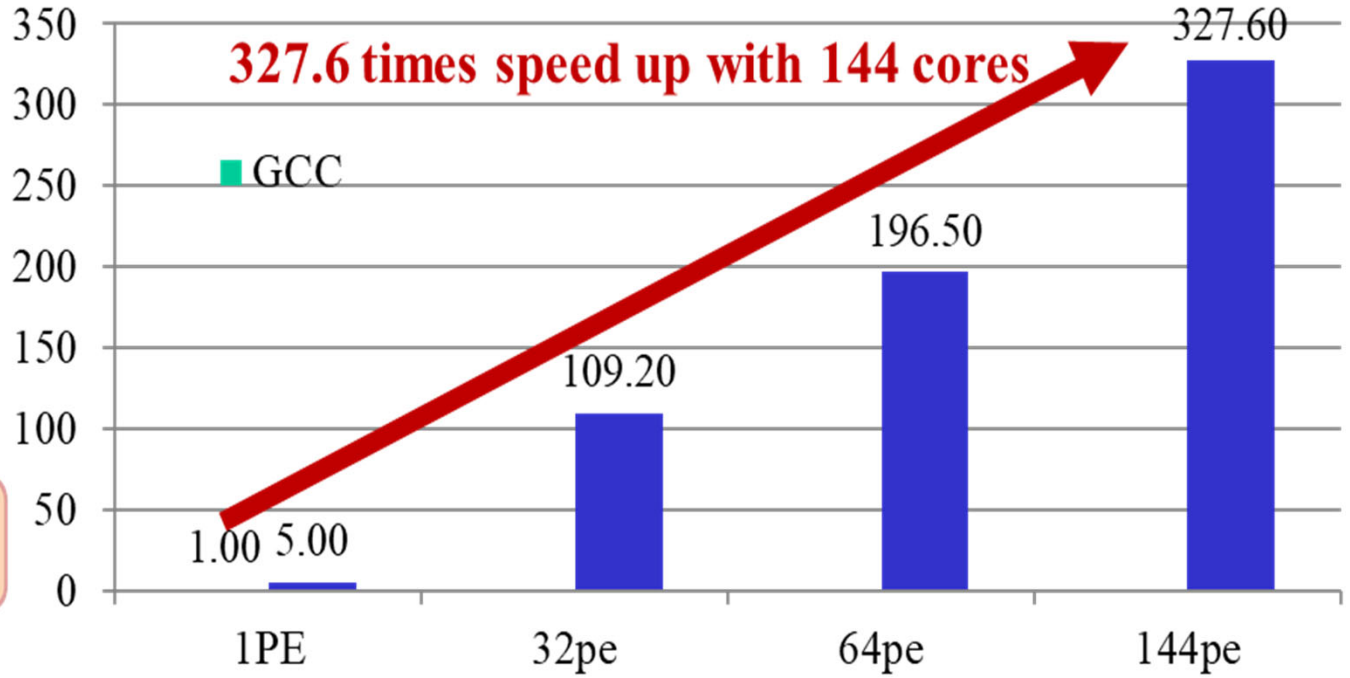


- Automatic parallelizing compiler available on the market gave us no speedup against execution time on 1 core on 64 cores
  - Execution time with 128 cores was slower than 1 core (0.9 times speedup)
- Advanced OSCAR parallelizing compiler gave us 211 times speedup with 128cores against execution time with 1 core using commercial compiler
  - OSCAR compiler gave us 2.1 times speedup on 1 core against commercial compiler by global cache optimization

Performance on Multicore Server for Latest Cancer Treatment Using Heavy Particle (Proton, Carbon Ion)

327 times speedup on 144 cores

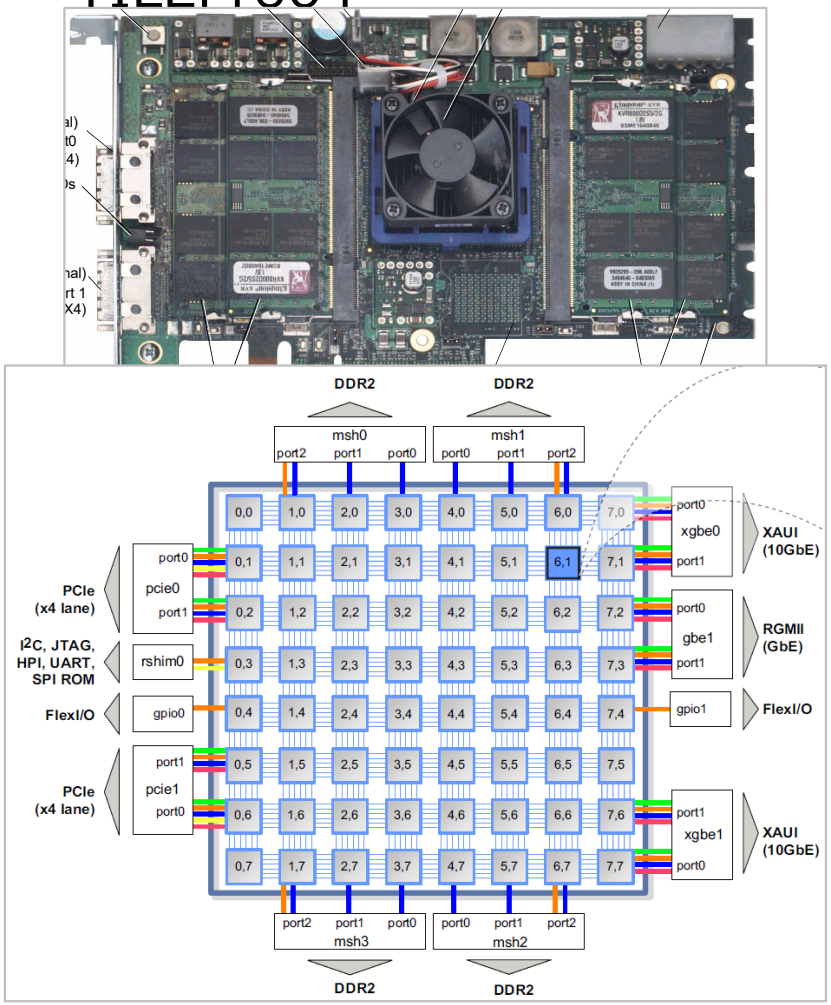
Hitachi 144cores SMP Blade Server BS500:
Xeon E7-8890 V3(2.5GHz　18core/chip) x8 chip

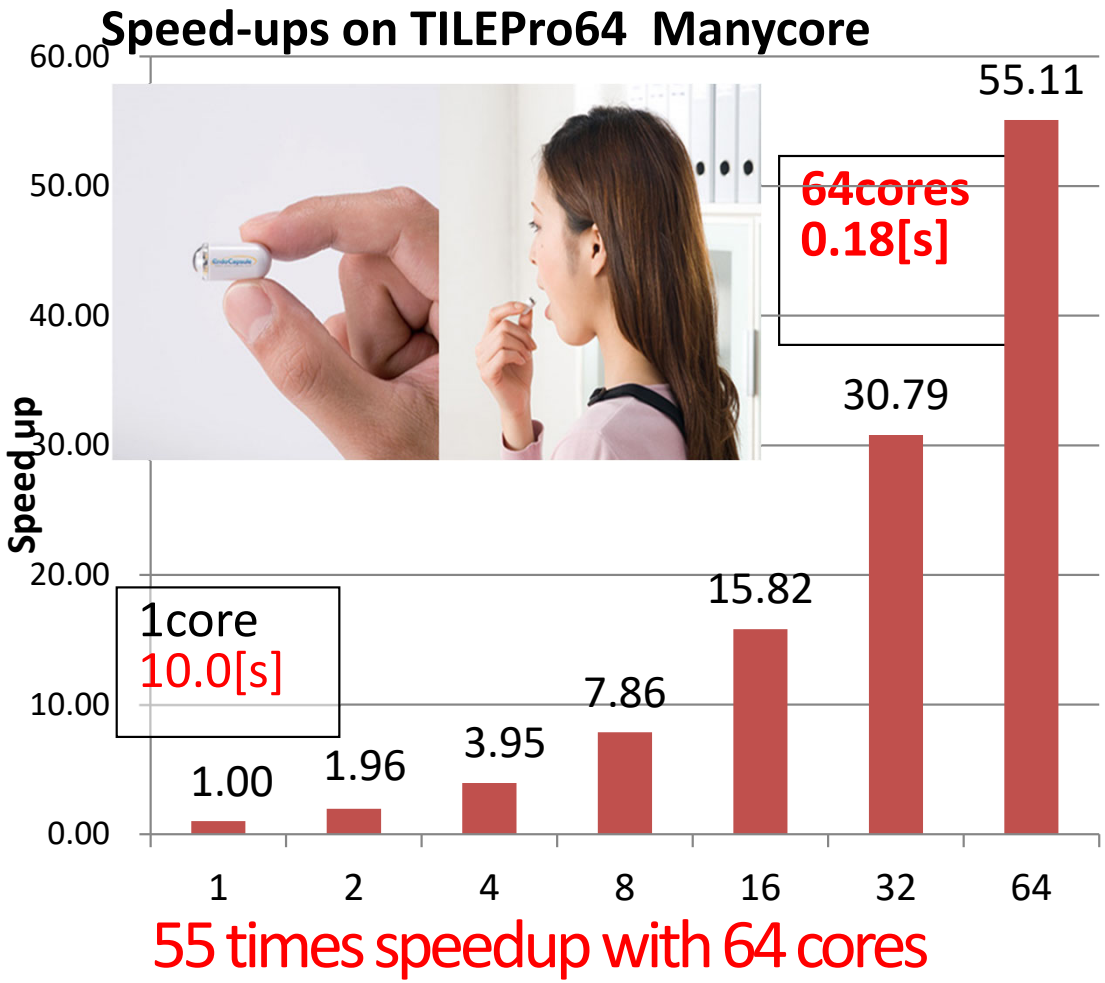


- Original sequential execution time 2948 sec（50 minutes）using GCC was reduced to 9 sec with 144 cores（327.6 times speedup）
  - Reduction of treatment cost and reservation waiting period is expected

Automatic Parallelization of JPEG-XR for Drinkable Inner Camera (Endo Capsule) on Tilera

- TILEPro64

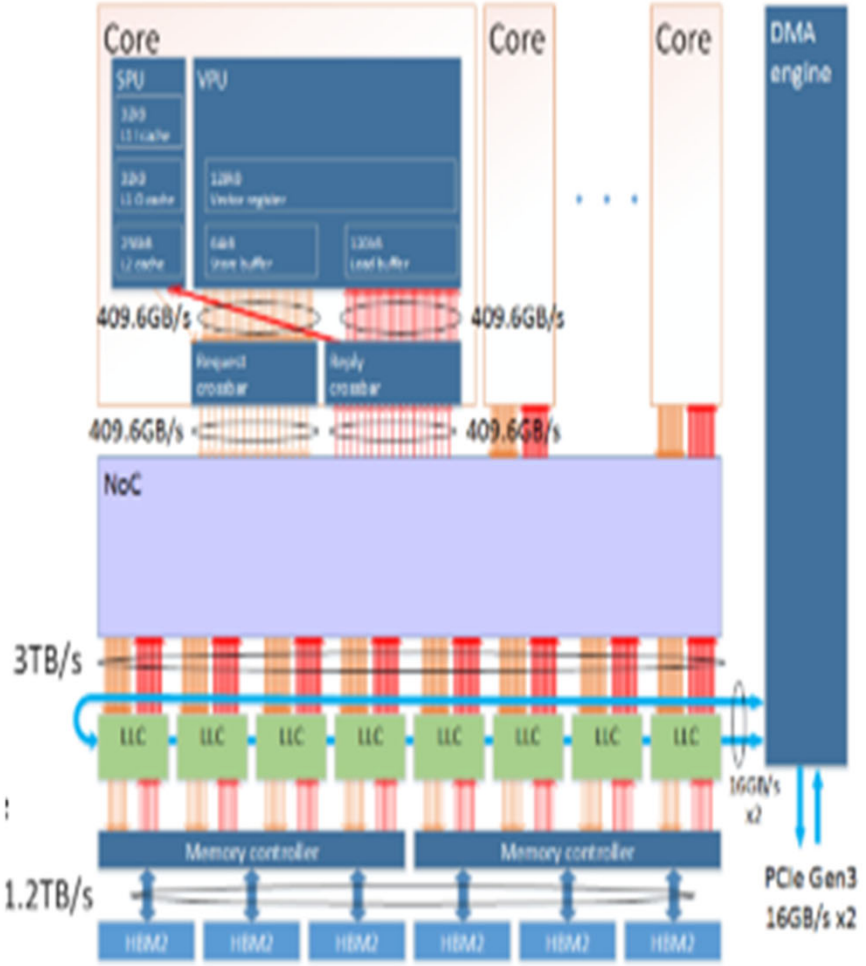


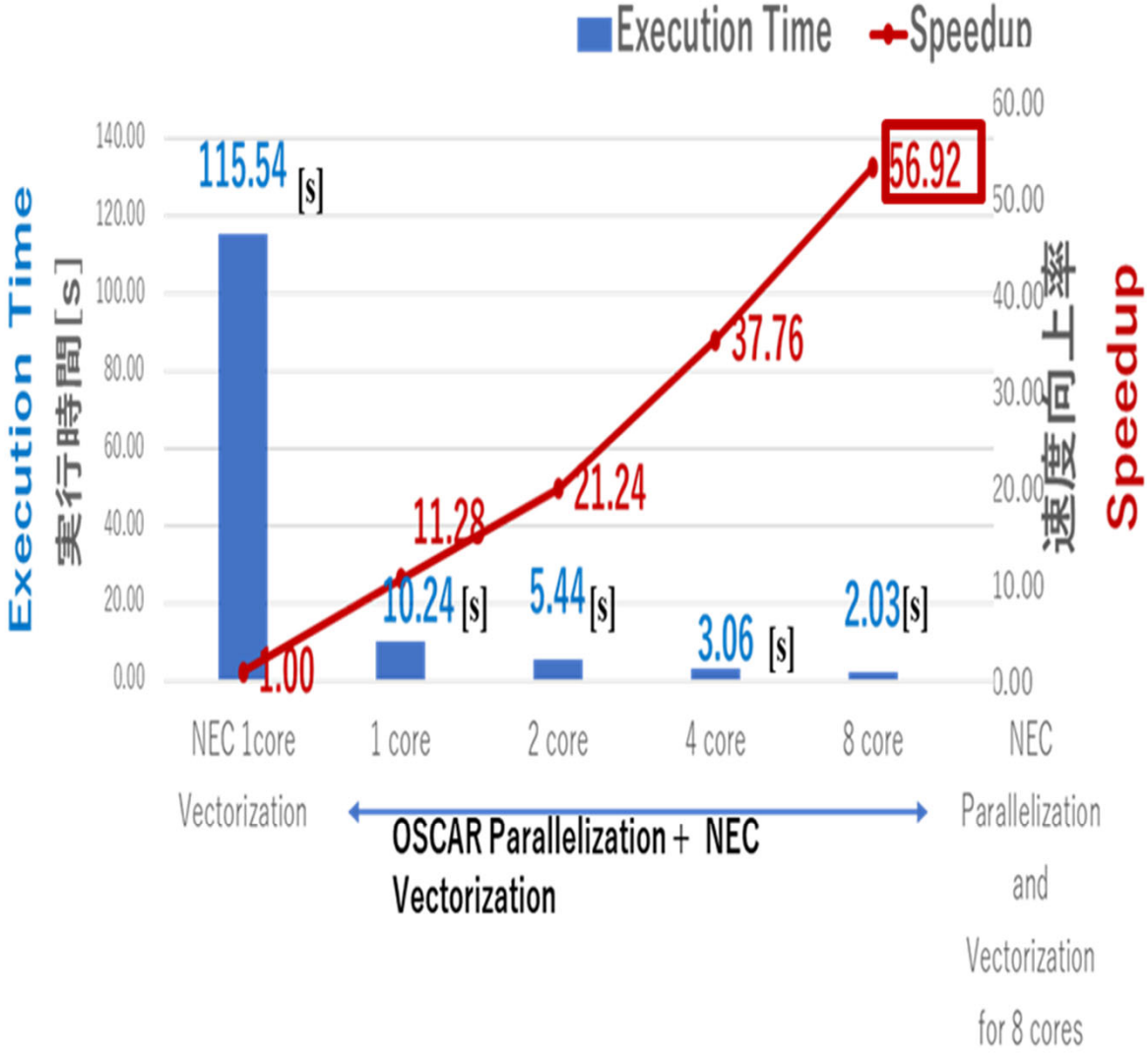


55 times speedup with 64 cores

Speedups of NPB/CG Scientific Code by OSCAR Compiler on NEC SX-Aurora TSUBASA A100-1 8 cores 10C VE

57 times speedup for 8 vector cores by OSCAR Parallelization & NEC Vectorization against NEC 1 core Vectorization



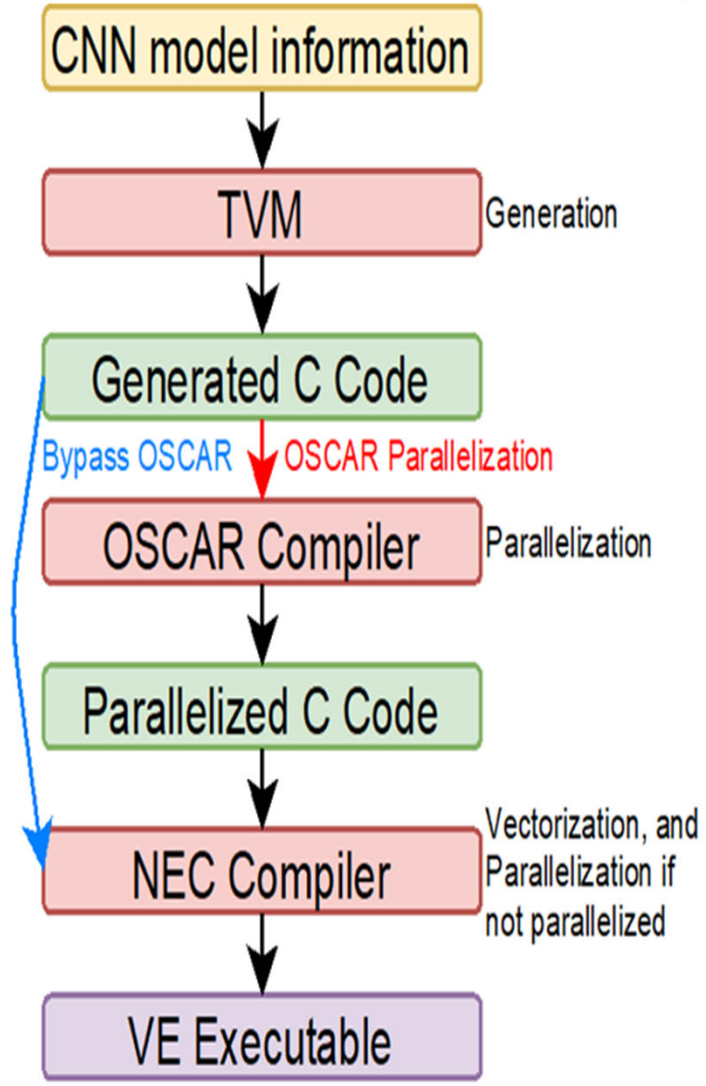


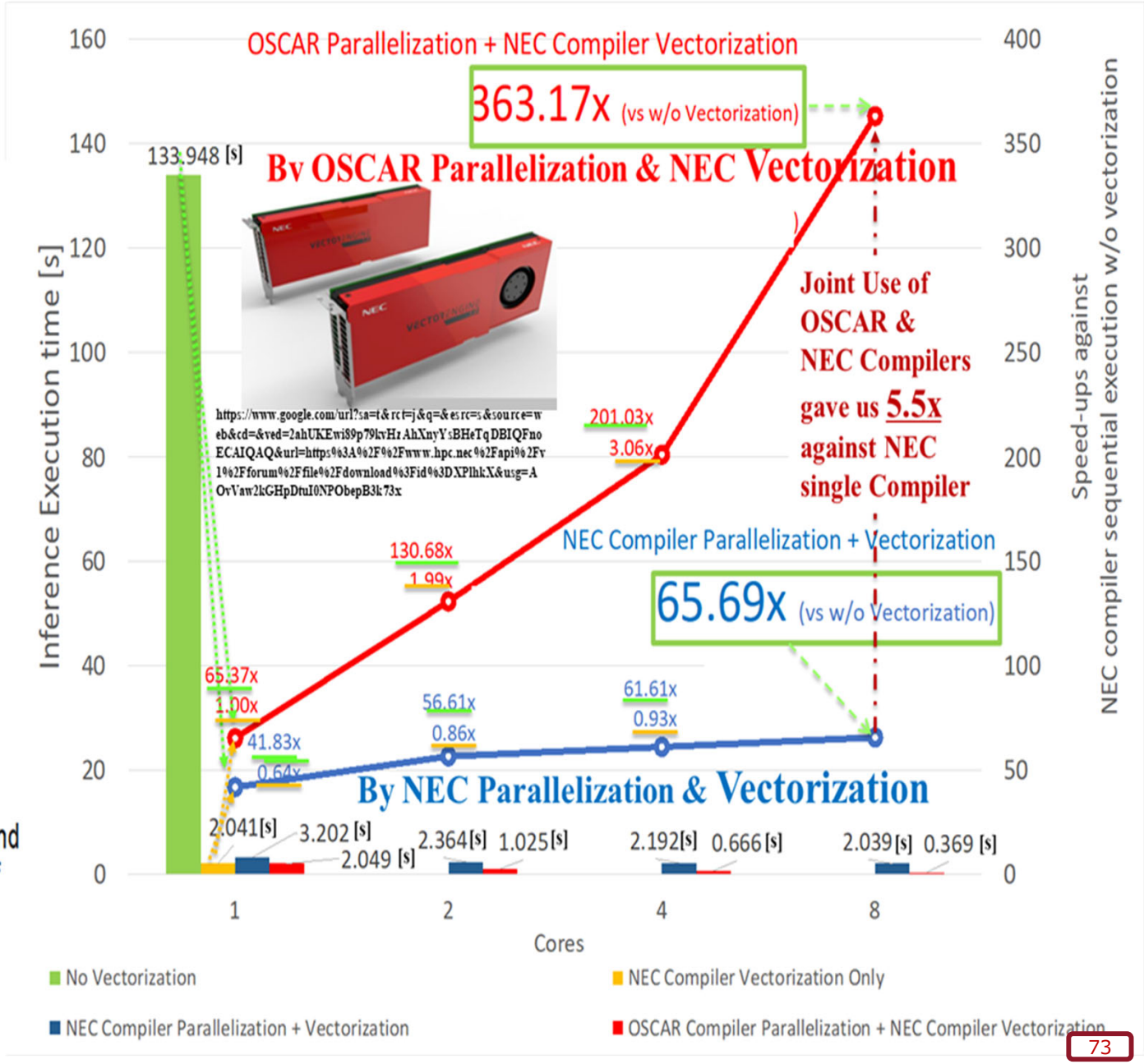
NEC Corporation. (2021) SX-Aurora TSUBASA Performance Tuning Guide. [Online]. Available: https://www.hpc.nec/documents/guide/pdfs/AuroraVE TuningGuide.pdf

Speedups of Deep Learning Winograd 2D-Convolution generated by TVM on NEC Personal Vector Supercomputer SX-Aurora TSUBASA 8 Core Type 10C

OSCAR Parallelization and NEC Vectorization gave us 363x Speedup against a Scalar Core

Parallelization of Deep Leaning C Code generated by TVM





INDUSTRY

https://www.ans.org/news/article-5842/amazon-buys-nuclearpowered-data-center-from-talen/

Amazon buys nuclear-powered data center from Talen

Thu, Mar 7, 2024, 10:01PM | Nuclear News

Susquehanna nuclear plant in Salem Township, Penn., along with the data center in foreground. (Photo: Talen Energy)

Talen Energy announced its sale of a 960-megawatt data center campus to cloud service provider Amazon Web Services (AWS), a subsidiary of Amazon, for $650 million.

The data center, Cumulus Data Assets, sits on a 1,200-acre campus in Pennsylvania and is directly powered by the adjacent Susquehanna Steam Electric Station, which generates 2.5 gigawatts of power.

Laramie County approves construction of what could become the largest data center in US

Wyoming is poised to become an artificial-intelligence powerhouse after Laramie County commissioners earlier this month unanimously voted to move forward with the construction of a 1.8 gigawatt data center designed to eventually scale up to 10 gigawatts, which would be the largest single AI campus in the U.S.

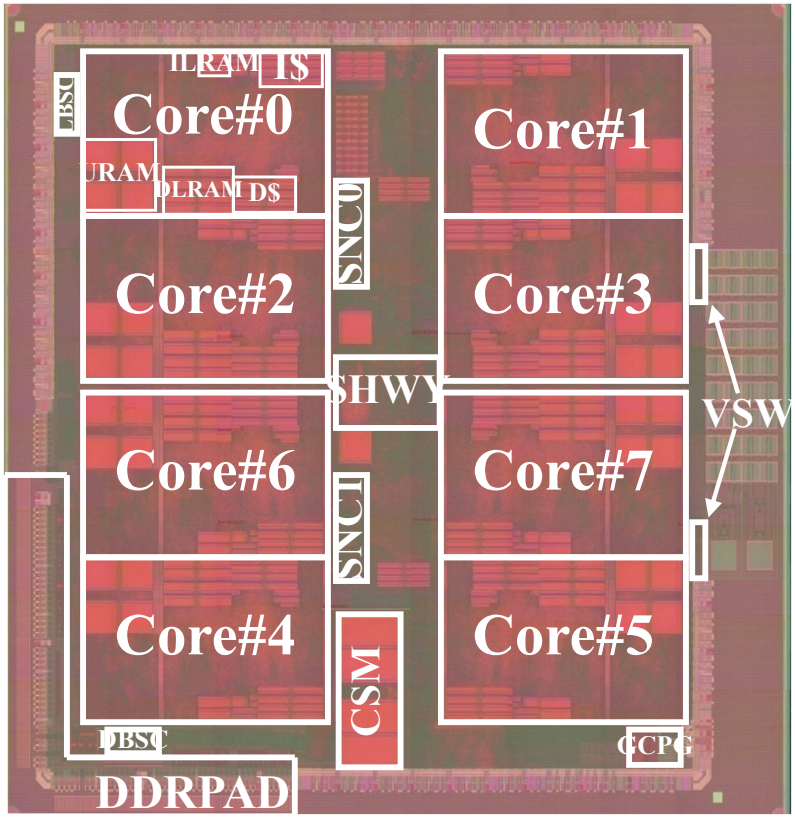
January 20, 2026

https://wyofile.com/laramie-county-approves-construction-of-what-could-become-the-largest-data-center-in-us/

ムーアの法則の終焉 (End of Moore's Law)

ムーアの法則(Moore's law): インテル創業者の一人であるゴードン・ムーア(Gordon E. Moore: IEEE Computer Pioneer Award)が、1965年の論文で提唱した経験則:
"半導体の集積率は18か月で2倍になる" "Transistors on a chip 2× every 1.5 year"

コンピュータの高性能化と低消費電力化にはマルチコアが必須



IEEE ISSCC08: Paper No. 4.5, M.ITO, … and H. Kasahara, "An 8640 MIPS SoC with Independent Power-off Control of 8 CPUs and 8 RAMs by an Automatic Parallelizing Compiler"

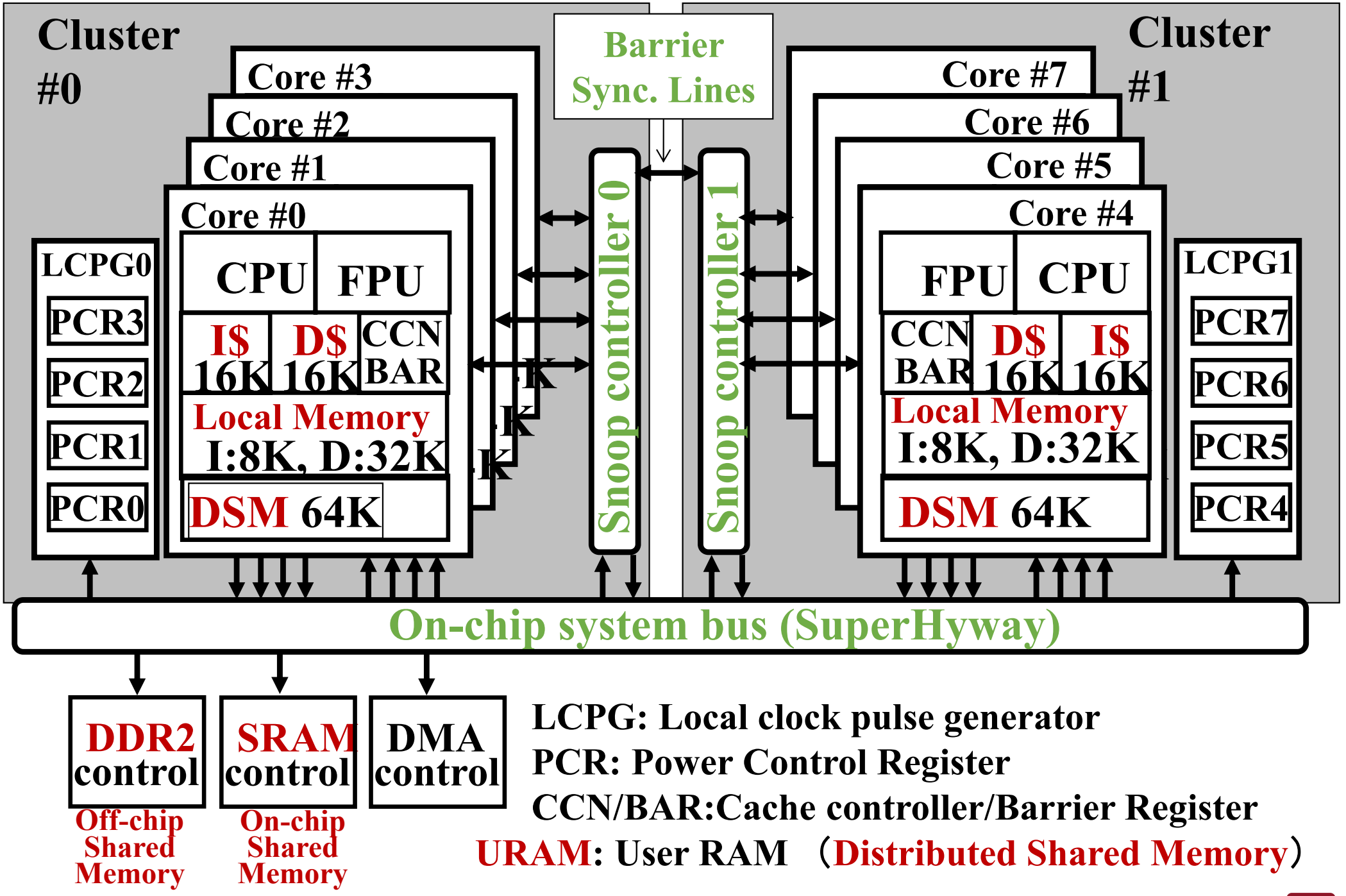
Power ∝ Frequency * Voltage^2

⇒ (Voltage ∝ Frequency)

Power ∝ Frequency^3

周波数 Frequency を1/4にすると
(Ex. 4GHz→1GHz),
性能は 1/4、消費電力は 1/64
Performance 1/4, Power 1/64
<マルチコア(Multicore)>
8cores をチップに集積すると,
Performance 2 timesでPowerは 1/8

8 Core RP2 Chip Block Diagram

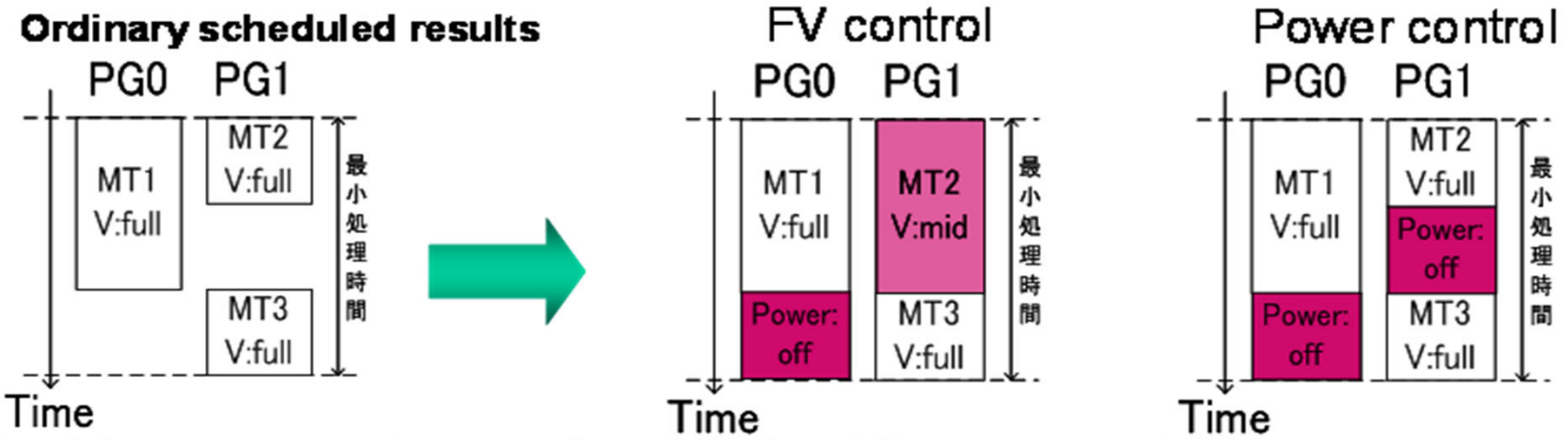


LCPG: Local clock pulse generator
PCR: Power Control Register
CCN/BAR:Cache controller/Barrier Register
URAM: User RAM （Distributed Shared Memory）

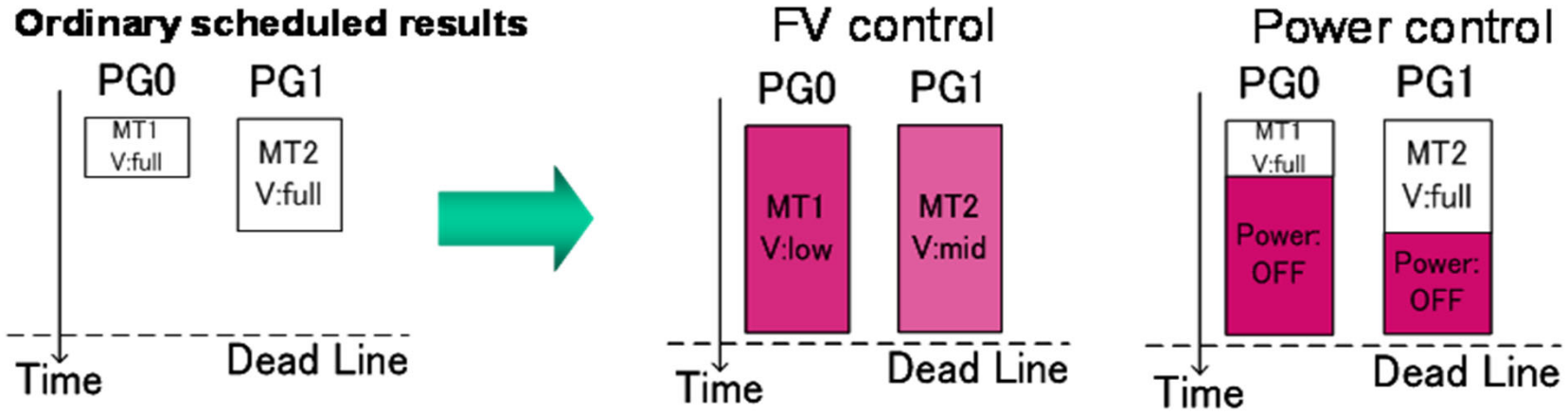
Power Reduction by Power Supply, Clock Frequency and Voltage Control by OSCAR Compiler

Frequency and Voltage (DVFS), Clock and Power gating of each cores are scheduled considering the task schedule since the dynamic power proportional to the cube of F (F^3) and the leakage power (the static power) can be reduced by the power gating (power off).

- **Shortest execution time mode**



- **Realtime processing mode with dead line constraints**



In this Fig. Frequency Full, Mid, Low

Power OFF: Power Gating

An Example of Machine Parameters for the Power Saving Scheme

- **Functions of the multiprocessor**
  - **Frequency of each proc. is changed to several levels**
  - **Voltage is changed together with frequency**
  - **Each proc. can be powered on/off**

state	FULL	MID	LOW	OFF
frequency	1	1 / 2	1 / 4	0
voltage	1	0.87	0.71	0
dynamic energy	1	3 / 4	1 / 2	0
static power	1	1	1	0

- **State transition overhead**

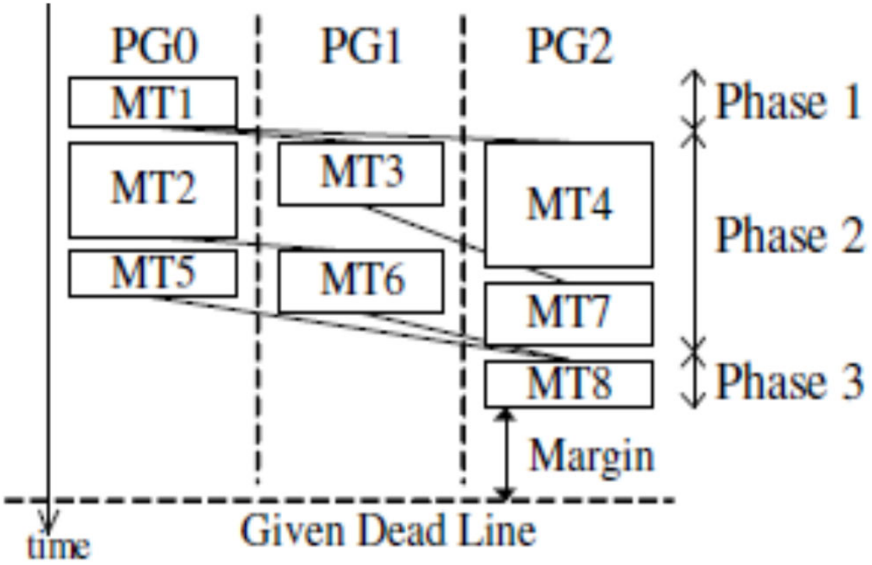
state	FULL	MID	LOW	OFF
FULL	0	40k	40k	80k
MID	40k	0	40k	80k
LOW	40k	40k	0	80k
OFF	80k	80k	80k	0

delay time [u.t.]

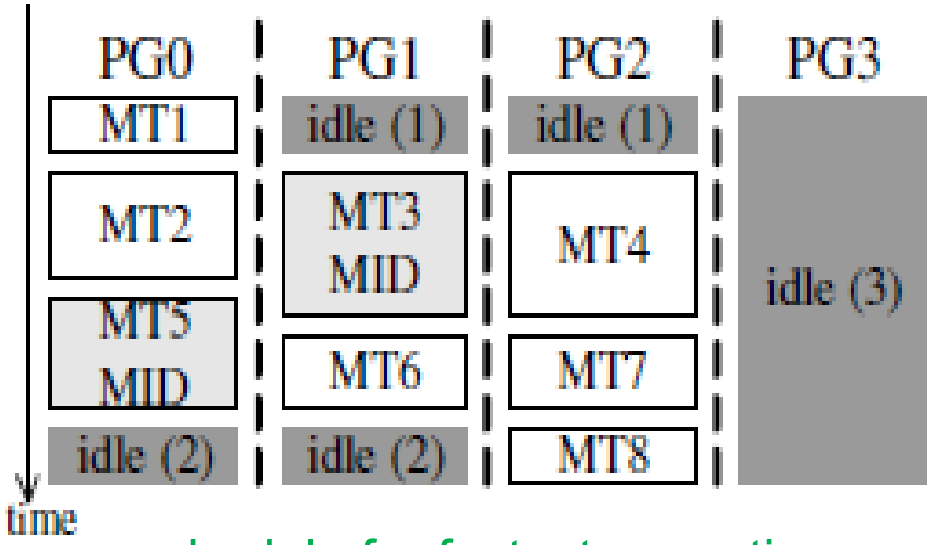
state	FULL	MID	LOW	OFF
FULL	0	20	20	40
MID	20	0	20	40
LOW	20	20	0	40
OFF	40	40	40	0

energy overhead [μJ]

Power Reduction Scheduling



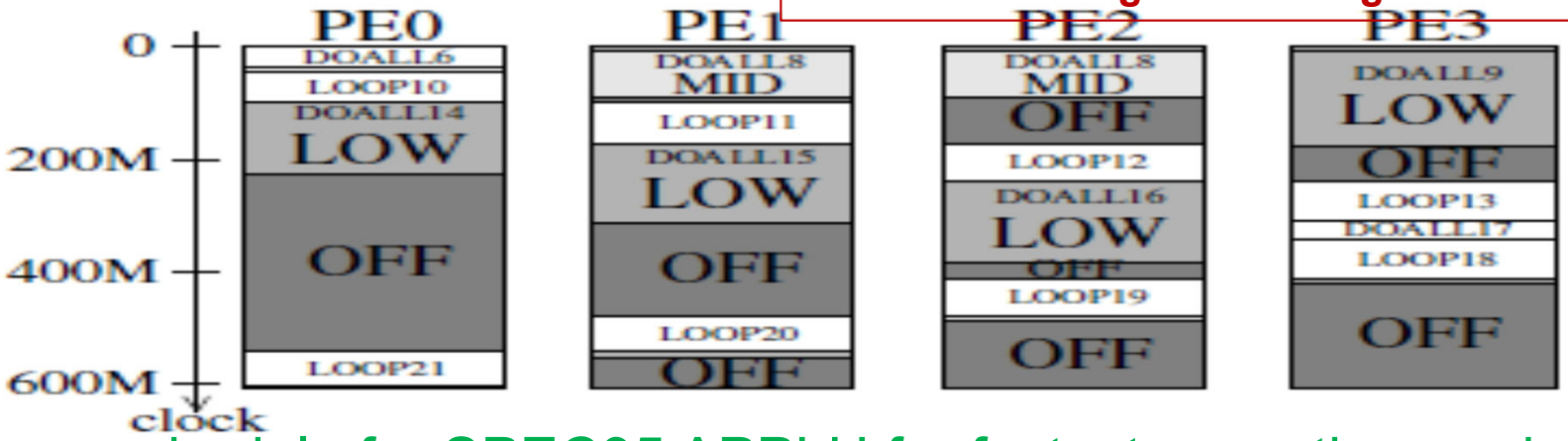
A macrotask graph assigned to 3 cores



A power schedule for fastest execution mode

Realtime scheduling mode
MTs 1,4,7,8 are on Critical Path (CP)

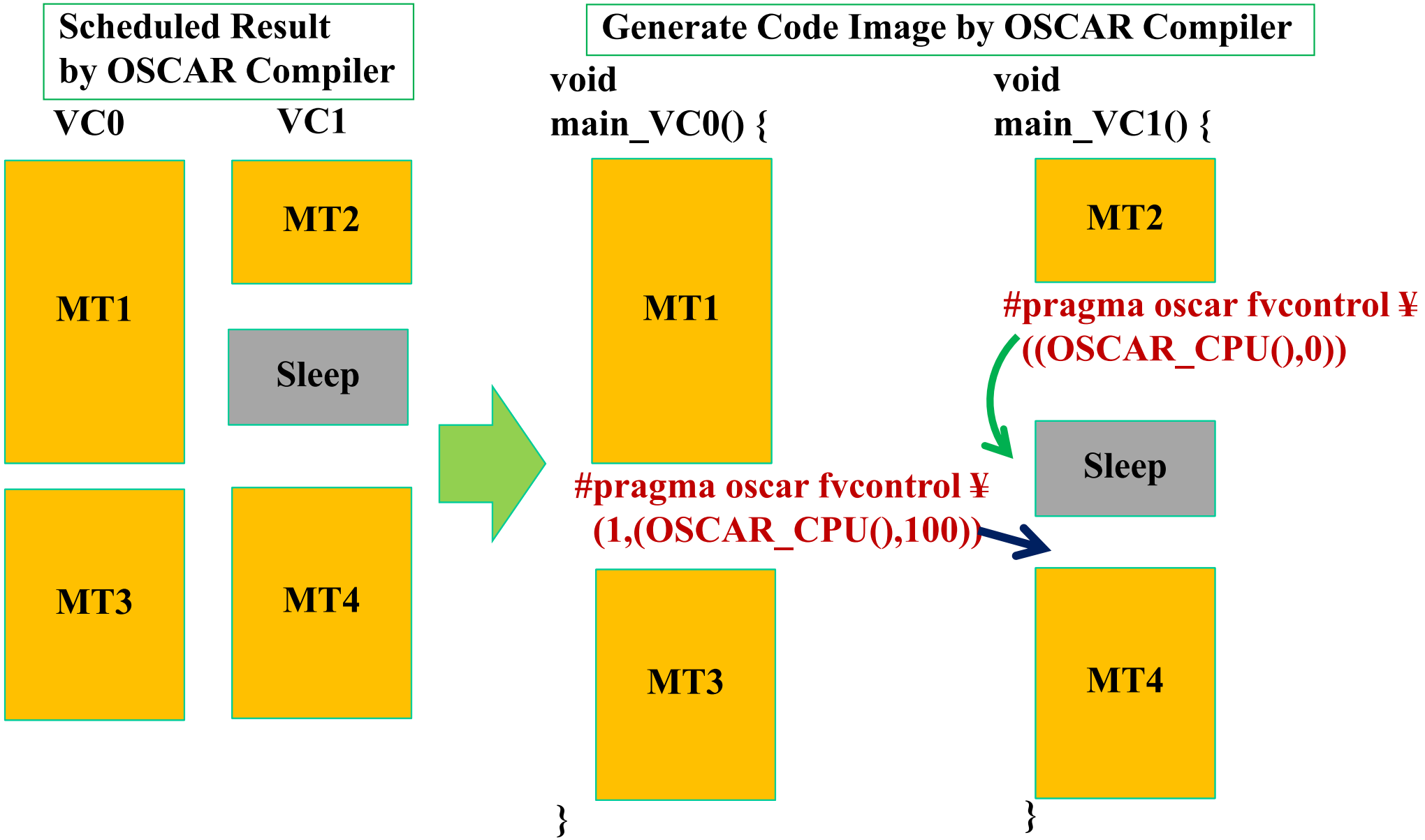
1) **Reduce frequencies (Fs) of MTs on CP considering dead line.**
2) **Reduce Fs of MTs not on CP. Idle: Clock or Power Gating considering overheads.**



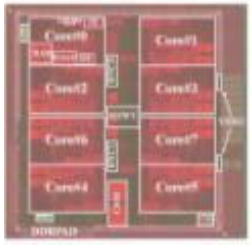
A power schedule for SPEC95 APPLU for fastest execution mode

Doall6, Loop 10,11,12,13, Doall 17, Loop 18,19,20, 21 are on CP

Low-Power Optimization with OSCAR API

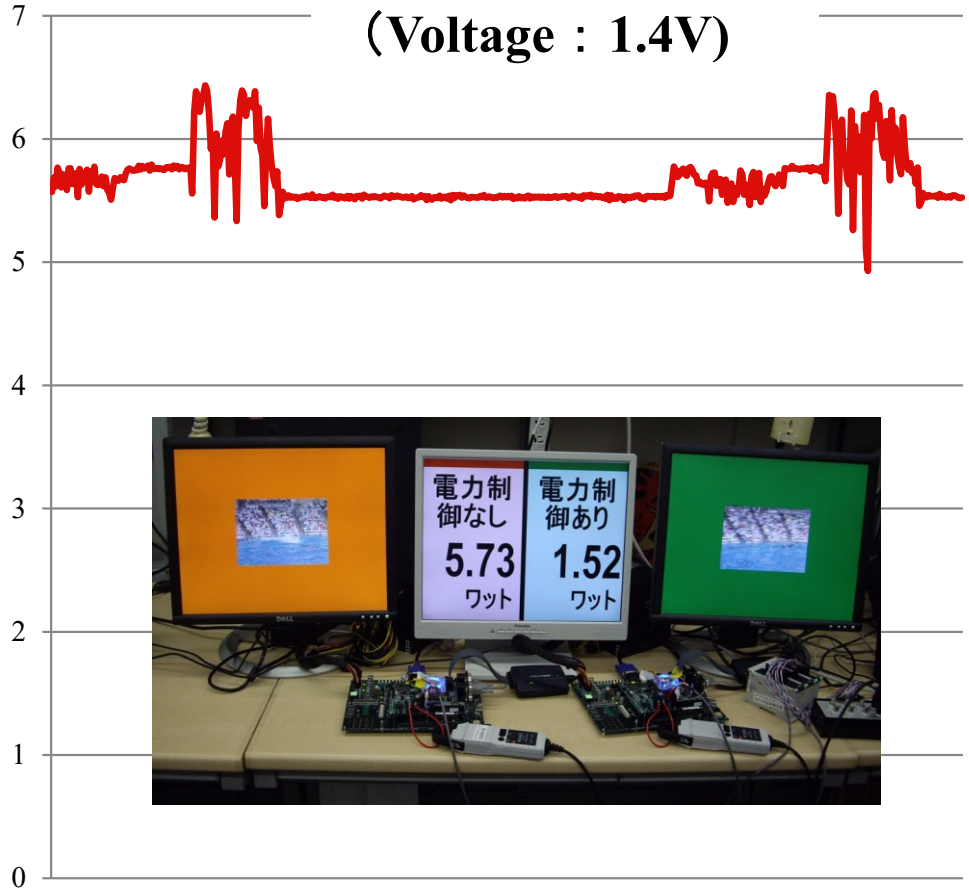


Power Reduction of MPEG2 Decoding to 1/4 on 8 Core Homogeneous Multicore RP-2 by OSCAR Parallelizing Compiler

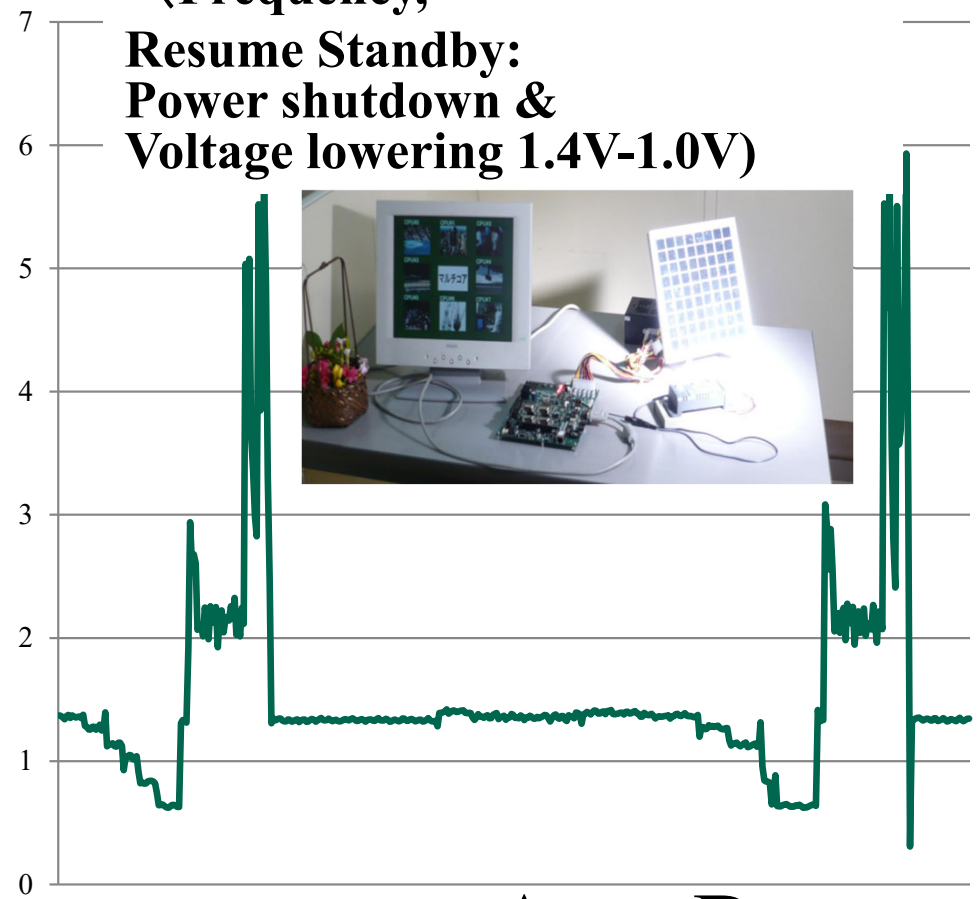


MPEG2 Decoding with 8 CPU cores

Without Power Control （Voltage： 1.4V)



With Power Control （Frequency, Resume Standby: Power shutdown & Voltage lowering 1.4V-1.0V)



Avg. Power 5.73 [W]

73.5% Power Reduction

Avg. Power 1.52 [W]

Demo of NEDO Green Multicore Processor for Real Time Consumer Electronics at Council of Science and Engineering Policy on April 10, 2008

http://www8.cao.go.jp/cstp/gaiyo/honkaigi/74index.html

第74回総合科学技術会議【平成20年4月10日】

第74回総合科学技術会議の様子(1)

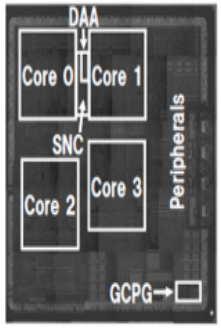
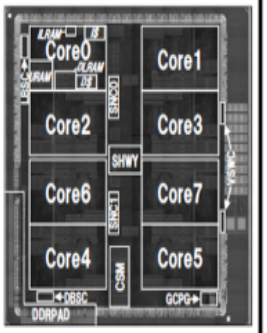
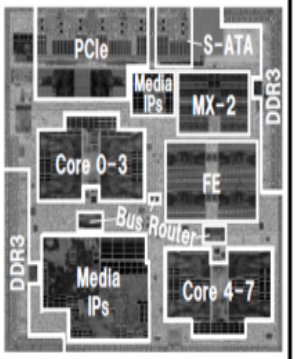
第74回総合科学技術会議の様子(2)

第74回総合科学技術会議の様子(3)

第74回総合科学技術会議の様子(4)

Codesign of Compiler and Multiprocessor Architecture since 1985

4 core multicore RP1 (2007), 8 core multicore RP2 (2008) and 15 core Heterogeneous multicore RPX (2010) developed in NEDO Projects with Hitachi and Renesas

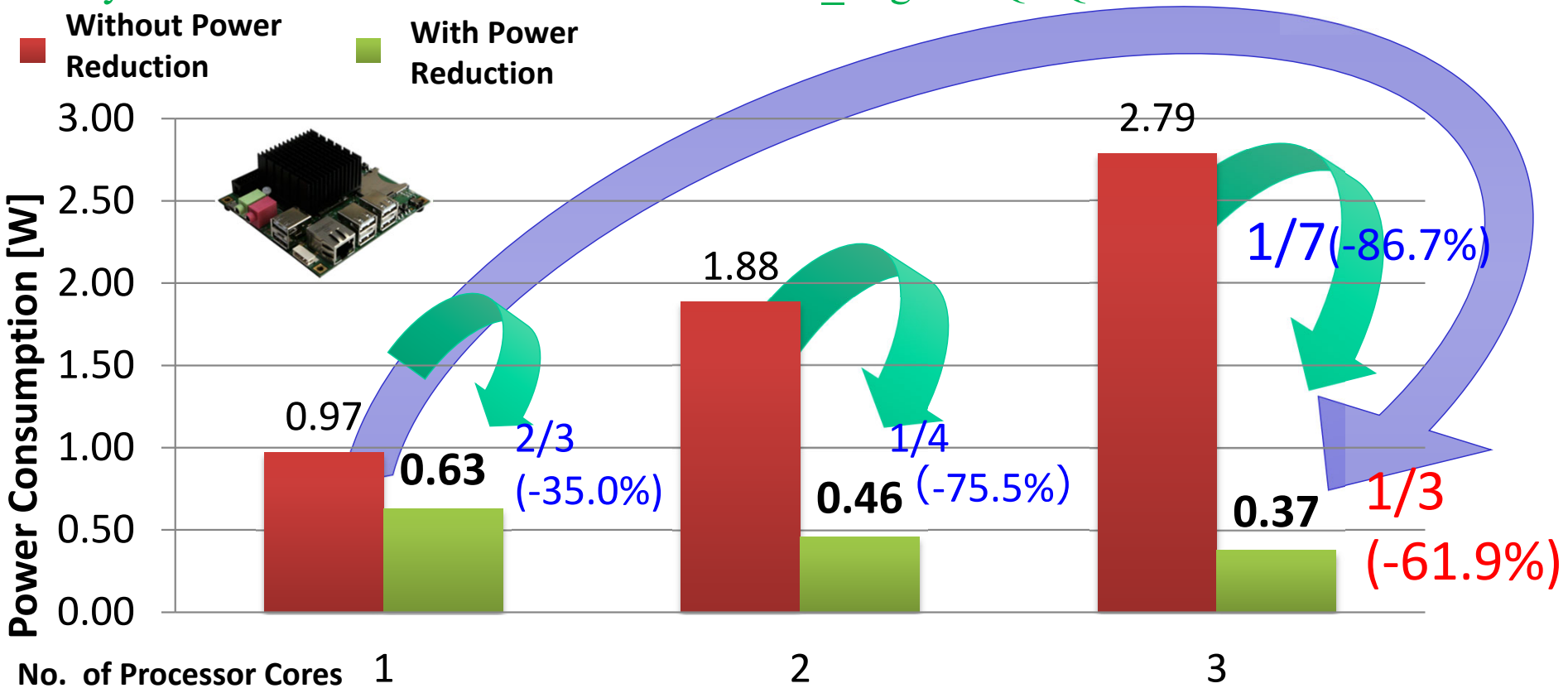
RP-1 (ISSCC2007 #5.3)	*RP-2* (ISSCC2008 #4.5)	*RP-X* (ISSCC2010 #5.3)
		
90nm, 8-layer, triple-Vth, CMOS	90nm, 8-layer, triple-Vth, CMOS	45nm, 8-layer, triple-Vth, CMOS
97.6 mm^2 (9.88 x 9.88 mm)	104.8 mm^2 (10.61 x 9.88 mm)	153.8 mm^2 (12.4 x 12.4 mm)
1.0V (internal), 1.8/3.3V (I/O)	1.0-1.4V (internal), 1.8/3.3V (I/O)	1.0-1.2V (internal), 1.2-3.3V (I/O)
600MHz ,4.32 GIPS,16.8 GFLOPS	600MHz , 8.64 GIPS, 33.6 GFLOPS	648MHz, 13.7GIPS, 115GOPS, 36.2GFLOPS
11.4 GOPS/W(32b換算)	18.3 GOPS/W(32b換算)	37.3 GOPS/W(32b換算)

Prime Minister FUKUDA is touching our multicore chip during execution.

Automatic Power Reduction for MPEG2 Decode on Android Multicore

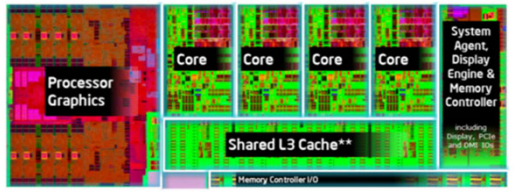
ODROID X2 ARM Cortex-A9 4 cores

http://www.youtube.com/channel/UCS43lNYEIkC8i_KIgFZYQBQ



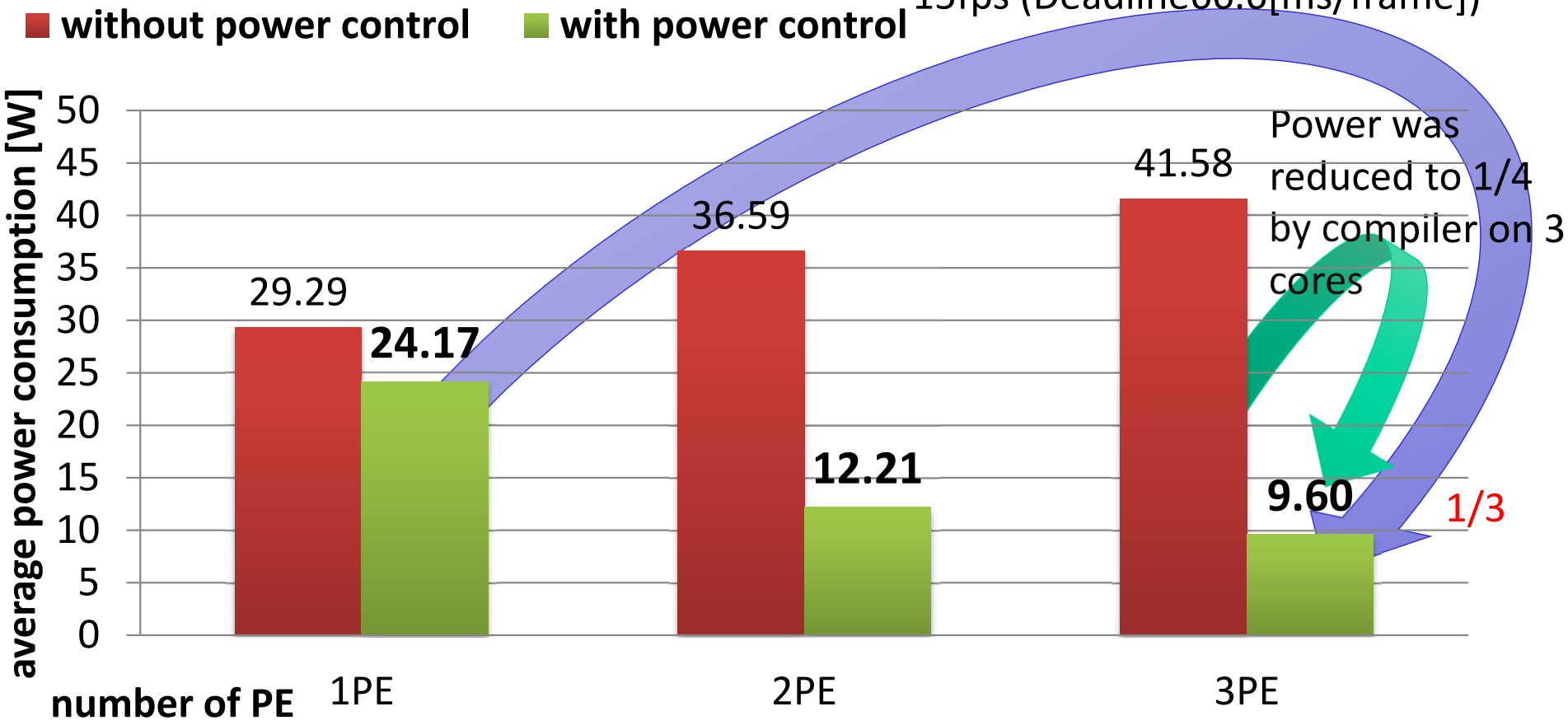
- On 3 cores, Automatic Power Reduction control successfully reduced power to 1/7 against without Power Reduction control.
- 3 cores with the compiler power reduction control reduced power to 1/3 against ordinary 1 core execution.

Power Reduction on Intel Haswell for Real-time Optical Flow



Intel CPU Core i7 4770K

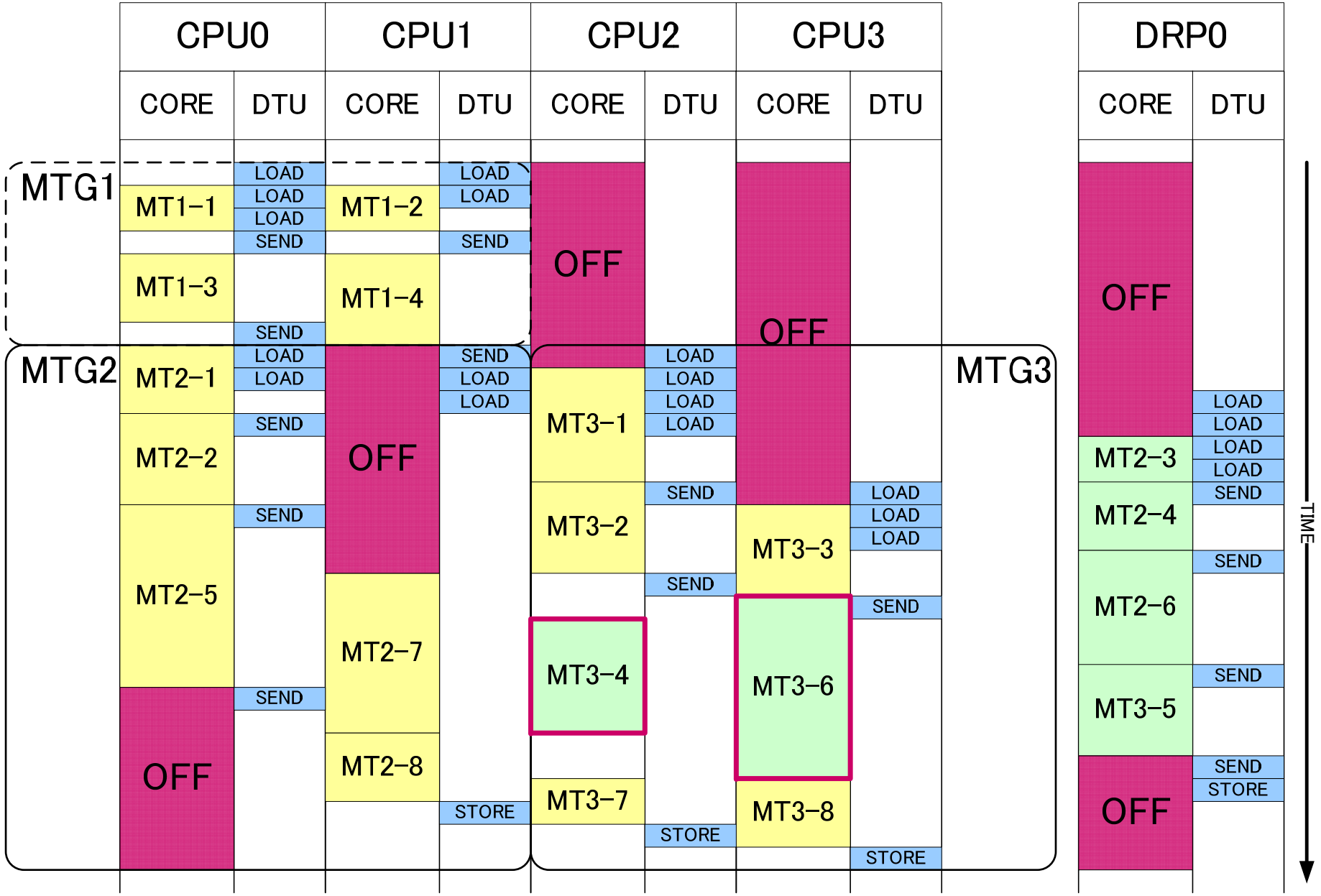
For HD 720p(1280x720) moving pictures
15fps (Deadline66.6[ms/frame])



Power was reduced to 1/4 (9.6W) by the compiler power optimization on the same 3 cores (41.6W).

Power with 3 core was reduced to 1/3 (9.6W) against 1 core (29.3W) .

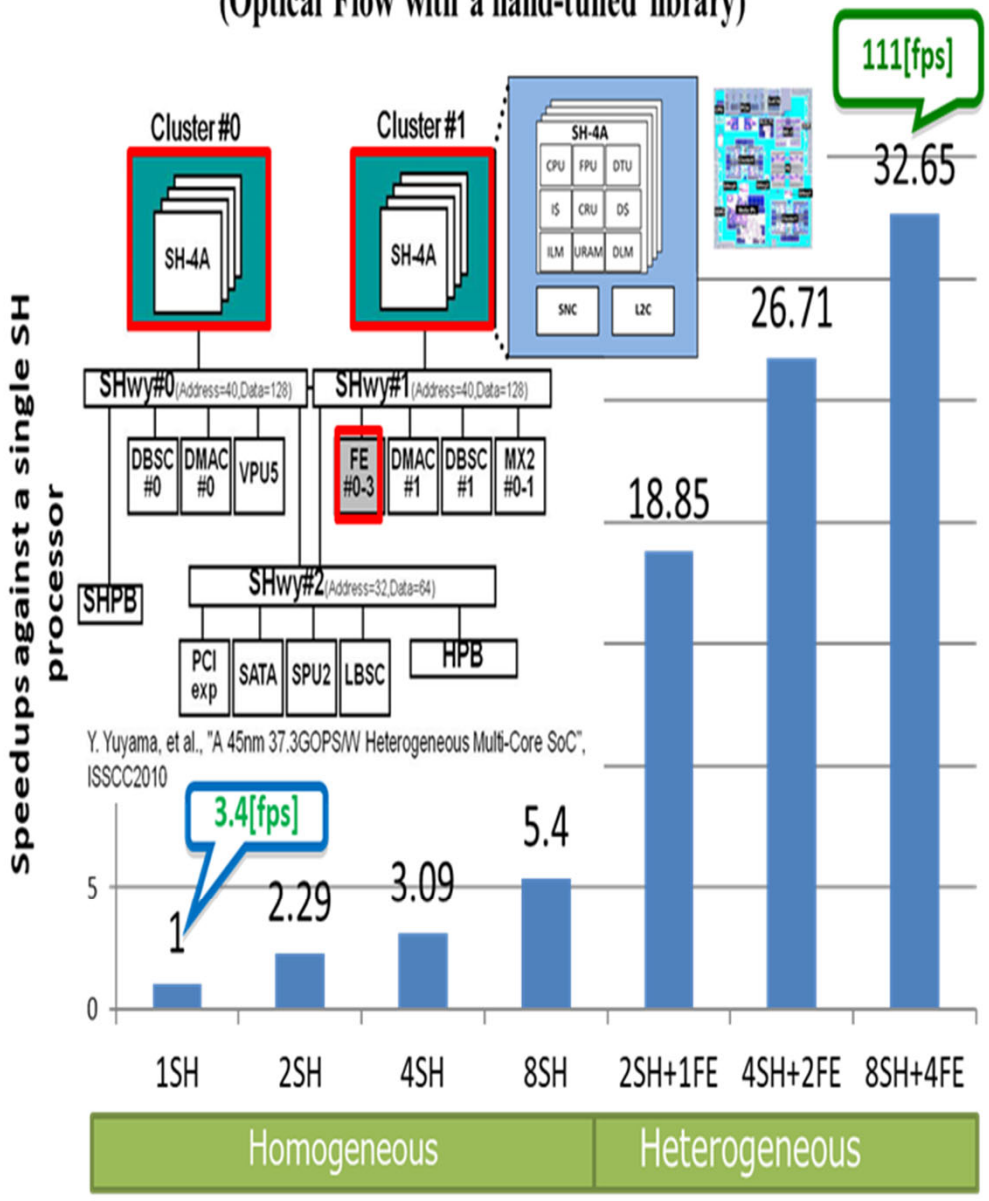
An Image of Static Schedule for Heterogeneous Multi-core with Data Transfer Overlapping and Power Control



Speedups & Power Reduction on RP-X Heterogeneous Multicore with 8 CPUs and 4 DRPs

33 Times Speedup Using OSCAR Compiler and API on Renesas RP-X with 8 CPUs & 4 DRP Accelerators

(Optical Flow with a hand-tuned library)



Power Reduction in a real-time execution controlled by OSCAR Compiler and OSCAR API on RP-X

(Optical Flow with a hand-tuned library)

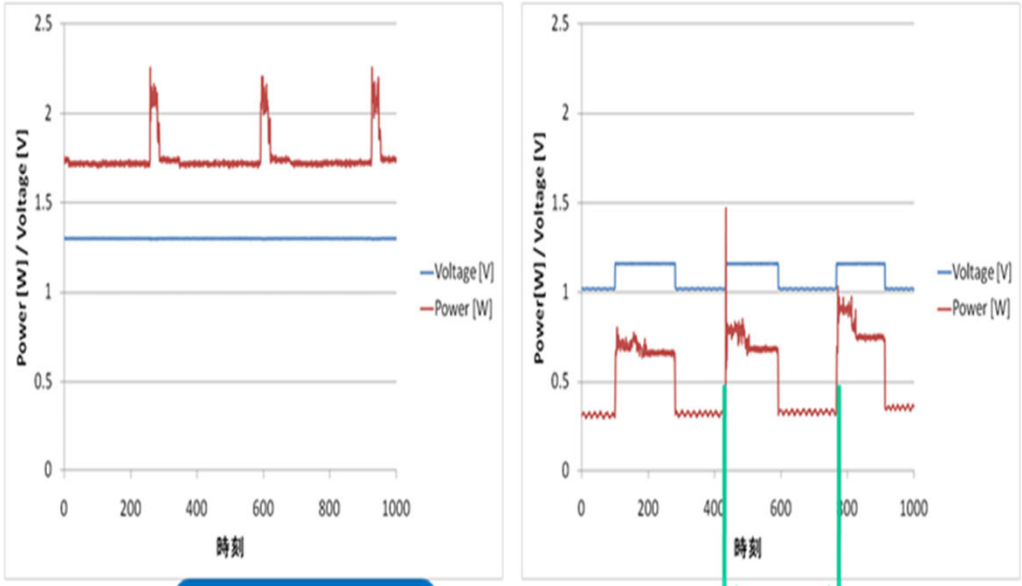
Without Power Reduction

With Power Reduction by OSCAR Compiler

70% of power reduction

Average:1.76[W]

Average:0.54[W]



1cycle : 33[ms]
→30[fps]

From Smartphones, Smart Cars to HPC use Multicores & Accelerators

Cores: ARM, IBM, Renesas, Infineon, SPARC, RISC V, **Accelerators:** NVIDIA, Intel, AMD, etc.

64-bit
iPhone 13
2021

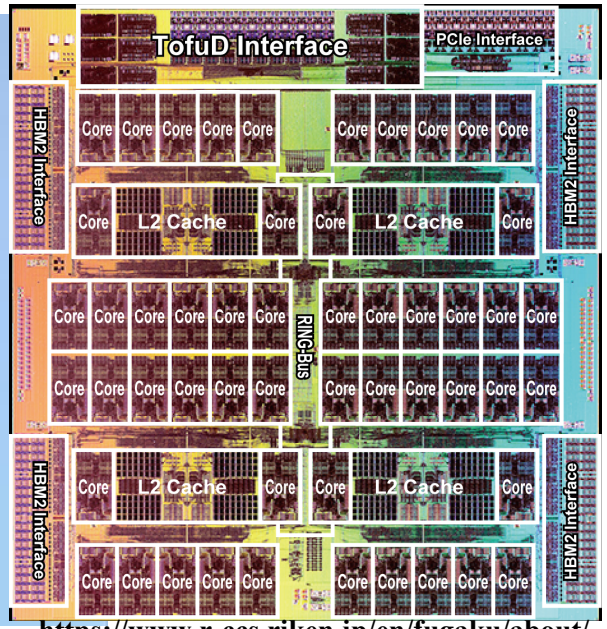
Launched September 14, 2021
Designed by Apple Inc.
Common manufacturer(s) : TSMC
Max. CPU clock rate to 3.23 GHz in iPhone 13 Pro
Technology node: 5 nm
6 Cores: 2 "Avalanche"High Perf. & 4 "Blizzard" Low Power
Instruction set: ARM A64, Transistors: 15 billion (15億個)
GPU(s): Apple-designed 5 core GPU in iPhone 13

https://www.apple.com/jp/shop/buy-iphone

https://en.wikipedia.org/wiki/Apple_A15

Riken Fugaku : Top1 June, 2020-Nov.2021

https://fugaku100kei.jp/fugaku/



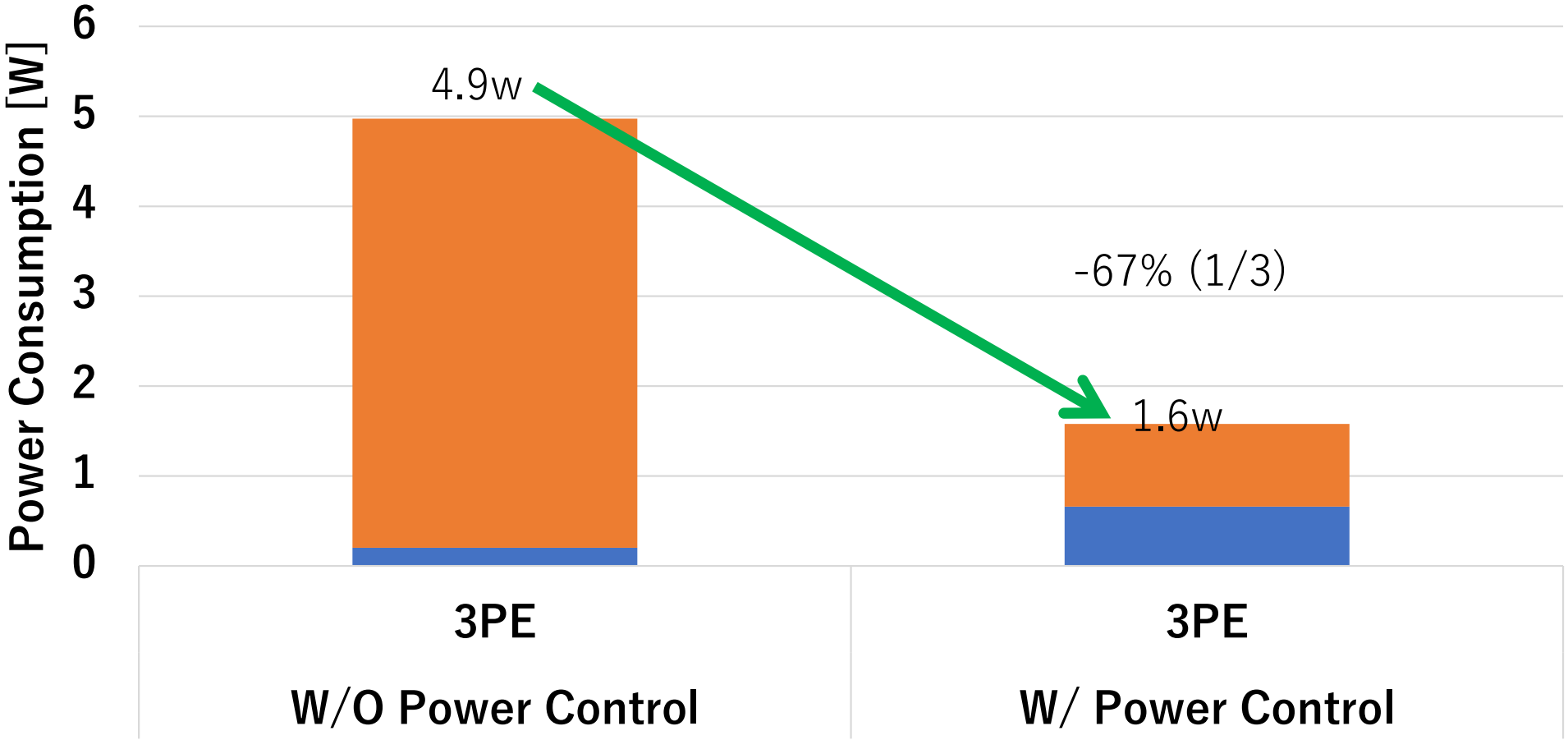
https://www.r-ccs.riken.jp/en/fugaku/about/

RIKEN Center for Computational Science,
Fujitsu（arm based processor）
Cores:7,299,072； Memory:4,866,048GB；
Processor:A64FX 48Cores, 2.2GHz
Interconnect: Tofu interconnect D
Linpack (Rmax)415,530 TFlop/s;
Theoretical Peak (Rpeak): 513PFLOPS
HPCG [TFlop/s]13,366.4; Power: 28.3MW

48cores/chip, 2.2GHz, 7 nm FinFET,
約7百30万コア, 28MW
理論最高性能:51京回浮動小数点演算/秒,
2020年6月時点

https://japanese.engadget.com/arm-super-computer-fugaku-top-500-034015910.html

Automatic Power Reduction of OpenCV Face Detection on big.LITTLE ARM Processor



- ODROID-XU3
  - Samsung Exynos 5422 Processor
    - 4x Cortex-A15 2.0GHz, 4x Cortex-A7 1.4GHz big.LITTLE Architecture
    - 2GB LPDDR3 RAM
      Frequency can be changed by each cluster unit

IEEE COMPUTER SOCIETY

https://www.computer.org/product/education/multi-core-video-lectures-bundle

Created by Multicore STC Chair Hironori Kasahara

Multi-Core Lecture Series consists of 11 one-hour lectures by some of the world's leading researchers in the field. This series is not a course and it consists of the presentation for those who are in the research field. This is more intended for research information sharing than educational training. Topics that are covered during these lectures are listed below. This series also includes an hour discussion of the lecturers.

Video Presentations:

- Automatic Parallelization by David Padua
- Autoparallelization for GPUs by Wen-Mei Hwu
- Dependences and Dependence Analysis by Utpal Banerjee
- Dynamic Parallelization by Rudolf Eigenmann
- Instruction Level Parallelization by Alexandru Nicolau
- Multigrain Parallelization and Power Reduction by Hironori Kasahara
- The Polyhedral Model by Paul Feautrier
- Vector Computation by David Kuck
- Vectorization by P. Sadayappan
- Vectorization/Parallelization in the IBM Compiler by Yaoqing Gao
- Vectorization/Parallelization in the Intel Compiler by Peng Tu
- Roundtable Discussion by all presenters

Dependences and Dependence Analysis by Utpal Banerjee Utpal Banerjee's research interests in computer science are in the general area of parallel processing and he has published four books on loop transformations and dependence analysis, with a fifth one on instruction level parallelism on the way.

Multigrain Parallelization and Power Reduction by Hironori Kasahara. Professor Kasahara has been researching on OSCAR Automatic Parallelizing and Power Reducing Compiler and OSCAR Multicore architecture for more than 30 years, and led four Japanese national projects on parallelizing compilers, multicores, and green computing.